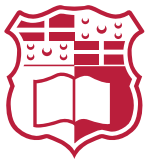
Learning to Select Actions for Complex Hand Tasks: A Comparative Study of Visuomotor Policies and Latent World Models

Graham Pellegrini

Supervisor: Prof. Alexiei Dingli

[PLACEHOLDER: Submission Date]

Submitted in partial fulfilment of the requirements for the degree of Master of Science in Artificial Intelligence.



L-Università ta' Malta
Faculty of Information & Communication Technology

Plagiarism Declaration

I, the undersigned, hereby declare that the work contained in this dissertation is my own original work and that I have not previously in its entirety or in part submitted it at any university for a degree.

I declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person, except where due acknowledgement is made in the text.

I declare that the use of artificial intelligence tools in the preparation of this dissertation was limited to writing assistance, code suggestions, and literature search support. All research design, experimental decisions, implementation choices, result interpretation, and academic claims are my own.

Signature: _____

Name: Graham Pellegrini

Date: _____

Abstract

This dissertation investigates whether a learned world model can improve the quality of action predictions made by a robot imitation policy on a bimanual dexterous manipulation task. The central question is whether coupling a policy that learns to imitate expert demonstrations with a model that predicts the likely future consequences of candidate actions leads to trajectory predictions that more faithfully reproduce the demonstrated motion.

The implemented system, WACT (World Model Augmented Action Chunking Transformer), operates on a large collection of egocentric hand-manipulation demonstrations drawn from sixty-three task categories. It compares three candidate selection strategies that share the same underlying imitation policy: a baseline that applies the policy directly, a hybrid strategy in which the policy proposes candidate motions and a world model evaluates them, and a world-model-driven strategy in which the world model selects the best candidate independently of the policy's own preference. Two world model families are evaluated within this structure. The first draws on general-purpose visual representations learned from a large-scale video corpus before seeing any manipulation data. The second learns entirely from the task demonstrations themselves, without any external pretraining.

The main evaluation is conducted on a held-out test set of 7,607 evaluation windows spanning sixty-three task categories. The headline finding is that world model guidance substantially improves how closely predicted wrist trajectories match the demonstrated motion. Under world-model-driven selection with the pretrained family, mean translation error is reduced by 77 per cent compared to the policy-alone baseline, and this strategy outperforms the baseline in 91.5 per cent of head-to-head window comparisons. The hybrid strategy also improves substantially, achieving 86.6 per cent of comparisons, and shows greater consistency across individual test windows. Analysis of path shape reveals that the improvement is concentrated in spatial correctness and trajectory geometry rather than in the speed or timing of the predicted motion: path straightness improves to an 89.4 per cent win rate and trajectory smoothing to 93.1 per cent, while velocity and acceleration profiles remain near chance.

The task-data-trained world model achieves a 71.7 per cent win rate against the baseline despite learning entirely from scratch, confirming that world model guidance is not contingent on large-scale external pretraining, though the pretrained family is substantially stronger. Expanding the training dataset from thirty-nine to sixty-three task categories did not further improve the pretrained world model, suggesting it is near a capacity ceiling under the current policy architecture.

The dissertation also documents the benchmark development required to reach

defensible conclusions. This includes the correction of a data coverage defect that had limited one model family to 2.1 per cent of the intended test surface, and the identification and correction of a selection logic error that had caused two of the three branches to produce identical outputs. Both corrections were necessary before the final comparisons could be treated as scientifically meaningful. All conclusions are bounded to offline prediction quality on recorded demonstrations and do not constitute claims about deployed task success.

Acknowledgements

Placeholder for acknowledgements.

Contents

List of Figures

List of Tables

List of Abbreviations

ACT Action Chunking Transformer.

ALOHA A Low-cost Open-source Hardware System for Bimanual Teleoperation.

BC Behaviour Cloning.

CDiT Conditional Diffusion Transformer.

CEMP Cross-Entropy Method with Policy.

CNN Convolutional Neural Network.

CVAE Conditional Variational Autoencoder.

DDIM Denoising Diffusion Implicit Models.

DDPM Denoising Diffusion Probabilistic Models.

DexWM Dexterous Manipulation World Model.

DiT Diffusion Transformer.

DP Diffusion Policy.

GATO Generalist Agent.

HDF5 Hierarchical Data Format 5.

I-JEPA Image Joint Embedding Predictive Architecture.

JEPA Joint Embedding Predictive Architecture.

LeWM LeWorldModel.

RMSE Root Mean Square Error.

SSL Self-Supervised Learning.

V-JEPA2 Video Joint Embedding Predictive Architecture 2.

V-JEPA2-AC Action-Conditioned Video Joint Embedding Predictive Architecture 2.

VAE Variational Autoencoder.

ViT Vision Transformer.

VLM Vision Language Model.

List of Notations

I Identity matrix; gives the isotropic unit Gaussian covariance.

$\mathcal{N}(0, I)$ Standard isotropic Gaussian prior.

ε Reparameterisation noise, $\varepsilon \sim \mathcal{N}(0, I)$.

μ, σ Encoder-predicted mean and standard deviation.

z CVAE latent variable sampled from the prior $\mathcal{N}(0, I)$.

$\bar{\alpha}_t$ Cumulative noise-schedule product at diffusion timestep t.

ε_θ Noise-prediction network parameterised by θ.

t_{prev} Next step index in a subsampled inference schedule.

$\hat{x}_0$ Predicted clean sample in a DDIM reverse step.

x_T Noise sample at the diffusion schedule endpoint T.

B_{global} Effective global batch size; equals $B \times G \times A$ where B is the per-device batch size, G the gradient-accumulation steps, and A the number of accelerator devices.

E Effective training exposure (total items consumed).

N Number of candidates in an evaluation pool.

S Number of optimisation steps.

1 Introduction

Imitation learning studies how computational systems can acquire behaviour from demonstrations rather than from manually specified rules. In manipulation settings, this is attractive because useful behaviour often depends on subtle motion patterns that are difficult to encode by hand. Behaviour cloning provides one direct route by learning a mapping from observations to demonstrated actions, while inverse reinforcement learning approaches the same problem by trying to infer the objectives that make a demonstration sensible [1, 2]. Both perspectives frame imitation as a way to learn from observed behaviour, but they do not by themselves resolve how a system should judge whether a predicted future motion is physically plausible.

This issue becomes sharper in dexterous manipulation. Human hand motion is temporally structured, object dependent, and sensitive to small changes in pose. This follows from the timing of natural prehension, the visuomotor transformation required for grasping objects, and the postural synergies used to shape the hand for tools and objects [3–5]. In imitation settings, this matters because small prediction errors can compound across a future sequence, so local agreement with a demonstration does not guarantee sequence-level alignment [6]. This thesis is therefore concerned with prediction and evaluation of future hand motion, rather than deployed control. The work remains relevant to robotic control because robotic manipulation systems ultimately require reliable motion selection, but the evidence in this dissertation is restricted to an offline computational benchmark.

Visuomotor policies are a common way to connect visual observations with future motion. In this thesis, the focused policy family is the ACT, which predicts temporally extended action chunks rather than isolated next-step actions [7]. This chunked formulation is useful because dexterous motion unfolds over short sequences, not only through single-frame decisions. However, a policy that proposes a future chunk still requires a criterion for choosing between candidate futures. This creates the opening for world models.

World models learn predictive representations of how a state may evolve. The initial role considered in this thesis was conservative: a world model could evaluate candidate action chunks proposed by ACT and help select motions whose predicted consequences looked more consistent with the demonstrated future. As the implementation developed, the question broadened. The benchmark also examines whether a world model can take greater authority over candidate selection, moving from support for a policy toward a stronger world model driven selection role. This progression from assistance to selection motivates the comparative structure of the thesis.

This dissertation studies that structure through WACT, which is used here as a comparative benchmark framework for evaluating whether world model support changes the quality of future hand-motion prediction. The framework compares an ACT-only baseline with world model selected variants that use selected and ported world-model families, including V-JEPA2-AC and LeWM. The families themselves are not claimed as original contributions of this thesis. The contribution lies in the controlled comparison, the implemented evaluation surface, and the analysis of how these model families behave when attached to the same policy setting.

1.1 Motivation

The motivation for this work sits at the intersection of robotics and machine learning. Modern manipulation research increasingly depends on learned models that can process visual observations, predict future motion, and adapt to complex physical interaction. At the same time, the pace of open-source machine learning development can make it difficult to separate robust scientific progress from promising but weakly benchmarked demonstrations. This concern is especially important for industrially relevant settings, where unreliable claims about physical reasoning or manipulation capability can lead to poor engineering decisions.

The work was also shaped by the RSO I activity and the University of Malta M.AI.ESTRO project, where the convergence of robotics, machine learning, and Industry 4.0 raised a practical need for clearer evaluation of emerging model families. In that setting, world models are particularly relevant because they promise a form of predictive checking that could eventually support safer or more reliable action selection. This dissertation does not claim to deliver such a deployment system. It instead treats the perceived need from that work as motivation for a narrower research question: whether world models measurably improve dexterous manipulation prediction under a controlled offline benchmark.

1.2 Research Question and Scope

The central research question is:

> *Do world models improve dexterous manipulation prediction when compared with, and integrated alongside, an ACT visuomotor policy under a controlled offline benchmark?*

This wording is a rescoping of the original proposal question. The proposal asked whether integrating a latent world model into an imitation learning policy would

improve physical grounding and robustness, measured through latent agreement error and task success under perturbations. The final implementation does not evaluate deployed task success or perturbation robustness, and latent mean-squared error is used only as a training diagnostic for the world-model transition head rather than as the main thesis result. The finished thesis therefore frames the question around offline prediction quality: whether world model support selects future hand-motion chunks that are closer to demonstrated motion under a shared benchmark.

The benchmark focuses on future bimanual hand motion and transform state. In implementation terms, the main prediction surface is built around future left-hand and right-hand transform chunks, but the detailed representation is defined later in Chapter 4 and Chapter 5. The Introduction therefore uses the more readable language of future hand motion and transform state, while later chapters specify the exact data contract, branch definitions, and metrics.

The scope is deliberately bounded. The thesis does not test closed-loop robotic deployment, real-world task completion, force interaction, contact stability, or general machine common sense. It also does not claim that any one world model family is universally superior. Instead, it asks whether selected world-model families improve an offline prediction benchmark when compared under a shared evaluation surface. The selected policy architecture is ACT, and the selected world-model families are V-JEPA2-AC and LeWM. These choices make the question concrete enough to evaluate while keeping the framework open to future model families and additional evaluation surfaces.

This scope also explains the structure of the comparison. The benchmark is organised around three branches:

- **ACT-only branch:** the policy baseline, where ACT provides the selected future motion without world model support.
- **Hybrid branch:** ACT proposes candidate chunks, and the world model evaluates or re-scores those candidates before selection.
- **World model driven branch:** no policy preference is used for selection. The selected future motion is chosen from the world model's own assessment of predicted future-state consistency.

The purpose of this structure is to separate the value of the world model from the value of the underlying policy as far as possible within an offline benchmark.

1.3 Contributions

This thesis contributes a comparative benchmark framework for studying world model support in dexterous manipulation prediction. The framework evaluates a fixed ACT policy setting against world model selected alternatives, allowing the effect of world model support to be examined without changing the core policy family. This provides a controlled basis for asking whether predictive future-state models improve selected hand-motion chunks under the same offline evaluation conditions.

The thesis also contributes an implemented comparison between two selected world-model families, V-JEPA2-AC and LeWM. These families are used as benchmark representatives rather than presented as newly invented models. Their inclusion allows the dissertation to compare not only different levels of world model authority, but also different world-model strategies inside the same overall evaluation logic.

Finally, the thesis contributes a bounded empirical analysis of world model support for future bimanual hand-motion prediction. The analysis is limited to the selected dataset, branch definitions, prediction targets, and metrics. This boundedness is intentional. It keeps the claims scientifically defensible while leaving the framework open to later world models, alternative policy families, and evaluation designs that move closer to deployment.

1.4 Dissertation Structure

——To be updated at the end ————

The remainder of the dissertation is organised as follows:

- Chapter 2 introduces the technical background required to follow the dissertation, covering transformers, self-supervised learning, variational autoencoders, rigid body pose, visuomotor policies, and world models at a foundational level.
- Chapter 3 positions the work within the relevant literature on imitation learning, visuomotor policies, world models, and offline evaluation.
- Chapter 4 defines the benchmark design, scope, branch structure, and comparison logic.
- Chapter 5 describes the implemented system and the main engineering decisions needed to produce the benchmark.
- Chapter 6 explains the evaluation surface, selected metrics, and reporting rules.

- Chapter 7 presents the benchmark results.
- Chapter 8 interprets the findings and states the main limitations.
- Chapter 9 closes the dissertation and identifies future work.

2 Background

This chapter assumes familiarity with the basic mechanics of a neural network and introduces six concepts central to this work: visuomotor policies, rigid-body pose in $\mathrm{SE}(3)$, latent representations, conditional VAEs, transformer attention, and self-supervised learning. These foundations support understanding of the specific architectures in Chapter 3 and underpin the candidate selection mechanism and world model evaluator design examined in Chapter 4.

2.1 Visuomotor policies

A visuomotor policy is a learned function that takes a visual observation as input and produces an action or sequence of future actions as output. In a manipulation setting, the visual observation is typically a single camera frame showing the hand and the object being manipulated, and the action describes how the hand should move next. The policy is called visuomotor because it connects vision directly to motor output, without requiring a hand-crafted intermediate representation of what the scene contains [1].

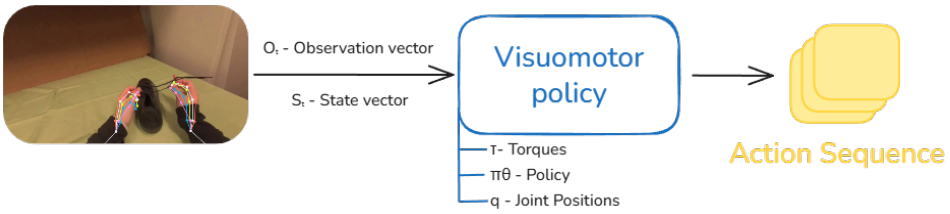


Figure 2.1 A visuomotor policy maps visual observations to motor actions. The input is a camera frame showing the hand and object, and the output is a proposed action for how the hand should move next.

Learning this mapping from data is appealing because useful hand behaviour depends on subtle visual cues that are difficult to specify by hand. The timing of a reaching movement, the grip shape adopted before contact, and the postural adjustments made in response to object geometry all depend on information visible in the camera image but difficult to encode as explicit rules [3–5]. A visuomotor policy learns these associations directly from recorded demonstrations, capturing the relationship between what the camera sees and what the hand does without requiring the programmer to describe it explicitly.

2.1.1 Behaviour cloning and compounding error

The most direct way to train a visuomotor policy from demonstrations is Behaviour Cloning (BC) [1]. Here an observed frame predicts the action taken by the expert at that moment, and the policy is trained to minimise the difference between its predicted action and the recorded expert action across all frames in the dataset [6]. The problem arises when a small prediction error causes the hand to move to a state that is slightly different from any state in the training data. This results in a compounding error: as the policy encounters increasingly unfamiliar states, its prediction errors grow larger over time.

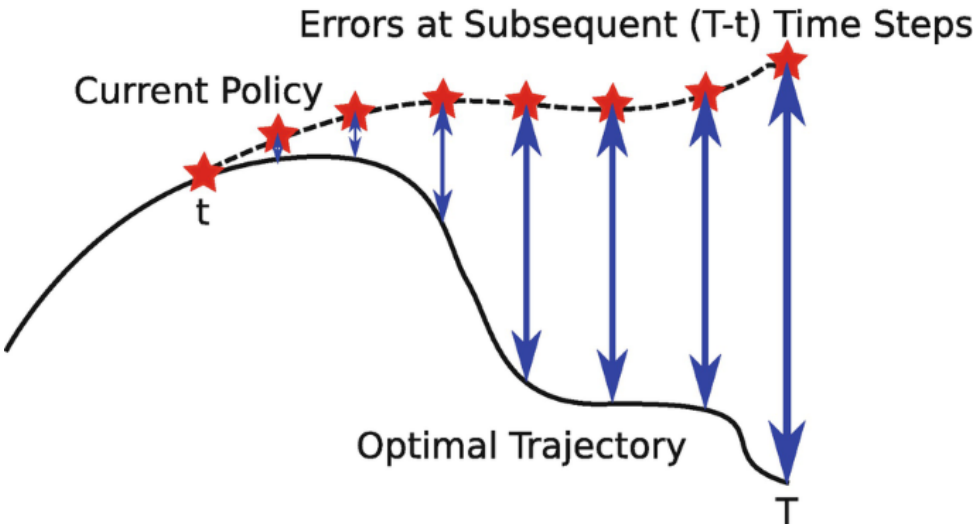


Figure 2.2 Compounding error in behaviour cloning. The policy trajectory drifts progressively further away at each step. The growing gap between the two paths illustrates how small errors can accumulate over time [8].

2.1.2 Action chunking

Action chunking addresses the compounding error problem by changing what the policy predicts. Instead of predicting a single next action at each step, the policy predicts a short sequence of future actions in one forward pass. This sequence is called an action chunk [7, 9]. This is why Figure 2.1 shows the action sequence as a stack of multiple action blocks. By committing to a chunk of k future steps, where k is a hyperparameter set per task, the policy makes one prediction decision every k frames rather than one per frame. With fewer decision points, there are fewer opportunities for errors to accumulate, and the hand stays closer to the training distribution for longer [10].

2.2 Rigid body pose and the $\mathrm{SE}(3)$ representation

To predict how a hand will move, a system needs a precise way to describe where the hand is and which way it is pointing. These two quantities together are called the pose of a rigid body. For the human wrist, pose is fully described by its position in space and its orientation relative to a fixed reference frame [11].

Translation – Position is represented as a vector of three numbers, one for each spatial dimension: how far the wrist is to the right or left, how far forward or backward, and how far up or down [3]. This vector $\mathbf{t} = [t_x,\ t_y,\ t_z]^\top$ is straightforward to predict with no special constraints on the output.

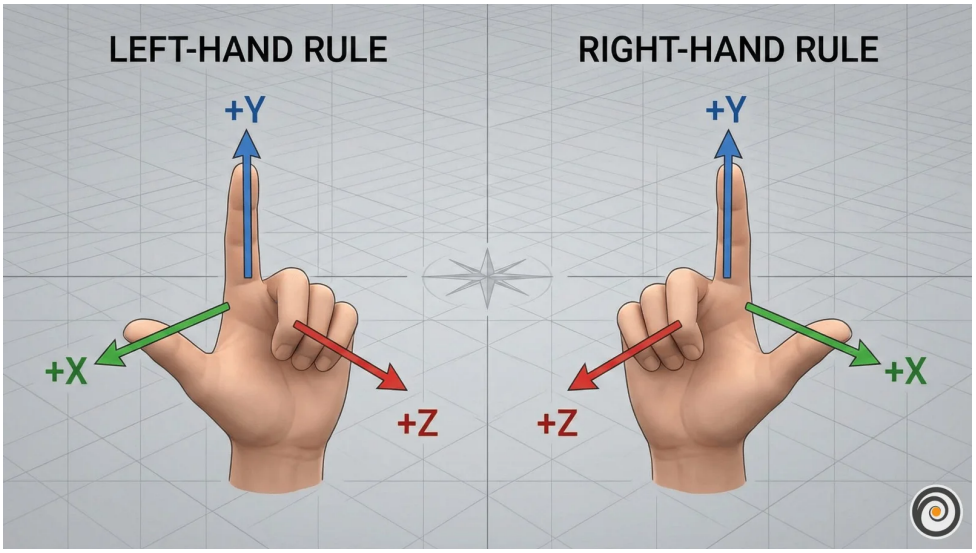


Figure 2.3 Translation for each arm can be represented by a movement along each of the three spatial axis [12].

Rotation – Orientation requires more care. A common representation uses three angles known as Euler angles, one per axis. These are intuitive but problematic for learning. At certain configurations the axes align and the system loses a degree of freedom, a singularity called gimbal lock. A subtler problem is discontinuity: a wrist rotated 359 degrees and one rotated 1 degree are almost physically identical, yet their angle values are numerically far apart. A network minimising prediction error will produce large errors near these boundaries even when the true motion is smooth [13].

A rotation matrix R avoids both issues: small physical rotations produce small numerical changes in the matrix entries, and there are no singularities.

The homogeneous transform – Translation and rotation are combined into a single 4×4 matrix, the standard representation of a pose in the special Euclidean group $\mathrm{SE}(3)$:

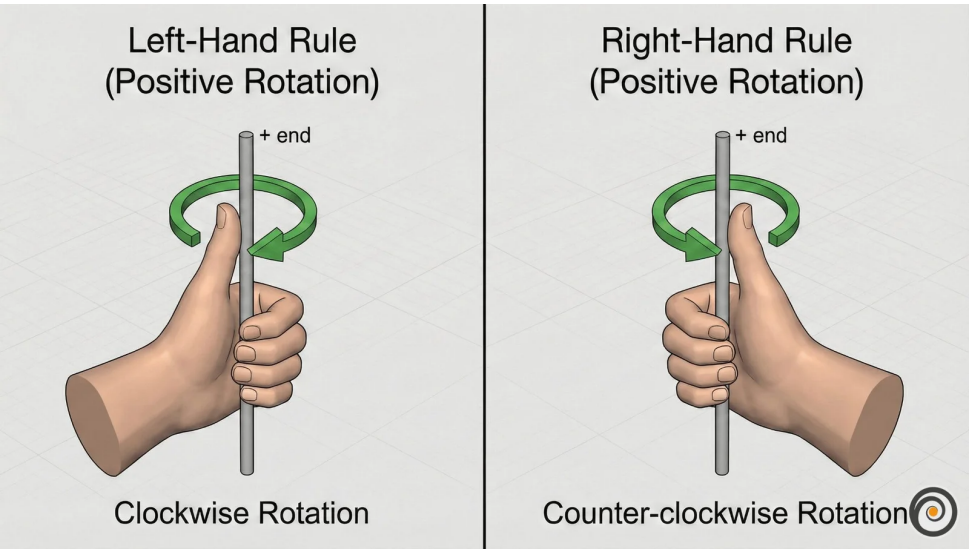


Figure 2.4 Rotation around the y-axis for each arm. The same circular movement can be imagined around the x and z axes; the total orientation of the wrist is the combined result of rotations around all three [12].

$$T = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \tag{2.1}$$

2.3 Latent representations

A latent representation is a learned internal description of an observation. An image starts as a grid of pixel values, where each pixel stores three or four channel intensities (typically red, green, and blue, with an optional transparency value). A model does not usually reason with these raw numbers directly. Instead, it transforms them through several layers into a new set of values that no longer correspond to individual pixel coordinates or colours. These new values form a compact encoding that captures patterns in the image such as shape, position, motion, and spatial structure. The collection of these encodings is often called the latent space. Figure 2.5 shows an example in which similar items cluster together in the learned representation.

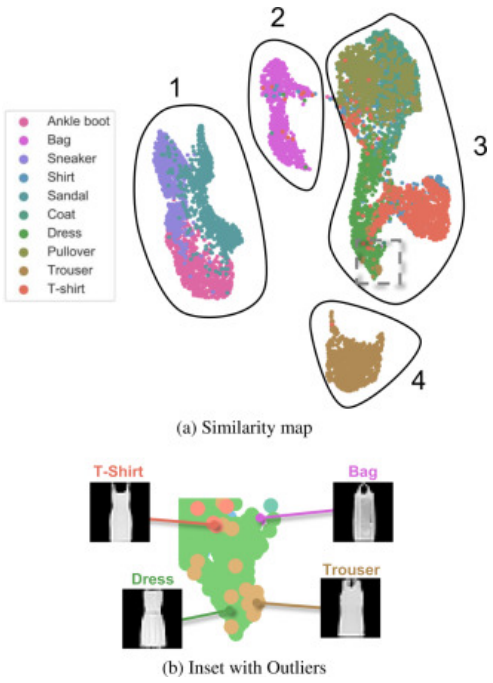


Figure 2.5 Example latent-space plot showing how structurally similar items cluster together in the learned representation. Structural similarity here refers to items that share shape, geometry, and likely physical behaviour [14].

The purpose of this transformation is to keep task-relevant structure while discarding unnecessary visual detail. For a manipulation scene, a useful latent state might encode hand position, object pose, contact progression, or motion trend without preserving every texture or lighting variation. A latent state is therefore not directly interpretable in the way an image is, but it can still preserve the information needed to reason about what happens next.

This is why latent prediction is often a better fit than pixel-level prediction for high-dimensional video observations. Predicting the next frame at the pixel level requires a model to account for every visual detail of the scene, including lighting changes, texture variation, and background movement that carry no information about task-relevant dynamics. A latent predictor instead operates on compact embeddings and concentrates predictive capacity on the structure that determines what actually happens next [15]. This design trade-off motivates the world model architectures reviewed in Chapter 3.

2.4 Conditional variational autoencoders

A VAE maps an input to a distribution of latent vectors described by a mean μ and standard deviation σ, rather than a single point. Sampling different latent vectors lets the decoder produce different but plausible outputs rather than one fixed answer. Figure 2.6a shows this standard encoder-latent-decoder flow [16].

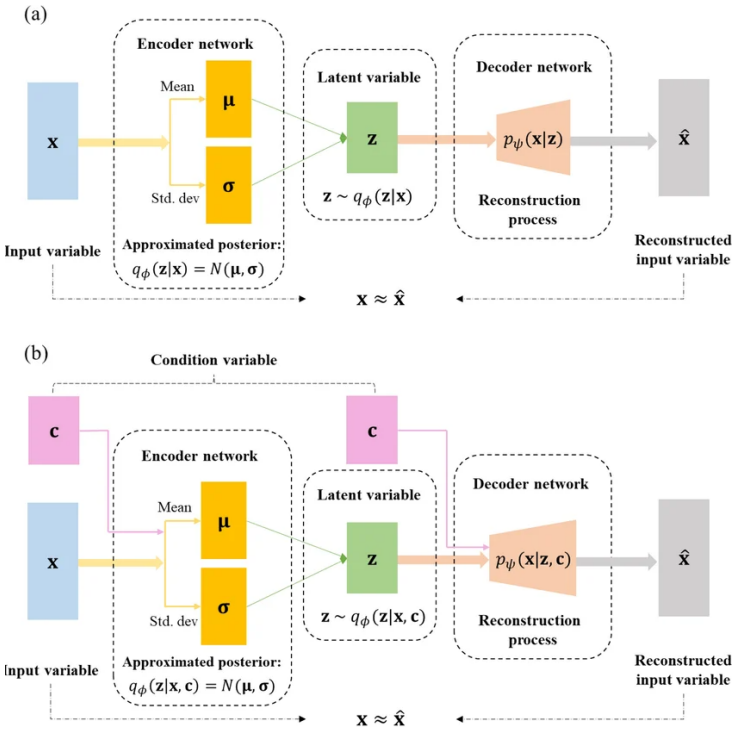


Figure 2.6 Standard VAE (a) and conditional CVAE (b) encoder-decoder pipelines [17].

For the policy to be trainable end-to-end, gradients must flow from the action prediction loss all the way back to the encoder. Sampling z directly from a distribution blocks this path. The reparameterisation trick resolves this by writing $z = \mu + \sigma\epsilon$, where ϵ is standard normal noise drawn independently of the learned parameters. Figure 2.7 shows the full encoder-latent-decoder pipeline alongside its training loss: a reconstruction term that rewards accurate output and a regularisation term that keeps the latent distribution close to a standard Gaussian, together forming the evidence lower bound [16].

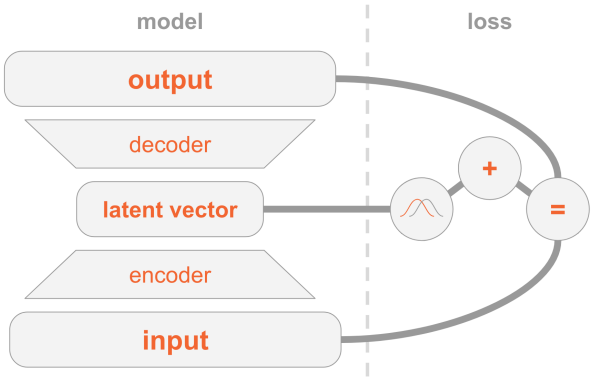


Figure 2.7 VAE encoder-decoder architecture and two-part training loss (reconstruction and regularisation) [18].

A CVAE adds an extra conditioning input to both encoder and decoder, shown in Figure 2.6b. In a visuomotor policy, this condition is the current observation frame,

so the latent variable only needs to capture how the future action varies within that scene [19]. The ACT architecture uses this to sample multiple latent vectors from one observation and decode each into a candidate action chunk [7].

2.5 Transformer networks and the attention mechanism

A transformer processes a sequence as a set of tokens. A token can represent a word, an image patch, a timestep in an action sequence, or any vector produced by an earlier model component. The architectural idea needed here is simple: each token is updated by looking across the other tokens and deciding which of them are most relevant. This relevance-weighted exchange of information is called attention [20].

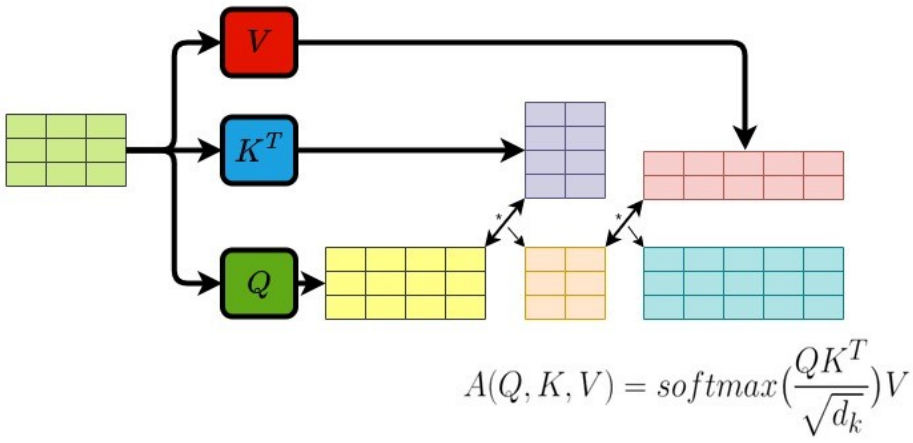


Figure 2.8 Transformer attention block: each token queries all others with Q, gathers weighted values V via keys K, and the weighted sum yields the updated output token [21].

Attention forms three learned views of every token:

- **Query** (Q) – what this token is looking for.
- **Key** (K) – what another token contains.
- **Value** (V) – what another token contributes if attended to.

Figure 2.8 shows how each token (highlighted) sends its query outward to every other token, receives weighted values in return, and produces an updated output. The bidirectional arrows indicate that every token attends to every other token simultaneously, and the crossing arrow beneath represents the weighted sum that combines those values into the final output. For each token, the transformer compares its query with the keys of all tokens. The resulting scores become weights, and those

weights are used to combine the corresponding values. This operation is usually written as [20]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{2.2}$$

Transformers usually repeat this operation in several heads and add positional information so the model knows where each token sits in the sequence. Multiple heads allow different relationships to be represented at the same time, while positional information preserves order [20].

2.6 Self-supervised learning

Machine learning methods can be grouped by how they receive feedback during training, as illustrated in Figure 2.9. Supervised learning trains on data with explicit labels: the correct answer is provided for every input, and the model learns to reproduce it. BC from Section 2.1.1 is a direct example: the expert demonstration acts as the label. Reinforcement learning instead receives feedback through a reward signal after taking actions in an environment; the model learns which state-action sequences lead to good outcomes. SSL sits between these two: it trains on unlabelled data by having the data itself provide the supervision signal. Part of the input is hidden, and the model learns to predict it from the remainder [22].

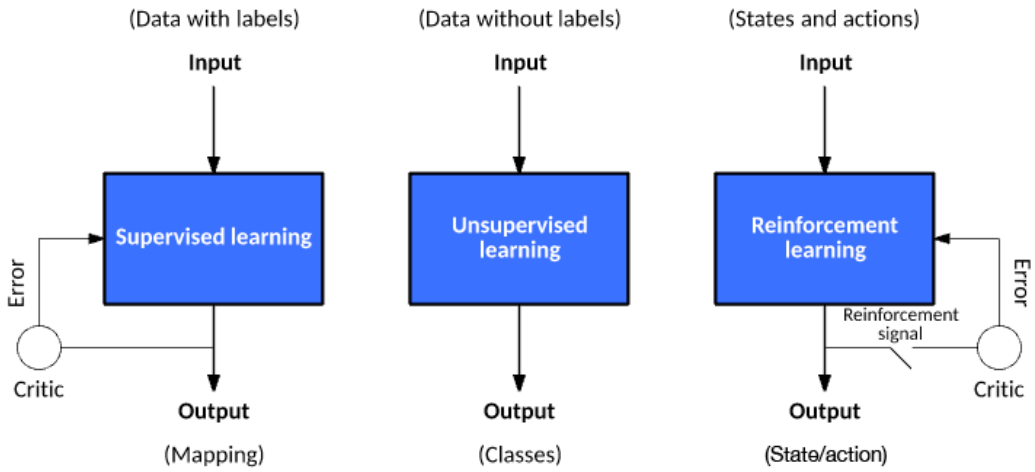


Figure 2.9 Three machine learning paradigms: supervised learning (explicit labels), reinforcement learning (reward signal), and self-supervised learning (unlabelled data provides its own supervision signal). The figure labels this third paradigm as unsupervised learning; the two terms are used interchangeably in this context [23].

Applied to video, SSL takes the following form: given a sequence of observed frames, hide one or more future frames and ask the model to predict them from the

frames that came before. No human annotation is required because the video recording itself provides the hidden target.

Predicting raw pixels runs into a problem described in Section 2.3: when the future is ambiguous the model cannot commit to a single outcome, so it predicts the average of all possibilities and produces a blurred image that does not match any real frame. Predicting in latent space avoids this. Instead of reconstructing every pixel, the model predicts the compact embedding of the future frame. Background motion and fine texture are discarded at the encoding step, so the predictor focuses on the structure that determines what actually happens next [15]. When the predictor is also given the action the agent intends to take, the result is a world model: a learned function that predicts how the scene embedding will change in response to a proposed action. The architectures that build on this idea are reviewed in Chapter 3.

3 Literature Review

This chapter surveys the literature motivating the three central design choices of this study's benchmark: that a specialist imitation policy is a strong base for bimanual manipulation, that a world model can usefully evaluate candidate action chunks in an offline benchmark, and that latent predictive learning is the right approach for building such an evaluator. Sections 3.1 and 3.2 review the two candidate imitation policies, ACT and Diffusion Policy, examining both the challenges they address and the trade-offs between them. Section 3.3 introduces world models as learned environment predictors, surveys the evaluator architectures most relevant to this role, and covers the three latent predictive architectures implemented in the benchmark, together with a negative control. Section 3.4 covers offline evaluation frameworks and their validity as proxies for real-world performance. The chapter closes with a summary of the literature gap that the benchmark addresses. The chapter assumes familiarity with the technical foundations in Chapter 2; world models are introduced here.

3.1 Action Chunking with Transformers (ACT)

ACT was introduced by Zhao et al. to address the compounding error and multimodal distribution challenges that make dexterous bimanual manipulation demanding for imitation learning [7]. On the A Low-cost Open-source Hardware System for Bimanual Teleoperation (ALOHA) bimanual platform it achieved success rates of up to 90% on precision tasks such as threading a cable tie, inserting a battery, and opening a translucent bag, outperforming behaviour cloning baselines and Generalist Agent (GATO) by substantial margins [7]. In SE(3) wrist-pose prediction specifically, small deviations in predicted position or orientation alter contact geometry in ways that cascade into grasp failure [1], and a policy trained with a naïve regression loss produces averaged predictions that may not represent any plausible trajectory when the demonstrated distribution is multimodal [9]. ACT's CVAE architecture addresses both of these failure modes directly.

ACT addresses both challenges through the use of a CVAE design (Section 2.4). The respective CVAE has an encoder-decoder structure depicted in Figure 3.1, where the encoder compresses an action sequence and joint observation into a style variable z (List of Notations) that captures the multimodal distribution of plausible futures, and the decoder takes visual observations and joint positions, together with z, through a transformer encoder, and predicts a sequence of future action steps as an action chunk with a transformer decoder. The encoder is discarded at test time. In the original formulation, z is set to the mean of the prior (zero) to produce a single deterministic

action chunk per observation [7]. When a pool of candidate chunks is required, multiple values of z are sampled independently from the prior and each is decoded into a separate candidate, all conditioned on the same observation.

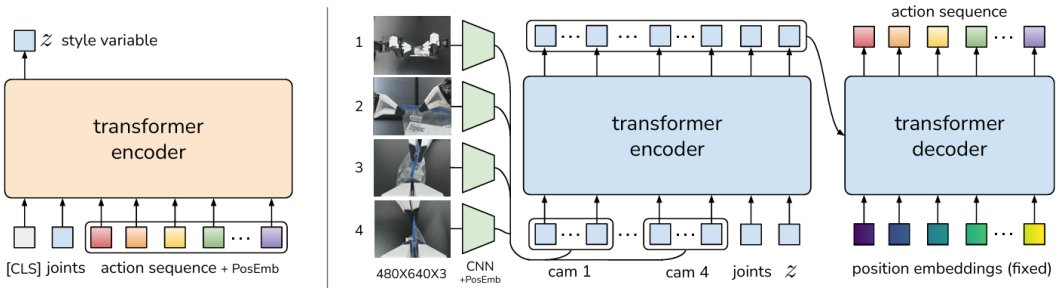


Figure 3.1 Architecture of the Action Chunking with Transformers (ACT) policy [7].

At inference time, the CVAE decoder produces each candidate action chunk by decoding an independently sampled z vector. A temporal ensembling technique is then applied: consecutive chunks overlap in their predicted time horizons, and a recency-weighted average is taken across overlapping predictions to produce the final action. This averaging smooths the executed motion and reduces jerkiness caused by abrupt chunk transitions. The result is a structured pool of plausible futures, all conditioned on the same observation, from which the ensembled output is drawn [7].

ACT carries known limitations alongside these strengths. The Gaussian prior on the style variable z constrains how faithfully the CVAE can represent highly multimodal action distributions: at transitions where qualitatively different trajectories are equally plausible, the latent space can average across modes rather than sample cleanly from each one, producing trajectories that are plausible in aggregate but imprecise at the most ambiguous decision points [9]. Chunk length is a fixed hyperparameter that must be tuned per task; longer chunks improve smoothness but commit the policy to a trajectory over a horizon that may no longer be appropriate as the scene evolves. Temporal ensembling introduces a recency-weighted lag that can slow the policy's response to sudden environmental changes [7]. These properties are well-documented in follow-on work comparing ACT with more expressive policy classes, and they motivate the inclusion of Diffusion Policy as the second candidate in the benchmark.

3.2 Diffusion Policy

Diffusion Policy offers a principled alternative approach to the same compounding error and multimodality challenges. Its core concept relies on starting from a noise distribution and iteratively refining action predictions through a diffusion process. By gradually denoising the predictions, Diffusion Policy captures complex multimodal

distributions effectively, producing plausible trajectories that align with the demonstrated data [9]. Figure 3.2 illustrates the policy overview:

- a) General formulation: at timestep t, the policy takes the latest T steps of observation data O_t as input and outputs T steps of actions A_t.
- b) CNN-based Diffusion Policy: FiLM (Feature-wise Linear Modulation) [24] conditioning of the observation feature O_t is applied to every convolution layer, channel-wise. Starting from K_t drawn from Gaussian noise (List of Notations), the output of noise-prediction network ϵ_θ is subtracted, repeating K times to get A_t^0, the denoised action sequence.
- c) Transformer-based Diffusion Policy: the embedding of observation O_t is passed into a multi-head cross-attention layer of each transformer decoder block. Each action embedding is constrained to only attend to itself and previous action embeddings (causal attention) using the attention mask illustrated [25, 26].

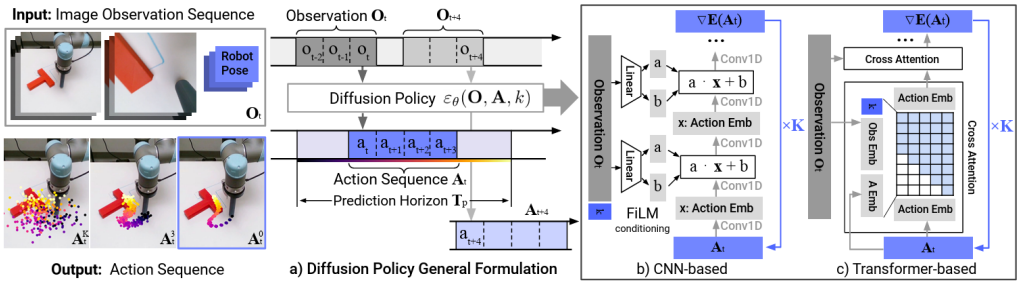


Figure 3.2 Diffusion Policy: general formulation (a), CNN variant (b), transformer variant (c) [9].

Chi et al. report that Diffusion Policy outperforms ACT on tasks with high action multimodality (such as multi-step object rearrangement and cup placement under visual ambiguity) while matching it on simpler reaching tasks [9]. The advantage arises because iterative denoising explores the full action distribution at each step rather than committing to a single latent mode as the CVAE prior must, making it more robust to the mode-averaging failure that affects ACT at ambiguous transitions.

The main limitation of Diffusion Policy is inference cost. Generating a single action chunk requires multiple sequential network evaluations: standard Denoising Diffusion Probabilistic Models (DDPM) variants use on the order of 100 denoising steps compared with the single forward pass of ACT's decoder, which substantially reduces the achievable control frequency [9]. Efficient transformer variants address this through progressive noise scheduling and parameter sharing, recovering much of the inference speed at added architectural complexity [26]. Both policies are included in the benchmark to span this representational trade-off: ACT's structured latent

sampling is computationally efficient but expressively constrained at high multimodality, while Diffusion Policy's iterative denoising is more expressive but more demanding at inference. Measuring the world model's effect across both allows the benchmark to determine whether evaluator gains are consistent across this axis.

3.3 World models

A world model is a learned predictor of how a system evolves over time. Given the current state and a proposed action, it estimates the state that is likely to follow. In a manipulation setting, the state may be a raw image, a latent representation of the scene (Section 2.3), or a structured description of hand and object configuration. Regardless of the representation chosen, the underlying principle is the same: the model internalises the dynamics of the environment and uses them to project forward from any given state [27].

World models become especially useful when a pool of candidate actions must be evaluated before any one is committed to. Each candidate is rolled forward through the model, producing a predicted future state; those predictions are ranked by predicted cost or consistency. The quality of this ranking depends heavily on the chosen representation: pixel-space world models must reconstruct every visual detail and suffer from mode-averaging at multimodal transitions, while models that operate in the compact learned latent space introduced in Section 2.3 can concentrate predictive capacity on the dynamics that matter for task outcomes [15]. The following subsections survey the literature supporting this evaluator design and describe the three world model architectures implemented in the benchmark.

3.3.1 World models as policy evaluators

WorldGym demonstrates that a learned world model used as a simulated environment for policy evaluation produces rankings that align with real-world task success with meaningful fidelity [28]. Candidate policies are evaluated not by deploying them physically but by rolling them out inside the model's predicted environment, reducing the cost of policy comparison to a forward pass. CLOVER extends this to closed-loop control, as shown in Figure 3.3. A visual planner generates a sequence of sub-goal frames; the executor measures the embedding-space error between the predicted sub-goal and the current observation. When the error exceeds a threshold, the system triggers an adaptive transition and replans rather than committing to the original trajectory. The feedback loop, absent from open-loop imitation policies, is the mechanism that translates world model prediction accuracy into task completion

gain [29]. Together, WorldGym and CLOVER establish that world model rankings are informative both before and during execution.

The broader question of whether such evaluator rankings transfer to physical outcomes, and the qualifications around out-of-distribution reliability, are examined in Section 3.4.

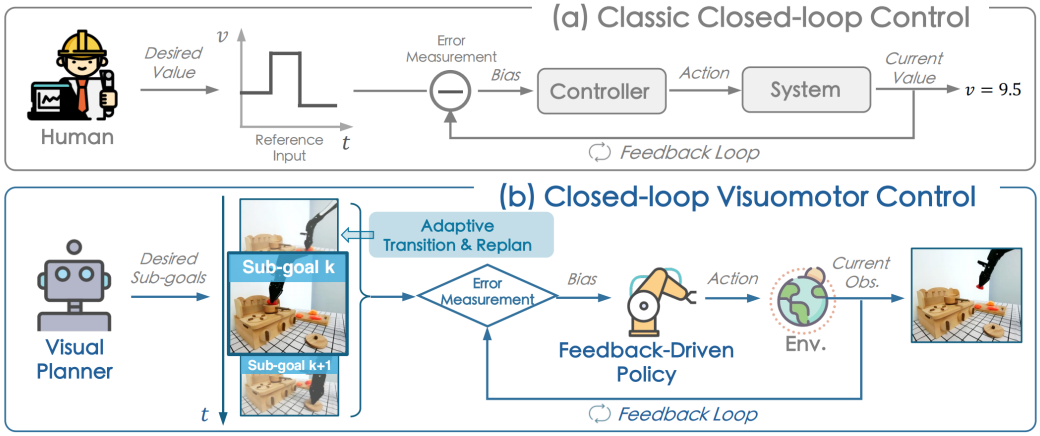


Figure 3.3 Classic closed-loop control (a) versus CLOVER's closed-loop visuomotor control (b) [29].

The practical value of this evaluator loop is measurable. Table 3.1 reproduces CLOVER's real-world evaluation on a multi-step robot manipulation suite, reporting the average number of consecutive sub-tasks completed (`avg_len`) for representative baselines and CLOVER [29].

Table 3.1 Average consecutive sub-task completions (`avg_len`) on a real-world multi-step manipulation suite. Higher is better. CLOVER's closed-loop feedback via world model prediction nearly doubles the completion depth of the strongest open-loop baseline. Data from [29].

Method	avg_len
ACT [7]	0.6
R3M (pretrained visual repr.) [30]	0.7
RT-1 [31]	1.1
CLOVER (closed-loop)	**2.1**

Alongside the task completion gain, CLOVER reduces embedding-space prediction Root Mean Square Error (RMSE) relative to open-loop baselines by correcting the policy's course when the world model detects a divergence from the expected trajectory, demonstrating that prediction accuracy and task success move together under the evaluator framing.

3.3.2 Latent predictive architectures

The energy-based framework of LeCun and Dawid establishes why prediction should operate in latent rather than pixel space [15]: the same representational argument introduced in Section 2.3 applies directly to the prediction target, favouring compact embeddings over pixel reconstructions that force the model to account for irrelevant visual detail and average over all plausible continuations at ambiguous transitions.

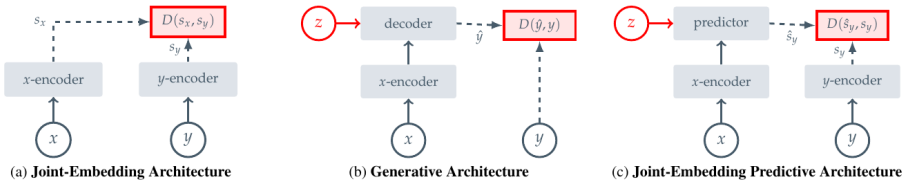


Figure 3.4 Three SSL prediction paradigms: joint-embedding (a), generative (b), and joint-embedding predictive (c) [22].

Image Joint Embedding Predictive Architecture (I-JEPA) establishes the joint-embedding predictive architecture as a viable approach to learning rich visual representations without any pixel reconstruction objective [22]. Panel (c) of Figure 3.4 shows the data flow directly: a context block x from the image is passed through the context encoder to produce latent tokens z_x. A lightweight ViT predictor then takes z_x together with positional tokens indicating where the masked target regions y are located, and outputs predicted latent representations $\hat{s}_y$ for those regions. A separate target encoder, whose weights are a slow exponential moving average of the context encoder, independently encodes the actual target pixels into ground-truth latents s_y. The loss $\text{MSE}(\hat{s}_y, s_y)$ is computed entirely in latent space: no decoder, no pixel generation, no reconstruction of lighting or texture. The outcome is a context encoder whose representations capture semantically meaningful structure (object shape, spatial layout, and scene dynamics) because that is all the predictor ever needs to get the target embeddings right. Training is approximately 2.5 times faster than pixel-reconstruction methods as a result. V-JEPA2 extends this same joint-embedding predictive architecture from images to video sequences.

Four architectures are covered in the following subsections, each representing a distinct representational strategy. V-JEPA2 and V-JEPA2-AC share the same pretrained video backbone: V-JEPA2 is the internet-scale pretrained encoder, while V-JEPA2-AC extends it with an action-conditioned transition head and is the form used directly in the benchmark. DexWM uses a pretrained image backbone with specialised hand-geometry supervision. LeWorldModel (LeWM) learns all representations from scratch on the task data itself.

V-JEPA2

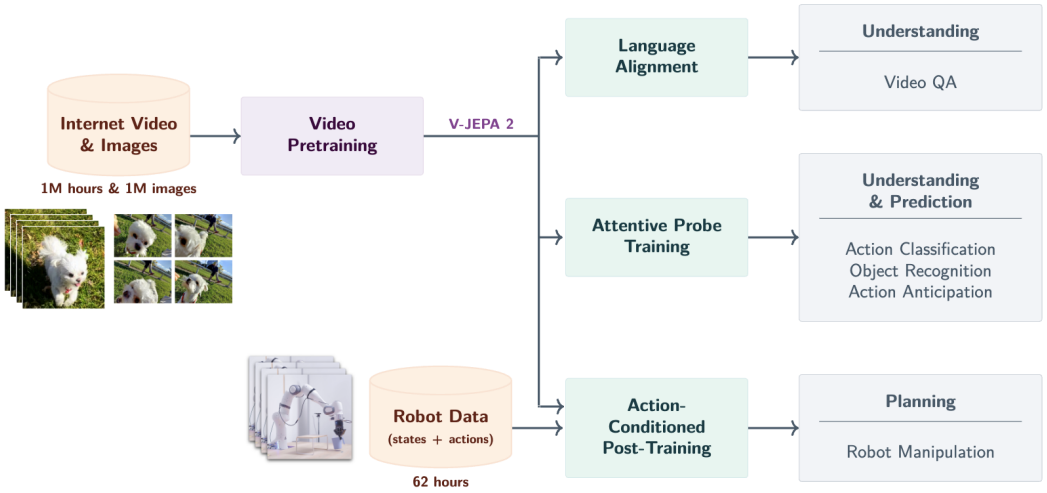


Figure 3.5 V-JEPA2 post-training pathways from internet-scale backbone to specialised applications [32].

V-JEPA2 extends the I-JEPA framework from static images to video, pretraining on over one million hours of internet video [32]. The pretraining shifts from spatial to spatio-temporal masking: the context encoder receives past observation frames, and the predictor must recover future latent states. The training objective combines teacher-forcing with multi-step autoregressive rollout losses, encouraging accurate short-horizon predictions and consistent long-horizon trajectory representations. The result is a frozen ViT-Large encoder, approximately 300 million parameters, that encodes physical priors about hand and object interactions acquired from internet-scale pretraining. As shown in Figure 3.5, the pretrained backbone supports three post-training pathways: Language Alignment, which grounds visual embeddings in natural language; Attentive Probe, which adds a lightweight classification head; and Action Conditioning, which extends the predictor to take robot actions as input and forms the basis of V-JEPA2-AC.

V-JEPA2-AC

The frozen ViT-Large encoder from V-JEPA2 serves as the backbone of V-JEPA2-AC, producing a fixed latent embedding for each observation frame with no gradient updates to the backbone weights. A lightweight action-conditioned transition head is trained on top of these stored embeddings: it takes the current frame latent together with a proposed action chunk (robot joint states and end-effector poses) as the conditioning signal z, and predicts the latent representation of the future frame. Candidate action chunks are then scored by comparing the predicted future embedding against the actual observed future embedding under an L1 objective. This

design, shown on the right of Figure 3.6, separates what the network must learn into two parts: the frozen encoder supplies rich visual features that generalise from internet-scale pretraining, and the small trained head learns only how actions propagate through those features.

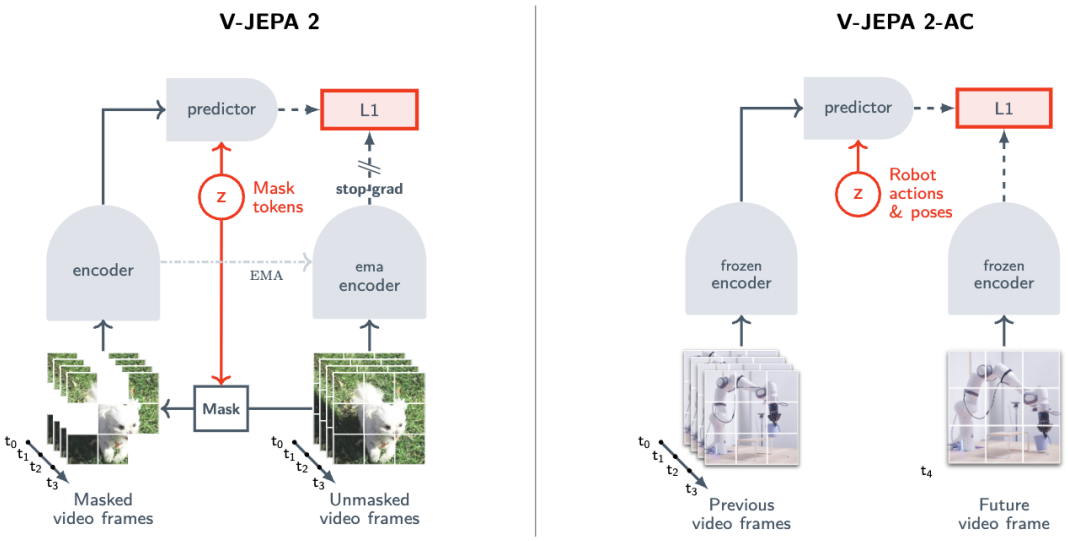


Figure 3.6 Standard V-JEPA2 (left) versus action-conditioned V-JEPA2-AC (right) [32].

An important qualification is raised by Tsagkas et al., who show that pretrained visual representations can encode spatial plausibility well while providing a weaker signal for fine-grained temporal dynamics [33]. A frozen encoder trained on internet video may represent hand shape and position accurately at each individual frame while providing a less reliable signal about whether the predicted trajectory is kinematically consistent across the full chunk horizon. This temporal limitation provides context for the path shape results in Chapter 8.

DexWM

DexWM [34] is a dexterous manipulation world model from Meta FAIR. As shown in Figure 3.7, its pipeline encodes each input frame with a frozen DINOv2 image encoder, producing a grid of spatial patch tokens rather than a single global embedding. These patch-level representations preserve fine-grained spatial information about hand positions and object locations, which gives the predictor strong generalisation across diverse environments and viewpoints. A CDiT predictor then takes the spatial latent tokens together with action and camera-motion inputs to predict the next latent state, and a lightweight decoder maps the result back to image and keypoint space. Where V-JEPA2-AC uses a video-pretrained backbone, DexWM uses a backbone pretrained on static images; the comparison therefore isolates the contribution of video-specific temporal pretraining to world model quality.

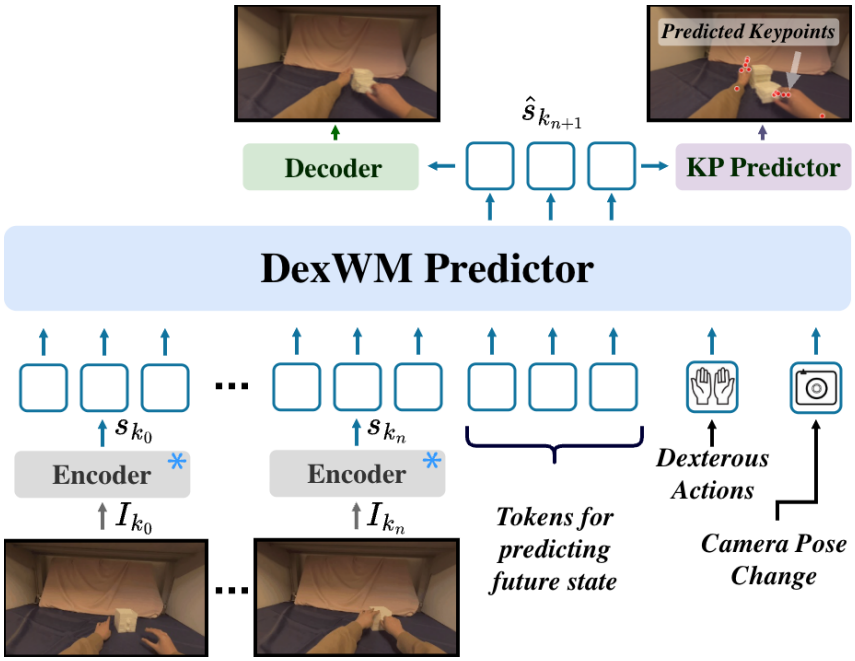


Figure 3.7 High-level layout of the DexWM world model pipeline, illustrating how predicted keypoints are incorporated alongside the encoder, CDiT predictor, and decoder stages [34].

A distinguishing feature of DexWM is the Hand Consistency (HC) loss: an auxiliary transformer head predicts fingertip and wrist heatmaps from the latent state, imposing a geometric prior that the predictor's latent trajectory must remain consistent with plausible hand configurations. This explicit hand-geometry supervision is absent from both V-JEPA2-AC and LeWM, whose latent spaces are shaped only by prediction error or general regularisation. Goswami et al. report a 50% improvement over Diffusion Policy on grasping and placing tasks, with an 83% zero-shot real-world grasping success rate under latent planning [34].

LeWorldModel

LeWM is a latent world model trained end-to-end from random initialisation on raw pixel observations [35]. Its architecture consists of two components: an encoder that maps each frame o_t to a compact latent vector z_t, and a predictor that models environment dynamics by estimating the next latent embedding $\hat{z}_{t+1}$ from z_t and the action a_t. The encoder is a ViT-Tiny with approximately 15 million parameters. Each frame is represented as a single 192-dimensional token rather than a spatial patch grid, which keeps the latent space compact and makes planning significantly faster than models that produce higher-dimensional patch representations.

As shown in Figure 3.8, the training pipeline consists of an encoder that maps each observation frame to a compact latent vector, a dynamics predictor that autoregressively estimates the next latent given the current embedding and action, and

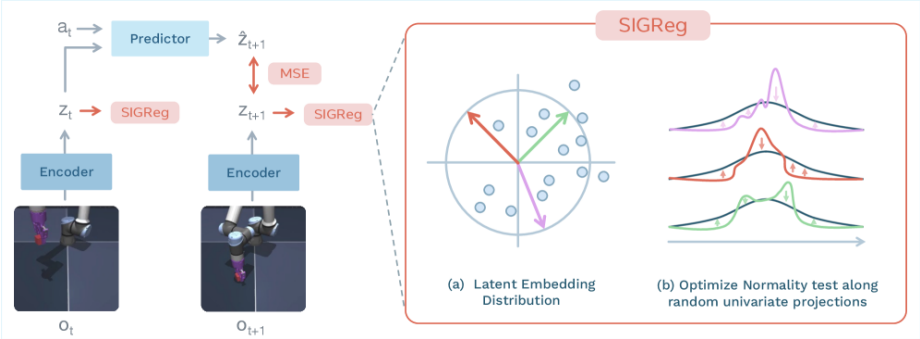


Figure 3.8 LeWM training pipeline: encoder, dynamics predictor, and SIGReg regulariser [35].

a SIGReg regulariser that enforces a Gaussian prior on the latent distribution by optimising normality along random univariate projections. The training objective reduces to two terms:

Z	batch of latent vectors produced by the encoder
λ	regularisation weight balancing prediction and SIGReg losses
SIGReg(Z)	Gaussian regulariser on random univariate projections of Z

$$\mathcal{L}_{\text{LeWM}} = \mathcal{L}_{\text{pred}} + \lambda\,\text{SIGReg}(Z) \tag{3.1}$$

where $\mathcal{L}_{\text{pred}}$ penalises inaccurate next-embedding forecasts and SIGReg(Z) stabilises end-to-end training without contrastive negative pairs or an exponential moving average target encoder [35]. This reduces the number of tunable loss hyperparameters from six, as required by prior Joint Embedding Predictive Architecture (JEPA)-style designs, to one.

DINO-WM, the closest comparable baseline, follows a similar latent prediction objective but uses a frozen DINOv2 backbone producing a grid of spatial patch tokens per frame [36]. Like DexWM, DINO-WM therefore carries the spatial resolution and generalisation benefits of DINOv2's patch-level representations, but at substantially higher computational cost. LeWM replaces the frozen backbone with a fully learned ViT-Tiny encoder that collapses each frame to a single 192-dimensional token, trading spatial resolution for training speed. In practice, LeWM can be trained on a single GPU in a few hours and achieves a 48$\times$ planning speedup over DINO-WM [35]. DexWM's CDiT predictor operating on full patch grids is significantly slower to train and requires substantially more GPU memory; this difference in training cost is expected to widen further at larger dataset scales and is quantified in Chapter 5. Despite its compact size, LeWM's latent space encodes physically meaningful structure: the model detects physically implausible state transitions and captures spatial information such as agent and object locations across frames.

The three-way comparison across V-JEPA2-AC, DexWM, and LeWM is

therefore not simply an architecture comparison. It is a comparison between fundamentally different representational strategies: video-pretrained transfer (V-JEPA2-AC), image-pretrained transfer with geometric supervision (DexWM), and self-supervised task-data learning from scratch (LeWM). This distinction makes the comparison principled and is the basis for the experimental design in Chapter 4.

PointWorld

PointWorld is a large pretrained 3D world model for in-the-wild rigid arm manipulation [37]. Unlike the latent-space models described above, it operates directly in 3D geometry: scene state is lifted from RGB-D images into a full-scene point cloud, and robot actions are represented as the 3D point flows traced by robot surface geometry through forward kinematics. This embodiment-agnostic representation allows the model to reason over physical interaction geometry rather than learned abstract features.

Such a model is not designed to handle a 16-dimensional dexterous hand action space, where fine-grained finger articulation drives the manipulation outcome. The forward-kinematics representation assumes rigid arm geometry, making the model structurally mismatched when applied to dexterous bimanual hand motion.

3.4 Offline evaluation

A central concern for any offline benchmark is whether its metrics produce rankings that are meaningful proxies for real-world performance. Without this property, an offline evaluation framework reduces to a measure of in-distribution accuracy with no predictive validity for deployment.

At large scale, DreamerV3 demonstrates the effectiveness of latent-space candidate evaluation across diverse continuous control domains, including locomotion and manipulation [27]. DreamerV3 is an online reinforcement learning system, not an offline benchmark; it is included here because it provides large-scale empirical evidence that latent-space evaluation is an effective selection mechanism when properly calibrated, which motivates its use in the offline setting examined in this study. Its world model maintains a compact recurrent latent state that transitions under action conditioning; candidate trajectories are rolled out entirely inside this imagined space and scored by a learned value function, with no return to the real environment between evaluations. TD-MPC2 confirms the principle with a scalable planner that optimises a temporal-difference value function defined over predicted latent trajectories rather than observed states [38]. Both systems show that ranking

candidate actions by predicted latent-space outcomes is competitive with direct online evaluation across a range of motor control benchmarks.

A critical qualification is that latent-space scoring becomes unreliable outside the training distribution. MOPO, MOReL, MBOP, and MOPP each establish that learned models produce inaccurate predictions for out-of-distribution states, requiring conservative candidate selection to avoid trajectory degradation [39–42]. None of these works addresses dexterous manipulation specifically; they benchmark locomotion and simple control tasks, leaving open how well the principle transfers to the fine-grained wrist-pose prediction setting.

Two further bodies of work establish that offline rankings can transfer to physical outcomes when the evaluation domain is well-calibrated.

SimplerEnv establishes the correspondence between simulation-based evaluation and physical robot performance for visuomotor policies [43]. The study evaluates multiple manipulation tasks and policy types in simulation and then physically, using a WidowX robot under matched visual and control conditions. Rankings produced from simulation-only evaluation closely track rankings from physical trials, with correlations holding across both policy families and task types. The key condition for transfer is domain calibration: visual appearance, camera viewpoint, and control interface must match between the simulation and the physical system. When these are well-matched, simulation rankings provide a reliable ordering of policies without requiring physical deployment.

WorldEval extends the validation to the world-model setting [44]. Rather than evaluating policies in a manually constructed simulator, WorldEval scores them using a learned world model that predicts future states from offline trajectory data. The world-model-based rankings correlate strongly with real-world task success: policies that score well in world-model evaluation also tend to succeed when deployed physically.

Both results carry a shared qualification: rankings hold under well-calibrated conditions, where the evaluation domain is consistent with the training distribution and the deployment context. When this alignment breaks down, offline rankings can degrade. The design implications of this constraint are discussed in Chapter 4.

Summary. The literature surveyed in this chapter establishes three supporting findings. First, world model rankings are informative evaluators both before and during execution, as demonstrated by WorldGym and CLOVER [28, 29]. Second, latent predictive objectives produce more efficient and better-calibrated representations than pixel-level reconstruction, as demonstrated by the JEPA family [22, 32]. Third, offline evaluation rankings can carry predictive validity for physical outcomes when the evaluation domain is well-calibrated, as demonstrated by SimplerEnv and

WorldEval [43, 44].

These findings leave two questions unanswered, which this dissertation addresses directly.

RQ1. Does augmenting a fixed imitation-learning policy with a latent world model evaluator improve candidate selection quality in an offline dexterous manipulation benchmark, and does any improvement hold across policy families?

The offline reinforcement learning literature establishes the need for principled candidate selection but benchmarks only locomotion and simple control tasks [39–42]. No controlled study isolates the world model evaluator's contribution on a fixed dexterous manipulation policy.

RQ2. Does the world model's representational strategy (pretrained video backbone, task-data-trained from scratch, or out-of-domain modality) produce meaningfully different selection guidance when applied to the same policy on the same evaluation data?

The interaction between world model architecture and policy performance has not been examined in this setting. Answering RQ2 requires a benchmark that holds the policy and evaluation surface fixed while varying only the world model family. The methodology for constructing and running such a benchmark is described in Chapter 4.

4 Methodology

This chapter describes the experimental design used to evaluate whether latent world model augmentation improves candidate selection quality when paired with imitation-learning policies, and whether any observed effect generalises across policy families and world model architectures. Five design decisions structure the evaluation: the choice of dataset and action space (Section 4.1); a three-branch framework that isolates the contribution of each selection mechanism (Section 4.2); the selection of two policy families that span distinct generation mechanisms (Section 4.3); the selection of five world model families that cover a range of pretraining regimes and input modalities (Section 4.4); and a fair experimental protocol that normalises training exposure across all trained components (Section 4.5). Evaluation metrics and scope limitations are addressed in Sections 4.6 and 4.7.

4.1 Dataset and Experimental Setup

4.1.1 EgoDex: source and selection rationale

The benchmark is built on EgoDex [45], a large-scale egocentric dataset of bimanual dexterous manipulation collected using Meta Quest Pro head-mounted cameras. It spans more than 100 task categories across its full release, including folding, slotting, pouring, and kneading, covering a wide range of fine-grained hand motions. The full release comprises five training parts and supplementary additional-data splits; this study uses the first three training parts, which together comprise more than 194,000 demonstration episodes and approximately 1.3 TB of paired video and annotation data. Each episode provides synchronised RGB video and full six-degree-of-freedom $\mathrm{SE}(3)$ wrist annotations for both hands at approximately 30 frames per second.

The choice of dexterous manipulation as the evaluation domain is not incidental. This dissertation forms part of the University of Malta's ongoing M.AI.ESTRO research project [46], which investigates the use of AI for scene and task prediction on dexterous manipulation tasks to support industrial transfer learning and human skill digitisation. The research domain was therefore set by this institutional context: the methodology operates on human hand demonstrations rather than robotic end-effector data because the target application is the transfer of tacit human dexterous skill. An evaluation framework built around robot arm trajectories would not address that objective. The introduction situates this motivation more broadly; the present chapter focuses on the methodological consequences of the domain choice.

Within the dexterous manipulation setting, EgoDex was selected on three

properties:

- **Bimanual hand action space.** The benchmark requires fine-grained articulation across an eighteen-dimensional bimanual wrist pose representation; large-scale corpora built around rigid gripper or arm contact do not provide this.
- **Egocentric viewpoint.** The first-person perspective aligns with the intended downstream deployment setting of a wearable assistive device.
- **Dataset scale and diversity.** The corpus is sufficient to support simultaneous training of five world model families and two policy architectures on declared non-overlapping splits while retaining a held-out test set of meaningful size.

EgoDex carries two acknowledged limitations. The demonstrations are produced by human subjects rather than a robotic system, which introduces a kinematic domain gap between the training distribution and any robotic deployment. No contact or force signals are present; the action representation is purely kinematic $\mathrm{SE}(3)$. Both are accepted as known constraints rather than experimental variables and are discussed further in Section 4.7.

4.1.2 Windowing protocol

The dataset is stored as paired `.mp4` and `.hdf5` episode files. A deterministic index stage discovers all valid paired episodes, infers their lengths from HDF5 metadata, and writes canonical observation and prediction windows for each split. This shared index is the single source of truth for all downstream modules: policy training, world model extraction, and evaluation all consume the same window file for their respective split, ensuring that output differences across the experimental branches can be attributed to branch design rather than to differences in the data presented to each component. The benchmark uses a fixed-stride windowing scheme as the canonical contract; a supplementary state-change-targeted scheme was developed during early exploration and used exclusively for qualitative inspection.

Strategy 1: Fixed-stride sampling. Each window spans 16 observation frames followed by 16 prediction frames, with consecutive window starts separated by a stride of 128 frames. At 30 frames per second this places starts approximately 4.3 seconds apart, corresponding to an observation context and prediction horizon of approximately 0.53 seconds each. This contract is encoded as `obs16_pred16_s128` and forms the foundation for all policy training and world model training (see Figure 4.1).

Strategy 2: State-change-targeted sampling. The fixed-stride scheme samples the episode timeline uniformly, which causes it to under-sample high-motion events: long near-static segments contribute windows at the same rate as active manipulation periods. As detailed in Chapter 5, this limitation motivated a revised strategy adopted

Figure 4.1 Fixed-stride windowing scheme (`obs16_pred16_s128`) used as the canonical training and evaluation index. Author-generated schematic.

for a subsidiary perceptual inspection task and qualitative development review. Rather than placing windows at fixed intervals, this approach computes smoothed hand-state change scores from the wrist transform and confidence sequences, selects candidate windows near high-change peaks, and applies episode-first task-stratified sampling with a fixed random seed (see Figure 4.2). A relative-position fallback retains slower episodes that would otherwise be excluded, yielding one frozen window per episode stratified across all represented task categories.

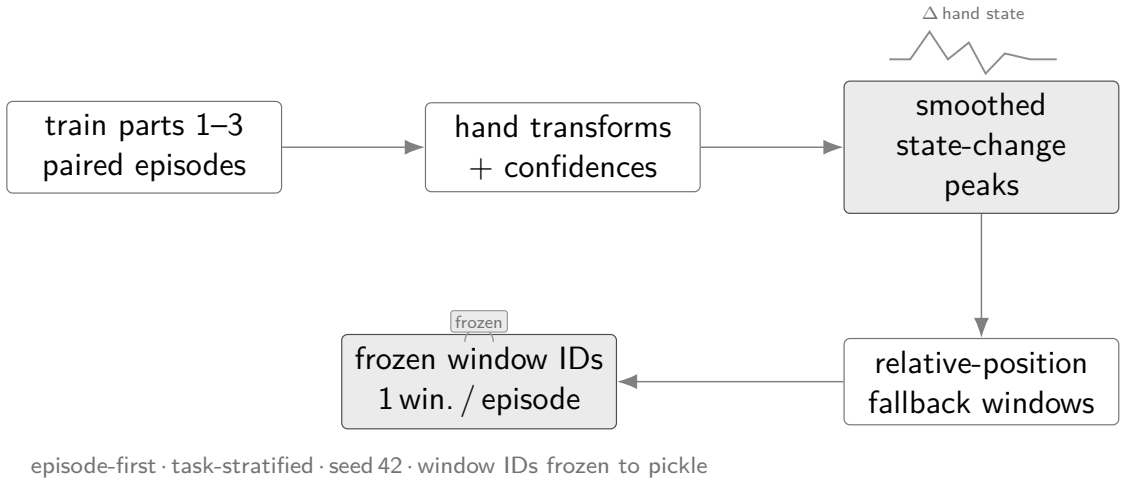


Figure 4.2 State-change-targeted sampling pipeline yielding one frozen window per episode. Author-generated schematic.

The canonical training and evaluation index uses the fixed-stride scheme across all experimental branches. The state-change-targeted subset was used exclusively for a subsidiary perceptual inspection task and qualitative development review; it does not form part of the primary benchmark evaluation.

4.1.3 Training splits and test set

Three cumulative training splits were assembled from the first three parts of the EgoDex release, each a strict superset of the previous; Table 4.1 summarises their composition. EgoDex ships five training parts plus supplementary additional-data splits; parts four and five and all additional-data splits are excluded from this study to bound the experimental scope. The held-out test split is the official EgoDex test set

and is never used during training at any stage by any model or policy. For the Strategy 1 benchmark, the same fixed-stride test index is used for quantitative evaluation and for the qualitative review examples, keeping the reported comparisons aligned with the training contract in Table 4.1.

Table 4.1 EgoDex splits under the `obs16_pred16_s128` windowing contract. Episode and window counts are derived from the canonical index.

Split	Tasks	Episodes	Windows	Role
`train_part1`	26	46,091	140,239	Stage 1 training
`train_part1_2`	39	140,654	291,369	Stage 1+2 training
`train_part1_2_3`	63	194,333	430,962	**Canonical training**
`test`	111	3,229	7,607	**Frozen evaluation**

The episode counts reflect the uneven collection density across parts: the 13 task categories added in part two average approximately 7,300 episodes each, roughly four times the average of the 26 part-one tasks, because part two focuses on high-frequency foundational primitives such as pick-and-place and folding that were collected at greater volume.

The canonical results in Chapter 7 use `train_part1_2_3` across all trained components. The test split is the official EgoDex held-out set spanning 111 task categories, which includes all 63 categories present in `train_part1_2_3` as well as 48 categories drawn from the excluded parts; evaluating on this broader set provides a measure of out-of-distribution generalisation. Its window index is frozen; no post-hoc filtering or re-sampling is applied. The incremental training curriculum across the three stages is described in Chapter 5.

4.2 Three-Branch Experimental Framework

The central question of this study is whether a latent world model improves candidate selection quality when paired with an imitation-learning policy, and whether that effect generalises across policy families. Answering both questions cleanly requires holding every other experimental factor constant. The benchmark therefore organises evaluation around three branches that share the same policy weights and the same evaluation windows; the candidate pool composition and selection rule differ across branches (Figure 4.3). Any measured performance difference between branches is intended to reflect the selection mechanism, since training data, architecture, and window coverage are held constant across branches.

Branch 1 (B1): Policy-only baseline. The policy generates its full candidate pool and selects by its own internal preference; no world model is queried. The ideal design

produces several structurally independent candidates so the downstream branches can compare meaningfully different trajectories. B1 establishes the trajectory prediction quality each policy achieves acting alone and is the baseline against which B2 and B3 are measured.

Branch 2 (B2): World-model-inclusive selection. The world model scores all candidates in the pool and selects the highest-scoring one unconditionally; no margin threshold is applied. The candidate pool is the same as B1. Scoring proceeds by predicting forward in the world model's latent space from the current observation and measuring prediction fidelity against each candidate action. The policy's own output is not privileged; it is selected only if the world model assigns it the highest score. For V-JEPA2 without action conditioning, the transition head required to produce candidate-specific latent predictions is absent; if the model assigns indistinguishable scores to all candidates, B2 degenerates to B1, directly testing whether action conditioning is the operative mechanism.

Branch 3 (B3): World-model-exclusive selection. The policy's preferred output is excluded from the evaluation pool; the world model selects the highest-scoring remaining candidate unconditionally. The ideal design ensures the remaining candidates are structurally independent so that exclusion meaningfully changes the selection outcome. B3 tests whether the world model maintains selection quality when the policy's top choice is unavailable. All five world model families apply this protocol.

Two policy families are evaluated: ACT and Diffusion Policy (DP) (Figure 4.3).

- **DP.** The three-branch progression follows the ideal description above. Five independent reverse-diffusion chains seeded from distinct noise samples serve as candidates. B1 uses DP's policy-internal argmax over those five; B2 passes the same pool to the world model scorer; B3 excludes DP's preferred sample and scores the remaining four independent chains.
- **ACT.** CVAE posterior collapse in the trained checkpoint means ACT produces one deterministic output per window ($N{=}1$); investigation and three retraining attempts are documented in Section 5.4. A CEMP planner generates four additional candidates for B2 and B3 by sampling SE(3) perturbations around ACT's output. The branch pools therefore differ from the ideal: B1 contains ACT's single output ($N{=}1$); B2 contains that output plus four CEMP-planned alternatives ($N{=}5$); B3 contains the four CEMP perturbations only. Because all four candidates are anchored to ACT's prediction, B3 for ACT is a world-model-guided local search rather than selection from a policy-independent pool.

Candidate generation details are in Section 4.3.

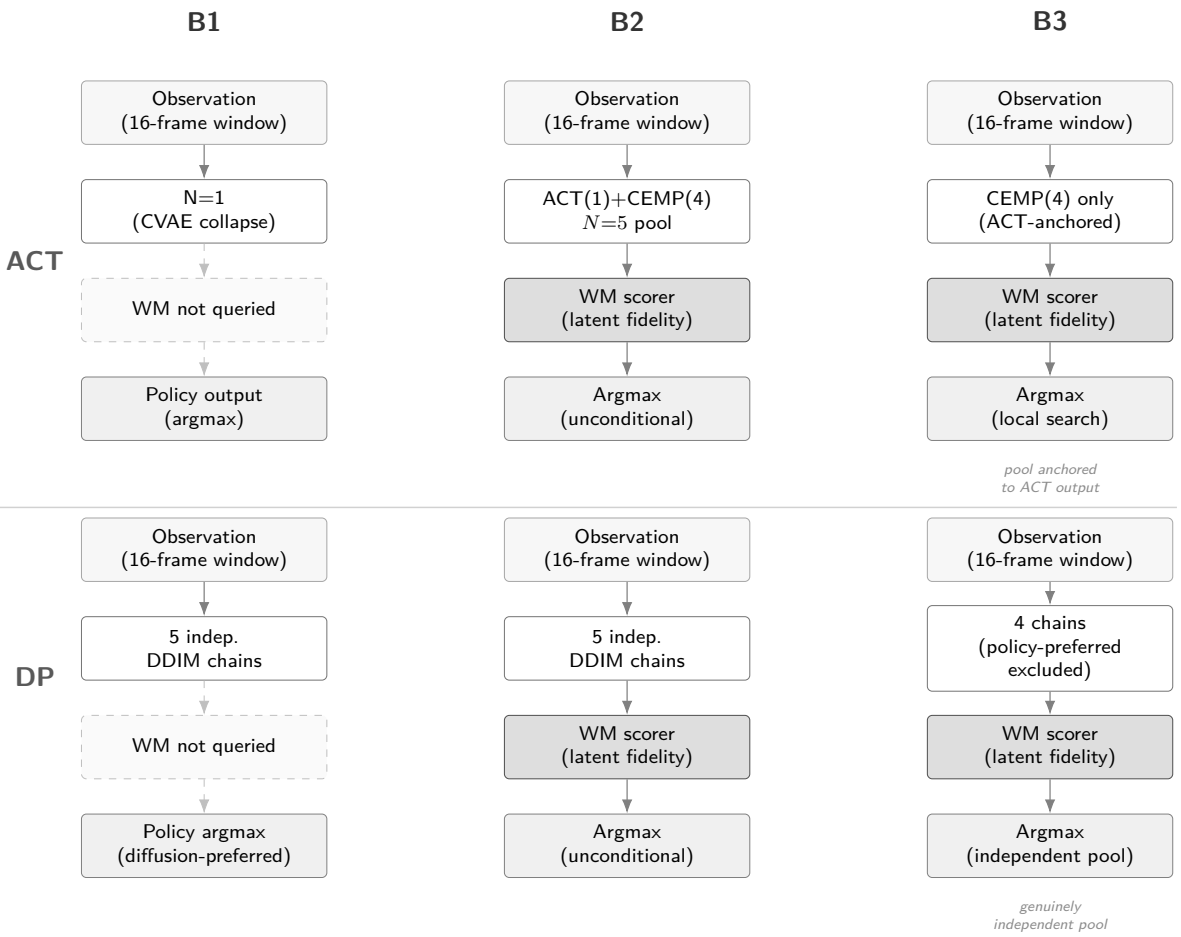


Figure 4.3 Three-branch evaluation framework (Section 4.2). ACT row uses CEMP candidates due to CVAE collapse; DP row follows the ideal design.

4.3 Policy Variants

The benchmark evaluates two imitation-learning policy families. Their architectures and training objectives are reviewed in Chapter 3; the properties relevant to this study are how each generates its candidate pool and what that implies for world-model scoring.

- **Action Chunking Transformer** (Section 3.1). ACT is architecturally designed to generate diverse candidates by sampling latent vectors $z \sim \mathcal{N}(0, I)$ from its CVAE prior and decoding each into a 16-step bimanual SE(3) action chunk. In the trained checkpoint used here, CVAE posterior collapse is observed: the decoder is insensitive to z at inference time, producing identical outputs regardless of the sampled vector (maximum output deviation under $z{=}10$ is $3{\times}10^{-7}$). ACT therefore contributes one deterministic candidate per window ($N{=}1$). The single forward pass requires one network evaluation. Three retraining attempts to resolve the collapse and the subsequent adoption of CEMP for B2/B3 candidate diversity are documented in Section 5.4.
- **Diffusion Policy** (Section 3.2). DP generates each candidate through an independent reverse-diffusion chain using a Denoising Diffusion Implicit Models (DDIM) schedule of ten steps (Section 3.2) [47]. Producing five candidates requires five separate chains and approximately fifty network evaluations per window, roughly an order of magnitude more than ACT. The Diffusion Transformer (DiT) backbone is used. Candidates are structurally independent because each starts from a fresh noise sample; diversity arises from stochastic initialisation rather than from a shared latent manifold.

Evaluating both families under the same three-branch protocol allows the study to ask whether world-model augmentation improves candidate selection in general or only under a particular generation mechanism. A result that holds across both constitutes stronger evidence for the generality of the approach.

4.4 World Model Families

Five world model families are evaluated as candidate selectors within the three-branch framework (Section 4.2). Their architectures and pretraining objectives are reviewed in Chapter 3; this section describes each family's methodological role. The five families span a deliberate range: from a large pretrained video backbone through lightweight from-scratch models to an action-specificity negative control, providing multiple axes of comparison.

At evaluation time each world model receives the current observation and the candidate action chunks from the pool, predicts forward in its latent space, and returns a scalar quality measure per candidate. Policy weights and evaluation windows are fixed; the only variable is which world model scores the pool.

V-JEPA2-AC: Pretrained video backbone. The architecture is reviewed in Section 3.3.2. A frozen V-JEPA2 [32] Vision Transformer (ViT)-Large backbone provides a 1024-dimensional frame embedding pretrained on large-scale video via the JEPA objective. A lightweight action-conditioned transition head, trained on benchmark data, maps a current embedding and an action chunk to a predicted future embedding; the L2 distance between prediction and observed future embedding serves as the candidate score. This family represents the upper end of the representational hierarchy: rich, general-purpose video features acquired at scale before any manipulation-specific training.

LeWM: From-scratch latent world model. The architecture is reviewed in Section 3.3.2. LeWM [35] is trained entirely from raw EgoDex pixels with no pretrained weights. A ViT-Tiny encoder and decoder are jointly optimised via a next-embedding prediction loss and a Gaussian regularisation term, without exponential moving average stabilisation or reconstruction of pixel values. This family represents the opposite end of the pretraining spectrum: all representations are learned from the benchmark training data alone, making it a provenance-clean baseline with no external data dependency. The expectation is that task-domain training from scratch compensates for the absence of video pretraining, testing whether manipulation-domain supervision alone is sufficient for action discrimination at this scale.

DexWM: Domain-specific architecture. The architecture is reviewed in Section 3.3.2. Dexterous Manipulation World Model (DexWM) [34] pairs a frozen DINOv2 image encoder with a CDiT predictor and incorporates a hand-consistency loss that explicitly supervises fingertip and wrist heatmap predictions. It is included to assess whether a model built with explicit hand-geometric supervision outperforms general video predictors on this task. A constraint on the training setup applies: as described in Section 4.7, publicly available model weights could not be sourced and the model was therefore trained from scratch on the benchmark's declared training splits, adopting the same provenance stance as LeWM: no external pretraining and no data outside the declared splits. Despite this constraint, the domain-specific inductive bias of the hand-consistency loss provides a comparison point for whether explicit hand-geometry supervision compensates for the absence of video pretraining.

V-JEPA2 (no action conditioning): Ablation control. This family uses the same frozen V-JEPA2 backbone as V-JEPA2-AC but omits the action-conditioned transition head. The proposed action chunk is not passed to the model. The future embedding is

predicted from visual context alone via a learned visual-only prediction head (Section 5.5.4); the candidate score therefore reflects visual plausibility rather than action-conditional consistency. The purpose is to isolate the contribution of action conditioning: if V-JEPA2-AC outperforms this variant, the advantage is attributable to the model's ability to reason about action consequences rather than to the representational power of the backbone alone. The expected outcome for this variant is a null result at B2 (zero win rate against B1), because action-blind scores are indistinguishable across candidates regardless of what action each proposes.

PointWorld: Action-specificity negative control. This family uses the same frozen V-JEPA2 backbone and transition head as V-JEPA2-AC, but the candidate action chunks are degraded before scoring. Each 18-dimensional bimanual $\mathrm{SE}(3)$ chunk is projected to a 7-dimensional robot arm space by extracting wrist orientation from the left hand only and collapsing finger motion to a scalar aperture, then zero-padded back to 18 dimensions. The right hand is discarded entirely; the 11 recovered dimensions carry no meaningful hand-motion information. The transition head therefore receives the correct visual features but systematically wrong action conditioning. This tests a different hypothesis from the V-JEPA2-No-AC ablation: whereas that variant passes zero actions (no conditioning at all), PointWorld passes degraded actions (wrong conditioning). The expected ordering is V-JEPA2-AC (correct actions) $\geq$ PointWorld (degraded actions) $\geq$ V-JEPA2-No-AC (zero actions), with V-JEPA2-AC providing the strongest discrimination.

Table 4.2 World model families and their methodological roles in the benchmark.

Family	Pretraining	Modality	Role
V-JEPA2-AC	Large-scale video	RGB video	Primary pretrained baseline
LeWM	None (scratch)	RGB video	Provenance-clean from-scratch
DexWM	None (scratch)*	RGB video	Domain-specific (hand-geometry loss)
V-JEPA2 (no AC)	Large-scale video	RGB video	Action-conditioning ablation
PointWorld	Large-scale video	RGB video†	Action-specificity negative control

* The published DexWM architecture targets pretraining on EgoDex and DROID; publicly available weights could not be sourced and the model used here was trained from scratch (Section 4.7).

† The original PointWorld architecture ingests RGB-D input as 3D point clouds. In this study, PointWorld is instantiated as a proxy using the V-JEPA2-AC visual backbone with degraded 7-dimensional robot-arm-style action conditioning; the observation modality is therefore RGB video, not point cloud.

The modality column reports each family's primary input modality. The four RGB video families do not predict or reconstruct pixels: as described in Section 3.3.2 of the literature review, all JEPA-family and encoder-based architectures translate the RGB input into a learned latent representation and perform all prediction and scoring operations in that latent space.

Together, the five families allow comparisons along three independent axes:

pretraining scale (V-JEPA2-AC vs LeWM), action conditioning quality (V-JEPA2-AC vs PointWorld vs V-JEPA2-No-AC, spanning correct to degraded to absent), and domain architecture (V-JEPA2-AC vs DexWM). No single family can be claimed responsible for a result that generalises across all three axes.

4.5 Fair Experimental Protocol

Comparing the five world model families and two policy architectures against each other requires holding every non-design variable constant. Two aspects of fairness are addressed: training exposure and evaluation contract.

4.5.1 Training exposure budget

Different model families have different computational costs per training step and different sensitivities to data volume. Without a principled exposure budget, a comparison could conflate the effect of architecture with the effect of receiving more or fewer training samples. To determine an appropriate shared budget, a convergence study was conducted prior to full training: each model family was trained independently at six data scales (1k, 5k, 10k, 25k, 50k, and 100k effective items) on a fixed subset of the canonical training data, and the resulting loss curves were examined for plateau behaviour. The study and its results are described in detail in Chapter 5; the finding relevant to the experimental design is summarised here.

The effective training exposure E is defined using the following symbols (see the List of Notations):

E	effective training exposure (items consumed)
S	number of optimisation steps
B	per-device batch size
G	gradient-accumulation steps
A	number of accelerator devices
B_{global}	effective global batch size

$$E \;=\; S \times B_{\text{global}} \tag{4.1}$$

where:

$$B_{\text{global}} \;=\; B \times G \times A \tag{4.2}$$

This formulation counts the total training items consumed regardless of how steps are distributed across hardware, allowing models with different batch sizes and GPU counts to be normalised to a common scale.

The convergence study, described in Chapter 5, established a universal budget of $E = 100{,}000$ effective items. This figure is applied uniformly across all model families

and all three training splits, ensuring that any measured performance difference reflects architecture and data rather than exposure volume.

4.5.2 Evaluation contract

All models receive the same RGB input at evaluation time: the observation frames from the canonical test window, without additional depth, point cloud, or semantic metadata. This ensures that any performance difference between world model families reflects their internal representation and prediction quality rather than access to a richer input signal. All five families, including PointWorld, operate on the same V-JEPA2 visual latent representations; PointWorld's distinctive characteristic is its degraded action conditioning rather than a different observation modality.

The evaluation windows are fixed at indexing time and shared across all branches and all world model families. No window is selected, filtered, or re-sampled after training. Policy weights and world model weights are frozen before evaluation begins and are not updated. The only variable at evaluation time is the selection rule applied to the candidate pool, isolating the contribution of each branch design and each world model family from all other sources of variance.

4.6 Evaluation Metrics

4.6.1 Offline trajectory prediction setting

EgoDex does not define an official offline evaluation metric. The original benchmark measures policy effectiveness via online task success rate on a physical robot deployment [45], which is outside the scope of this study. The evaluation setting here differs fundamentally: rather than measuring task execution, the study evaluates offline visuomotor trajectory prediction, measuring how closely each configuration's predicted wrist pose sequences align with ground-truth demonstrated trajectories in the held-out test set. Results should therefore be interpreted as trajectory prediction quality under each branch and world model configuration, not as claims about downstream task success. The relationship between offline trajectory prediction quality and online task performance is addressed in Section 3.4 of the literature review and in Section 4.7.

4.6.2 Primary metrics

The action representation throughout this study is a sequence of $\mathrm{SE}(3)$ wrist transforms for both hands (see Section 4.1). Evaluation metrics are defined over this representation directly, decomposed into translation and rotation components.

Translation L2 error. For each predicted action chunk, the Euclidean distance between the predicted and ground-truth wrist positions is computed at each of the 16 prediction steps and averaged across the chunk and across both hands. This yields a mean per-step translation error in metres, providing an interpretable measure of spatial accuracy over the prediction horizon. Translation L2 is used as the primary evaluation metric in related offline benchmarks [7, 9].

Rotation error. The geodesic distance between the predicted and ground-truth wrist orientations is computed at each step, expressed in degrees. As with translation, values are averaged across the chunk and across both hands. Geodesic distance on $\mathrm{SO}(3)$ is used in preference to Euler-angle difference, which is sensitive to gimbal singularities and axis ordering [45].

The following supplementary metrics are computed alongside the two primaries and reported for completeness. All are expressed as pairwise win rates against the B1 baseline unless stated otherwise.

- **Rotation win rate.** Fraction of windows where the selected candidate achieves lower rotation error than B1; the rotational counterpart to the translation win rate.
- **Endpoint accuracy win rate.** Win rate computed over the mean translation error of the final four prediction steps only, capturing whether selection quality holds through to the end of the prediction horizon.
- **Velocity win rate.** Fraction of windows where the selected candidate's step-to-step velocity profile (first-difference of wrist positions) is closer to the ground-truth velocity profile than B1's selection.
- **Acceleration win rate.** Same as velocity win rate, computed on the second-difference (acceleration) of the wrist position sequence.
- **Straightness win rate.** Fraction of windows where the selected candidate's trajectory is geometrically closer to the ground-truth path shape, penalising unnecessary curvature.
- **Oversmoothing win rate.** Fraction of windows where the selected candidate avoids excessive temporal smoothing relative to the demonstrated trajectory; complements straightness by penalising insufficient motion variation.
- **Nonlinear acceptance rate (B2 only).** Fraction of test windows in which the B2 veto gate fires, i.e. the world model's preferred candidate differs from the policy's and its margin exceeds the threshold. This is a diagnostic for B2 rather than a quality metric; it characterises how often the world model actively intervenes.

4.6.3 Candidate selection comparison

Comparing headline metric values directly across world model families is complicated by the fact that the five families produce scores on different numerical scales, and that absolute metric differences may reflect policy rather than world model quality. To support clean cross-family comparison, candidate selection is also evaluated via a pairwise win rate: for each test window, the candidate selected by a given branch or world model family is compared against the candidate selected by B1 (policy-only), and a win is recorded if the selected candidate achieves lower translation L2 on the ground-truth trajectory; translation L2 is used as the win criterion because it provides the most spatially direct and interpretable measure of selection quality across the 16-step prediction horizon. Win rates are averaged across all test windows (Table 4.1) and reported as percentages. This measure is scale-free and directly answers the question of whether world-model involvement produces a better selection decision than the policy alone, independent of the absolute metric level. The offline evaluation literature supports pairwise comparison as a robust ranking method [44].

4.7 Evaluation Scope and Limitations

The following scope constraints and known limitations apply to the experimental design.

Offline evaluation only. No physical robot is used and no task-completion signal is available; all metrics are computed against held-out demonstrated trajectories (Section 4.6.1). Results should be read as evidence about candidate selection quality in the offline trajectory prediction setting, not as claims about downstream robot task success.

Kinematic action space and embodiment. The action representation is the bimanual $\mathrm{SE}(3)$ wrist space described in Section 4.4, with two consequences for scope:

- *No finger, contact, or force signals.* Finger-level articulation, contact forces, and haptic feedback are absent; tasks whose outcome depends on fine-grained in-hand manipulation or contact dynamics lie outside the evaluation scope.
- *Human-to-robot kinematic gap.* Demonstrations are produced by human subjects, whose wrist kinematics differ from robot manipulators in range of motion and joint compliance. Downstream deployment would require bridging this embodiment gap.

Training budget as a compute-constrained lower bound. The universal 100k-item budget (Section 4.5.1) is not a saturation point for all families; some were still

improving at this scale. Performance differences may partly reflect differing distances from respective saturation budgets rather than intrinsic architectural differences.

DexWM trained from scratch. Published DexWM weights are unavailable [34]; the architecture was trained from scratch on the benchmark splits, without the domain-specific EgoDex and DROID pretraining it was designed to exploit (Section 5.5.3). Results for this family should be interpreted accordingly.

5 Implementation

This chapter describes the concrete implementation decisions made to realise the experimental design specified in Chapter 4. Eight concerns structure the account:

- repository layout and execution model (Section 5.1);
- data pipeline and windowing infrastructure (Section 5.2);
- consolidated training configuration for all model families (Section 5.3);
- policy training, ACT and Diffusion Policy (Section 5.4);
- world model training, all five families (Section 5.5);
- convergence study and universal training budget (Section 5.6);
- B2 margin threshold calibration and guard evaluation infrastructure, together with the fairness safeguards built into it (Sections 5.7–5.9);
- implementation challenges that required significant diagnosis and correction (Section 5.10).

5.1 Repository and execution model

The research stack is organised as a single Python package under `src/`, with subsystem entry points exposed under `scripts/`. The canonical shell surface follows a per-model naming convention: `policy.sh`, `jepa_wm.sh`, `lewm.sh`, `dexwm.sh`, and one `<model>.sh` per additional model family, complemented by the higher-level orchestration wrapper `run_stage.sh`. Docker is the canonical runtime surface for all stages: datasets, caches, and run outputs are kept outside the repository checkout so that code, data, and generated artefacts remain separated. All reported results are outputs of this staged execution model, tied to frozen run artefacts rather than to manually reconstructed notebook states.

The run output directory follows a split-first, then model layout. Each training job writes its artefacts under `fair_training/<split>/<model>/`, which includes a `result.json` recording the completion status, wall time, and final metric. This layout makes it straightforward to audit which combinations have completed and to compare results across splits for the data-scaling analysis.

5.2 Data pipeline and windowing

5.2.1 Index construction

The index stage is the foundation of the entire stack. It discovers valid paired EgoDex episodes, requiring both a `.mp4` video file and a `.hdf5` pose annotation file, infers

episode lengths from the Hierarchical Data Format 5 (HDF5) metadata, and writes canonical observation and prediction windows. The fixed-stride index is configured as `obs16_pred16_s128` following the window protocol described in Section 4.1.2: 16-frame observation windows followed by 16-frame prediction windows, sampled every 128 frames. All training, world model extraction, and guard evaluation in this study consume this windowing contract. The index stage writes resumable checkpoints to allow interrupted builds to continue from the last completed episode.

As established in Section 4.2, the shared window definition ensures any measured performance difference is attributable to branch design or architecture rather than to differences in the data each component received.

5.2.2 Incremental training curriculum

Three cumulative training splits were assembled from the first three parts of the EgoDex release, as detailed in Table 4.1 of the methodology. Each split is a strict superset of the previous: the larger splits were assembled using relative symbolic links rather than copying data, allowing incremental construction without duplicating approximately 1 TB of earlier data on disk.

This curriculum allows the data-scaling axis of the benchmark to be evaluated: all five world model families and both policy architectures are trained independently on each of the three splits at the universal budget of 100,000 effective items (Section 5.6), enabling a direct measure of how additional training data diversity affects prediction quality independent of training exposure volume.

5.3 Training configuration

Table 5.1 consolidates the hardware and batch configuration for all model families at the universal budget $E = 100{,}000$. Settings marked *(shared)* are kept identical across all trainable models. The effective-item budget is verified for each trainable family via $E = S \times B_{\text{global}}$, where S is the number of optimisation steps and B_{global} is the effective batch per step. V-JEPA2 (no AC) and PointWorld require no model training and reuse the Phase 1 extraction shared with V-JEPA2-AC.

Table 5.1 Training configuration for all model families at $E = 100{,}000$. Approximate step time is shown for trainable phases only; Phase 1 of V-JEPA2-AC is a one-time offline extraction with no optimiser step. V-JEPA2 (no AC) and PointWorld reuse the V-JEPA2-AC Phase 1 embeddings and require no additional training (––– denotes not applicable).

Family	GPUs	Batch/device	Steps S	B_{global}	s/step
ACT	2×A100	32	1,562	64	≈0.5
Diffusion Policy	1×A100	32	3,125	32	≈2.5
V-JEPA2-AC Ph. 1 (extraction)†	4×A100	—	—	—	—
V-JEPA2-AC Ph. 2 (AC head)	4×A100	32	781	128	<0.01
LeWM	4×A100	4	6,250	16	≈0.047
DexWM	3×L40S	4	12,500	12	≈28
V-JEPA2 (no AC)	—	—	—	—	—
PointWorld	—	—	—	—	—

† Phase 1 is a one-time offline extraction (4×A100, sharded, 27.4 windows/s, no backward pass). Measured wall times: `train_part1` ≈0.75 h, `train_part1_2` ≈1.4 h, `train_part1_2_3` ≈2.2 h. Phase 2 head training completes in under ten seconds per split (≈0.01 s/step on cached embeddings). DexWM total wall time across all three training splits was approximately 130 h (≈28 s/step × 12,500 steps × 3 splits); LeWM total wall time was approximately 15 minutes (≈0.047 s/step), substantially faster than DexWM despite the identical effective budget.

5.4 Policy training

Both policy families share the same observation encoder: a ResNet-18 visual backbone [48] processes the single observation RGB frame (96×96 pixels) through four residual stages (64, 128, 256, and 512 channels), producing a 512-dimensional feature vector that is concatenated with the bimanual wrist state before being passed to the policy-specific decoder (Figure 5.1). Image resizing to 96×96 is applied at loading time and is consistent across all branches. Wrist pose sequences are normalised to zero mean and unit variance using statistics computed from the training split; these statistics are frozen after training and consumed by the world model scoring stages to ensure consistency.

Action Chunking Transformer

The ACT architecture is reviewed in Section 3.1 and illustrated in Figure 3.1. The implementation follows the architecture and training procedure of Zhao et al. [7] with no dependency on an external codebase. Custom components include the EgoDex HDF5 window loader, the 18-dimensional bimanual wrist action space adapter, and a checkpoint serialisation layer that freezes the CVAE configuration and normalisation statistics alongside the model weights. The CVAE encoder-decoder and transformer

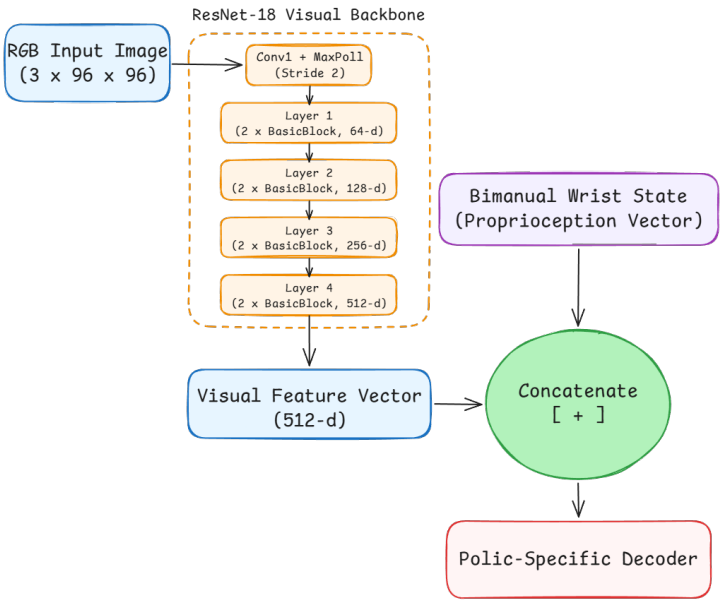


Figure 5.1 Shared ResNet-18 visual backbone preceding both policy decoders. Adapted from [48].

decoder are realised directly from the paper specification. At training time, the CVAE encoder compresses a ground-truth action chunk together with the current observation into a style variable $z \sim \mathcal{N}(\mu, \sigma^2)$; the transformer decoder reconstructs the chunk from z and the observation. The encoder is discarded at evaluation time, and z is drawn from the standard Gaussian prior $\mathcal{N}(0, I)$. The design intent is to generate up to N diverse candidates by sampling independent z vectors and decoding each under the same observation. Training uses two A100 GPUs with a per-device batch of 32 (global batch 64): $E = 1{,}562 \times 64 \approx 100{,}000$ (Table 5.1). Each run writes a `best.pt` checkpoint selected by lowest validation loss, a `last.pt` checkpoint, per-epoch metrics in `metrics.jsonl`, and a frozen `manifest.json` recording the configuration and window file consumed.

CVAE posterior collapse: investigation and retraining. The trained ACT checkpoint exhibits CVAE posterior collapse: at inference time the decoder is latent-insensitive, and sampling five independent z vectors from $\mathcal{N}(0, I)$ produces five identical action chunks. A diagnostic confirms the extent of the collapse: injecting $z = 10\sigma$ (ten standard deviations from the prior) via a forward pre-hook on the latent projection layer shifts the output by approximately 3×10^{-7} in absolute value, which is effectively zero. The mean pairwise difference across five independently drawn candidates is 0.000000 on all three training splits.

This is a well-documented failure mode of conditional VAE models rather than a mis-implementation [49]. When the conditioning signal (the visual observation) is sufficiently informative to reconstruct the target without consulting z, the decoder learns to ignore z entirely. The KL divergence term in the training objective regularises

the *encoder* distribution toward the prior but cannot compel the *decoder* to use the latent; the collapse is therefore not resolved by increasing the KL weight. Three retraining runs were conducted to test this:

- *v1*: `kl_weight=10` (default) — collapse unchanged.
- *v2*: `kl_weight=100` — weighted KL contribution ($100 \times 0.25 \approx 25$ nats) exceeded reconstruction loss (~ 15), yet collapse persisted.
- *v3*: cyclical KL annealing [50] (`kl_cycle_batches=200`, `kl_cycle_ramp_frac=0.5`) and 30 % observation dropout to force decoder reliance on z — collapse persisted.

All three runs produced mean pairwise differences indistinguishable from zero on every split (Table 5.2). The EgoDex visual observations are sufficiently informative that the decoder finds a loss minimum that ignores z regardless of training pressure, and correcting the collapse without an architectural change (discrete latents, vector-quantisation, or an explicit diversity head) is outside the scope of this thesis.

Table 5.2 ACT CVAE posterior collapse: three successive retraining attempts. `kl_weight`, cyclical annealing schedule, and observation dropout were varied in sequence; all runs produced a mean pairwise candidate difference of zero on every training split.

Run	`kl_weight`	Cyclical KL	Obs. dropout	Mean pairwise diff
v1	10 (default)	no	0 %	0.000000
v2	100	no	0 %	0.000000
v3	100	yes (200-batch cycle, 50 % ramp)	30 %	0.000000

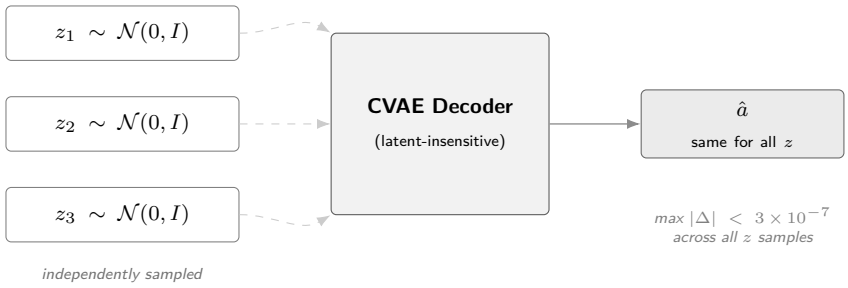


Figure 5.2 CVAE posterior collapse in the trained ACT checkpoint. Three independently sampled latent vectors z_1, z_2, z_3 are passed to the CVAE decoder (dashed arrows indicate the latent path the decoder effectively ignores). All inputs produce an identical output $\hat{a}$; the decoder has learned to reconstruct the action from the visual observation alone, making the latent redundant.

The collapse is treated as an empirical finding. It motivates the use of an external candidate generator, described below, and is reported in Section **??** as a characteristic of the ACT policy on this dataset.

CEMP candidate augmentation. To restore the five-candidate evaluation pool, ACT-based guards use a CEMP planner: an adaptation of the Cross-Entropy Method [51] initialised from the policy's output rather than a uniform prior, following the model-based planning approach of TD-MPC2 [38]. A perturbation is a small SE(3) displacement applied to the base action chunk, shifting each wrist pose by a sampled translation and rotation offset while preserving temporal coherence across the 16-step chunk. Starting from the collapsed ACT B1 output, three iterations each draw 64 such perturbations around the current mean, score them with the world model's latent prediction function, and retain the top 25 % (16 elites); the standard deviation decays by $0.7\times$ per iteration. The four highest-scoring candidates across all iterations are returned (Figure 5.3).

For Branch 2 the pool contains these four alternatives plus the ACT B1 output ($N{=}5$); for Branch 3 the B1 output is excluded, leaving the four alternatives only. Because the same world model scores candidates inside CEMP and performs the final selection, the B2 and B3 results measure the quality of the combined world-model-plus-planner system rather than the world model's scoring ability in isolation.

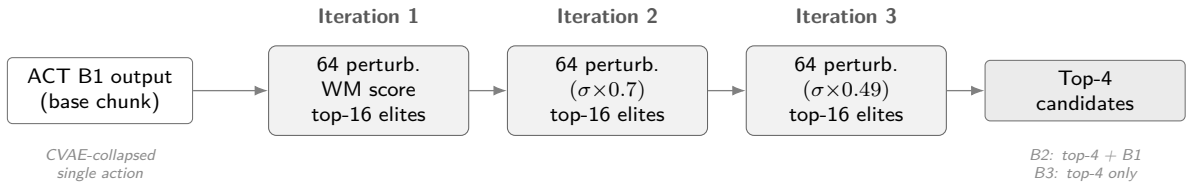


Figure 5.3 CEMP candidate augmentation. Three iterations of the Cross-Entropy Method initialised from the ACT B1 output refine SE(3) perturbations; the standard deviation decays by $0.7\times$ per iteration. The four highest-scoring candidates form the B2/B3 evaluation pool.

Diffusion Policy

The Diffusion Policy implementation follows the transformer-backbone variant (DP-T) described in Section 3.2 and illustrated in Figure 3.2. The implementation follows Chi et al. [9] with no external codebase dependency. Custom components include the 18-dimensional bimanual wrist action space adapter, the EgoDex HDF5 window loader, and the five-candidate DDIM evaluation protocol (five independent noise seeds decoded under the shared conditioning signal). The same ResNet-18 observation encoder produces a conditioning vector `obs_cond` passed to a DiT denoising backbone via cross-attention. At evaluation time, five independent DDIM denoising chains are run on the same observation to produce five candidate action chunks, each starting from independently sampled Gaussian noise and converging to a distinct trajectory

under the shared conditioning signal. Training uses one A100 GPU with a per-device batch of 32: $E = 3{,}125 \times 32 = 100{,}000$ (Table 5.1). The same checkpoint, metrics, and manifest artefacts as ACT are written at the end of each run.

DDIM subsampling correction.

$\bar{\alpha}_t$	cumulative noise schedule product at step t (List of Notations)
t_{prev}	next step in the subsampled inference schedule (List of Notations)
$\hat{x}_0$	predicted clean sample (List of Notations)
x_T	Gaussian noise at the schedule endpoint (List of Notations)

An implementation audit identified an error in the reverse-diffusion sampler [47]: the original implementation passed $\bar{\alpha}_{t-1}$ (the consecutive index in the training schedule) instead of $\bar{\alpha}_{t_{\text{prev}}}$:

$$\text{Bug: } \bar{\alpha}_{t-1} \quad \longrightarrow \quad \text{Fix: } \bar{\alpha}_{t_{\text{prev}}} \tag{5.1}$$

For a ten-step schedule $t \in \{90, 80, \ldots, 0\}$ this substitutes $\bar{\alpha}_{89}$ for the correct $\bar{\alpha}_{80}$ at the first denoising step, miscalibrating the $\hat{x}_0$ weighting throughout. The diversity guarantee is unaffected: each candidate begins from an independent $x_T \sim \mathcal{N}(0, I)$, so the collapsed-latent failure mode present in ACT cannot arise. The fix supplies the explicit next-in-schedule index to `ddim_step`; all three evaluation splits are re-run under the corrected sampler and the corrected results supersede the initial runs.

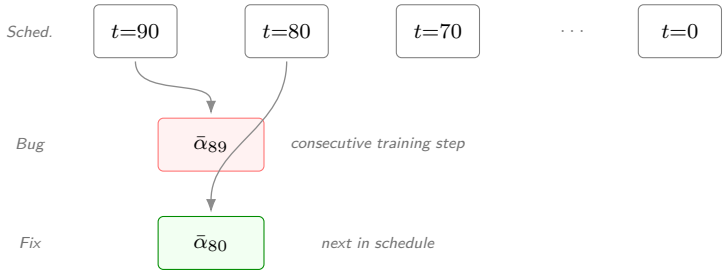


Figure 5.4 DDIM subsampling correction. The ten-step inference schedule spans $t \in \{90, 80, \ldots, 0\}$. At each reverse step the correct $\bar{\alpha}_{t_{\text{prev}}}$ is the *next timestep in the subsampled schedule*; the bug substituted the consecutive training-schedule timestep $\bar{\alpha}_{t-1}$, miscalibrating the $\hat{x}_0$ weighting at every step.

5.5 World model training

5.5.1 V-JEPA2-AC: extraction and transition head

The V-JEPA2-AC architecture is reviewed in Section 3.3.2. Figure 3.6 illustrates the standard V-JEPA2 backbone alongside the action-conditioned variant that forms Phase 2. The frozen ViT-Large backbone is loaded via PyTorch Hub

(facebookresearch/vjepa2) and cached locally; the action-conditioned transition head is implemented and trained from scratch. Training separates into two phases.

In Phase 1, the frozen ViT-Large encoder processes all indexed windows in parallel across four A100 GPUs via a sharded extraction pipeline. The encoder takes 16-frame observation sequences and produces a latent embedding vector per window; these are written to `embeddings.npy` together with a companion `window_ids.txt` recording the exact identifier (split, task, episode, t_0) of each row. Phase 1 is a one-time offline extraction: there is no backward pass, no optimiser, and no epoch loop. A critical bug in an early run applied a 2,000-window default cap producing only approximately 1,000 training pairs from a split of 140,000 windows; this is documented in Section 5.10.

In Phase 2, a small MLP transition head is trained on the stored embeddings to predict the next embedding z_{t+1} from the current embedding z_t and an action conditioning vector derived from the bimanual wrist state delta (`action_repr=policy_state_delta`). Training runs on four A100 GPUs: $E = 781 \times 128 = 99{,}968 \approx 100{,}000$ (Table 5.1). Wall-clock time is under ten seconds because each step is a small MLP forward–backward pass with no backbone inference. The trained head is exported via an `act_adapter.json` contract loaded by the guard stage at runtime.

5.5.2 LeWM: end-to-end training

LeWM is reviewed in Section 3.3.2 and illustrated in Figure 3.8. The architecture is implemented following the published specification [35] and trained entirely from scratch with no external weights. Training jointly optimises the ViT-Tiny encoder, the autoregressive predictor, the action embedder, and the projector from random initialisation using the next-embedding prediction loss and the SIGReg regulariser (Equation 3.1) that enforces a Gaussian prior on the latent distribution [35].

EgoDex episodes are exported into contiguous HDF5 sequences containing pixel frames, per-step bimanual wrist state, per-step action (state delta), and per-episode length metadata. Two distinct export paths exist with different frame caps. The training export (`_extractions/lewm_export/`) uses `max_frames_per_episode=128` (the default in `lewm_export_episodes.py`). The guard/evaluation export (`evaluation/lewm_test_export/`) uses `max_frames_per_episode=5280`, derived from the maximum window start position across splits (up to frame 5,280 in `train_part1_2`). An earlier evaluation-export cap of 256 frames caused 97.9 per cent of test windows to be silently excluded; the full resolution of this defect is documented in Section 5.10.

LeWM training runs on four A100 GPUs with a per-device batch of 4 (global batch 16, no gradient accumulation): $E = 6{,}250 \times 16 = 100{,}000$ (Table 5.1). Wall-clock

training time on `train_part1_2_3` was approximately 4.9 minutes per run (≈0.047 s/step × 6,250 steps), for a total of approximately 15 minutes across all three training splits, reflecting the lightweight ViT-Tiny + MLP architecture relative to DexWM.

A normalisation statistics defect discovered during the evaluation audit caused the guard stage to load `action_mean` and `action_std` from the test-set export rather than the training-set export, introducing a 1.74× scale mismatch in the LeWM cost function across all initial guard runs. The full description and resolution are documented in Section 5.10.

5.5.3 DexWM: from-scratch training

DexWM is reviewed in Section 3.3.2. Its pipeline pairs a frozen DINOv2 image encoder with a CDiT predictor and incorporates a hand-consistency loss that supervises fingertip and wrist heatmap predictions (Figure 5.5). The DINOv2 ViT-Large encoder is loaded from the `facebookresearch/dinov2` HuggingFace repository; the CDiT predictor and HC head are implemented from the published architecture [34] and trained from scratch.

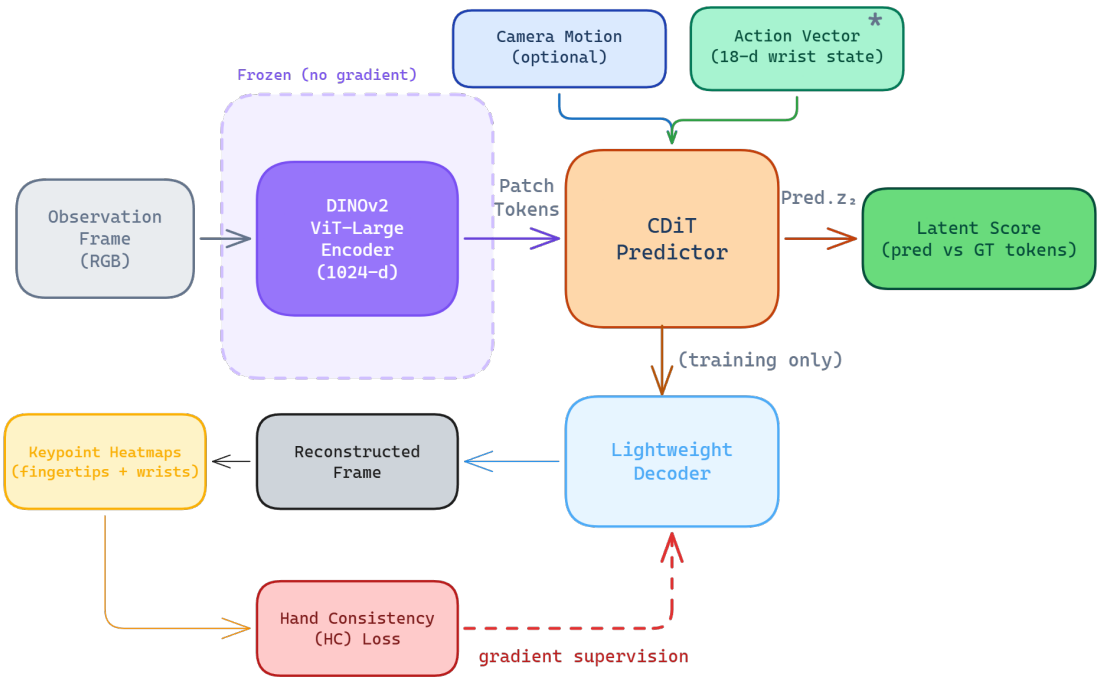


Figure 5.5 DexWM pipeline as implemented, comprising a frozen DINOv2 encoder, CDiT predictor, lightweight decoder, and HC loss supervision path. Adapted from [34].

Checkpoint sourcing investigation

The published DexWM architecture [34] does not provide publicly available model weights. The associated HuggingFace repository (`facebook/dexwm`) contains only trajectory data in `.hdf5` format; no weight file is present. The GitHub repository README states that pretraining requires 32 nodes of 8 GPUs each for several days on the EgoDex and DROID datasets, and does not link to a downloadable checkpoint. Automated download attempts confirmed a 404 response for all candidate weight paths. The codebase initially contained a silent fallback that substituted an identity stub (`_CDiTStub`) returning zero scores for all candidates without any user-visible error. This stub was replaced with an explicit hard-failure preventing any experiment from silently using zero-score predictions.

Because no external checkpoint could be sourced, DexWM was trained from scratch on the benchmark's declared training splits, adopting the same provenance stance as LeWM: no external pretraining and no data outside the declared splits. The DINOv2 ViT-Large encoder (1,024-dimensional patch tokens) remains frozen; the CDiT predictor and HC head are trained from random initialisation. An early run misconfigured the ViT-Base variant (768-dimensional), causing all guard-stage inference to fail silently; see Section 5.10.

DexWM training runs on three L40S GPUs with a per-device batch of 4 (global batch 12). The step count was doubled to 12,500 to compensate for the reduced global batch relative to the original four-GPU design, yielding: $E = 12{,}500 \times 12 = 150{,}000$ (Table 5.1). Wall-clock training time was approximately 43 hours per split (≈28 s/step), for a total of approximately 130 hours across all three training splits, reflecting the substantially higher GPU memory footprint of the patch-level spatial representations relative to LeWM's global embedding approach.

An evaluation audit subsequently found that the initial generic guard runs for DexWM loaded the ACT Branch 1 output as the sole candidate ($N = 1$), making the world model's scoring a vacuous no-op. The canonical DexWM evaluation uses a dedicated pipeline that draws eight retrieved training demonstrations per window; the full description is in Section 5.10.

5.5.4 V-JEPA2 (no action conditioning): ablation control

The methodological role of this ablation is described in Section 4.4. In implementation, the variant is realised by reusing the Phase 1 embeddings extracted for V-JEPA2-AC; no additional training is required (Table 5.1). At guard time, the scorer passes a zero action vector to the transition model for all candidates, producing action-blind scores that reflect visual plausibility alone. A dedicated result artefact records the persistence

loss (mean MSE plus 0.1$\times$ cosine distance between consecutive frame embeddings, using the same weighted objective as the AC head training loss) as a diagnostic metric. This loss is computed directly as the weighted distance between consecutive stored frame embeddings from `embeddings.npy` — no transition model is invoked for this diagnostic, confirming the world model makes no action-differentiated prediction when conditioned on a zero action vector.

5.5.5 PointWorld: action-degradation control

The methodological role of the PointWorld control is described in Section 4.4. The implementation uses the same frozen V-JEPA2-AC backbone and transition head as the V-JEPA2-AC family, requiring no additional model training (Table 5.1). The sole difference is that candidate action chunks are degraded before being passed to the transition head.

A `HandToArmActionAdapter` extracts the six left-wrist rotation dimensions (rot6d, dims 0–5) and computes a scalar summary as the mean of the remaining twelve dimensions (left translation, right rotation, right translation) via the `wrist_plus_gripper` strategy, then zero-pads the result back to 18 dimensions. The right-hand pose is collapsed into the scalar summary dimension and not recoverable from the output; it does not survive as a distinguishable signal in the degraded representation. The transition head therefore receives the correct visual features but systematically corrupted action conditioning.

The convergence study diagnostic additionally computes a proxy loss:

$$\mathcal{L}_{\text{PointWorld-proxy}} = \mathcal{L}_{\text{persist}} + 0.5 \cdot \text{mismatch} \tag{5.2}$$

where $\mathcal{L}_{\text{persist}}$ is the frame-to-frame embedding distance and $\text{mismatch} = 1 - \|\mathbf{arm}\|/\|\mathbf{hand}\|$ is the fraction of action-vector norm discarded by the projection (clipped to $[0, 1]$). This diagnostic confirms that the adapter introduces genuine information loss: the bimanual hand action space does not map faithfully to the 7-DOF arm representation, as expected for a degradation control.

5.6 Convergence study and universal training budget

As described in Section 4.5.1 of the methodology, a convergence study was conducted prior to full fair training to determine an appropriate shared exposure budget. Each of the five primary model families was trained independently at six data scales (1k, 5k, 10k, 25k, 50k, and 100k effective items) on a fixed stratified subset of `train_part1_2_3`. The two control families (V-JEPA2 no-AC and PointWorld) were

included in the sweep but excluded from the budget selection decision.

5.6.1 Loss curves and plateau detection

Table 5.3 reports the validation loss at each scale for all seven families. Each row uses a different training objective so raw values are not comparable across families; the relevant signal is each family's own trend across scales. Bold entries mark the earliest scale at which a family's loss saturates (improvement below 0.5 per cent relative to the next scale).

Table 5.3 Validation loss across six training scales for all seven model families. Bold marks each family's first scale of saturation; losses are objective-specific and not cross-family comparable.

Family	1k	5k	10k	25k	50k	100k
ACT	**0.134**	0.134	0.129	0.133	0.130	0.129
Diffusion	10.86	5.22	4.25	3.11	2.57	2.14
DexWM	4.31	2.41	2.00	1.90	1.69	1.36
LeWM	0.335	0.209	0.165	0.145	**0.130**	0.130
V-JEPA2-AC	1.796	0.375	0.308	0.286	0.257	0.227
V-JEPA2 (no AC)	0.460	0.477	0.373	0.352	0.353	0.377
PointWorld	0.655	0.679	0.628	0.629	0.632	0.592

The bold entries identify the two families that saturate within the sweep: ACT is essentially flat from 1k onwards (a 4 per cent improvement over a 100-fold data increase), and LeWM plateaus between 50k and 100k (0.5 per cent improvement). V-JEPA2-AC, Diffusion Policy, and DexWM carry no bold entry because all three are still declining at 100k, as Figure 5.6 shows. The two controls show no consistent downward trend, consistent with their respective proxy designs having no learnable relationship with additional data.

5.6.2 Universal budget decision

The plateau budget per primary family is summarised in Table 5.4. Given that Diffusion Policy, DexWM, and V-JEPA2-AC all continue to improve at 100k, the universal budget is set to $E = 100{,}000$ as a compute-constrained lower bound. ACT and LeWM, which plateau earlier, receive the same 100k budget by design: the extra exposure cannot hurt them and ensures all families are compared under the most favourable common condition reachable within the compute envelope.

This budget is a compute-constrained lower bound rather than a true saturation point for all families. The performance gaps between families reported in the results

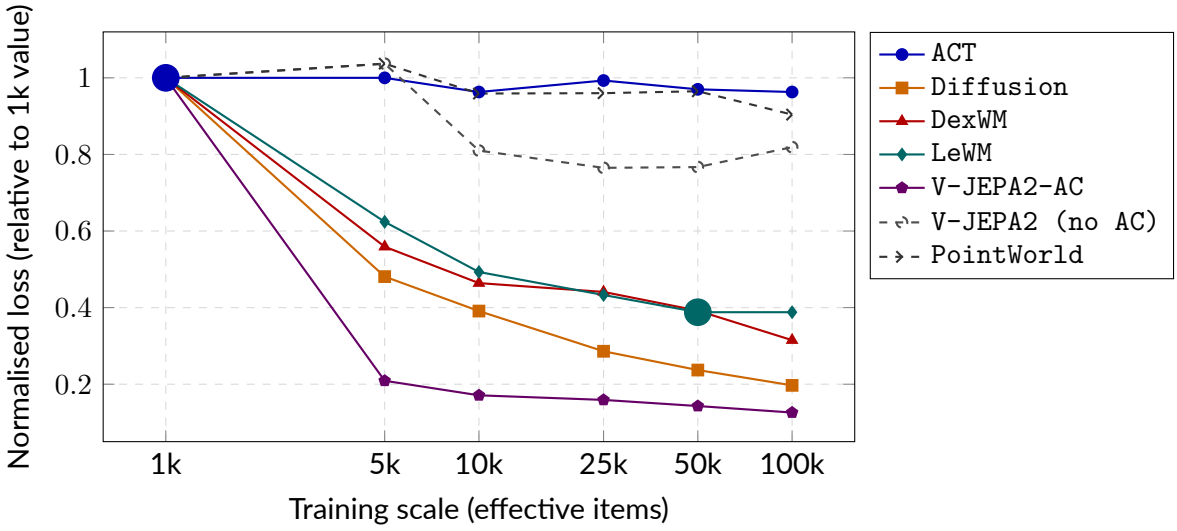


Figure 5.6 Normalised validation loss (each family's value divided by its 1k loss) across six training scales. Solid lines are primary models; dashed lines are controls. Large filled circles mark saturation points matching the bold entries in Table 5.3. Families with no marker are still declining at 100k.

Table 5.4 Plateau budget per primary model family. The universal budget is $E = 100{,}000$ effective items.

Family	**Plateau budget**	**Evidence**
ACT	1k	Flat from 1k; 4% drop over 100$\times$ more data
LeWM	50k	50k$\rightarrow$100k improvement of 0.5%; effectively converged
V-JEPA2-AC	$>$100k	11% drop at 50k$\rightarrow$100k; still declining
Diffusion	$>$100k	17% drop at 50k$\rightarrow$100k; still declining steeply
DexWM	$>$100k	20% drop at 50k$\rightarrow$100k; still declining
Universal budget		$\boldsymbol{E = 100{,}000}$

chapter may partly reflect differing distances from their respective saturation budgets rather than intrinsic architectural differences. This limitation is acknowledged in Section 4.7 of the methodology.

5.7 B2 margin threshold calibration

The Branch 2 guard includes an optional margin threshold δ: the world model's override is accepted only when its preferred candidate outscores the policy output by at least δ. Setting $\delta = 0$ yields unconditional override on every disagreeing window; higher values suppress overrides and collapse B2 toward B1.

A development sweep over $\delta \in \{0.25, 0.50, 1.00\}$ was run on the `train_part1_2` validation split using retrieval-based candidate pools for the V-JEPA2-AC guard. The results followed a clear monotone pattern: at $\delta = 0.25$, approximately 21% of windows were overridden with the largest loss improvement relative to B1; at $\delta = 1.00$, the override rate fell below 0.2% and only 6% of potential gain was recovered.

The canonical evaluation adopts $\delta = 0$ (`selection_mode: best_score`, `override_margin=0.0`), driven by two considerations:

1. **CEMP renders a post-hoc gate partially redundant** (Section 5.9). The internal CEM scoring loop already pre-filters candidates toward world-model-preferred actions before the guard selects; adding a margin gate would reintroduce a hyperparameter that varies across model families and pool types.
2. **Unconditional argmax isolates discriminative quality directly.** Measuring the world model's preference without a threshold suppression layer produces a cleaner experimental signal and avoids per-family calibration.

The sweep results are retained for transparency but do not contribute to any reported result. DexWM was excluded from the sweep: its initial guard runs were found to be vacuous due to a candidate-loading defect (Section 5.10); the canonical DexWM evaluation uses the dedicated `guards_native_dexwm/` pipeline (Section 5.8).

5.8 Guard evaluation

The three-branch framework is defined in Section 4.2; the implementation realises it without bespoke deviation.

5.8.1 Branch 1: policy-only baseline

No world model is called. For ACT, CVAE posterior collapse (Section 5.4) yields a single deterministic chunk, which is recorded directly. For DP, B1 selects the chain with the

lowest denoising residual across the five independent samples.

5.8.2 Branch 2: world-model-inclusive selection

The guard scores the five-candidate pool with `selection_mode: best_score` and no margin override. Each world model uses its canonical latent scorer: DexWM via its CDiT patch-token predictor; LeWM by encoding a rollout prefix to establish latent context before scoring each candidate; PointWorld via the proxy function described in Section 5.5.5. V-JEPA2 without action conditioning cannot produce candidate-specific predictions and reduces to B1 by default.

An evaluation audit (Section 5.10) found that V-JEPA2-AC, V-JEPA2 (no AC), and PointWorld initially used retrieval-based pools rather than CEMP; all three were re-evaluated with `candidate_source=planner` to enforce a consistent candidate source across all families.

5.8.3 Branch 3: world-model-exclusive selection

All five world model families use the same generic pipeline: four CEMP alternatives per window with `selection_mode: world_model_only`. Manifest correctness is enforced by verifying that every B3 run records all 7,607 windows under `world_model_only_best_non_act`, confirming the policy chunk is absent. This invariant was used to detect and correct the selection-mode error documented in Section 5.10.

5.8.4 Manifest and checkpoint design

Every guard run emits a frozen manifest at completion recording the selection mode, candidate source, aggregate score statistics, and per-reason override counts. Guard runs also write per-window JSONL checkpoints flushed after each window, allowing interrupted runs to be resumed from the last completed window via a `--resume-dir` flag rather than restarted from scratch.

5.9 Evaluation safeguards

Oracle goal elimination. In early experiments, the latent goal used for scoring was derived from the ground-truth future frame of each episode, giving Branches 2 and 3 access to information unavailable at deployment time. This was corrected by setting `goal_source=none` in all canonical benchmark runs. All results in the results chapter use this corrected configuration.

Candidate pool invariant. Both Branch 2 and Branch 3 score five-candidate pools generated independently per window by the CEMP planner. Branch 2 pools contain the ACT output plus four planned alternatives; Branch 3 pools contain four planned alternatives only. The key controlled variable between the two branches is pool membership (policy output present vs. absent), not the number of candidates, which is five for both. The pool-size correctness of every B3 run is verified through the guard manifest: all 7,607 override-reason records must read `world_model_only_best_non_act` with zero policy fallbacks. This invariant was used to detect the Branch 3 selection-mode error documented in Section 5.10.

Shared comparison surface. The `policy_chunk_compare` stage computes all cross-branch and cross-family metrics on the same prediction targets rather than relying on controller-native summaries, ensuring that translation L2 and rotation error values are computed identically across all configurations. The evaluation window-count shortfall documented in Section 5.10 was detected by checking `num_windows` in each manifest against the expected 7,607.

Test-split isolation. The test split was held out throughout all stages of the project. All training, hyperparameter selection, threshold calibration, and qualitative development work used training and validation splits only.

Candidate source standardisation. An audit conducted after the initial evaluation runs found that V-JEPA2-AC, V-JEPA2 (no AC), and PointWorld used retrieval-based candidate pools (`candidate_source=retrieval`) while LeWM used CEMP-planned alternatives (`candidate_source=planner`). Because the candidate source is a controlled variable in the B2/B3 comparison, a source asymmetry confounds cross-model comparisons: differences between families could reflect candidate quality rather than world model quality. All five families were therefore re-run with `candidate_source=planner` before any results were reported. The original retrieval-based runs are archived under `*_retrieval_archived/` subdirectories and are not used in this thesis. The full audit is documented in Section 5.10.

5.10 Implementation challenges and resolutions

Thirteen significant implementation challenges arose during the project and were diagnosed and resolved before the final benchmark results were reported. The first seven concern training and infrastructure; the remaining six emerged during the evaluation audit and affected the canonical guard evaluation results.

V-JEPA2 extraction window cap.

- *Symptom:* Default 2,000-window extraction cap produced approximately 1,000 training pairs from a split of 140,000 windows (95-fold shortfall).
- *Impact:* All V-JEPA2-AC transition head runs under the capped extraction were invalid.
- *Fix:* Added `--full` flag to the extraction entry point; re-runs confirmed results consistent with convergence study predictions.

LeWM CPU inference.

- *Symptom:* Hard-coded `.to("cpu")` in the model-loading code placed the guard model on CPU despite available GPU access.
- *Impact:* Estimated guard run time of approximately 54 hours for 7,607 test windows at five candidates each.
- *Fix:* CUDA availability check at load time; model and all tensors moved to GPU. Throughput improved to approximately 19.5 windows per second, reducing guard run time to under seven minutes.

LeWM episode coverage defect.

- *Symptom:* Initial `max_frames_per_episode=256` export caused windows with start positions beyond frame 256 to be silently excluded.
- *Impact:* First LeWM evaluation covered only 160 of 7,607 intended test windows (2.1 per cent coverage).
- *Fix:* Frame cap computed from actual benchmark window lists (5,280 frames); export and training rebuilt; coverage raised to 100 per cent.

LeWM CPU-only training shortfall.

- *Symptom:* Omission of `--train-accelerator gpu --train-devices 4` flags caused training to run on a single CPU.
- *Impact:* Four-fold effective-item shortfall relative to the intended budget.
- *Fix:* Flags added; all affected runs invalidated and re-run with the correct configuration.

DexWM guard dimension mismatch.

- *Symptom:* DINOv2 ViT-Base (768-dimensional) loaded instead of ViT-Large (1,024-dimensional); guard inference failed with a dimension error.
- *Impact:* Silent fallback stub produced zero scores for all candidates, masking the failure in guard outputs.
- *Fix:* Silent fallback replaced with explicit hard-failure; correct ViT-Large configuration confirmed throughout.

Branch 3 selection mode error.

- *Symptom:* Branch 3 launched without `--selection-mode world_model_only`; default `best_score` mode combined policy and world model scoring.

- *Impact:* Branch 3 produced output identical to Branch 2 across all 7,607 windows; detected via a 0.000 win rate in pairwise comparison.
- *Fix:* Re-ran Branch 3 with correct flag; verified via manifest override count (7,607 world-model decisions, 0 policy fallbacks).

HDF5 file handle exhaustion.

- *Symptom:* LeWM guard opened `episodes.h5` inside the per-window scoring function, producing approximately 38,000 open/close operations per test-set guard run.
- *Impact:* Risk of descriptor exhaustion on systems with low file-handle limits.
- *Fix:* File opened once during runtime initialisation; handle held for the duration of the run.

Diffusion Policy DDIM subsampling error.

- *Symptom:* The DDIM reverse step consumed $\bar{\alpha}_{t-1}$ (consecutive training-schedule index) instead of $\bar{\alpha}_{t_{\text{prev}}}$ (previous index in the subsampled inference schedule). A ten-step schedule spanning $t \in \{90, 80, \ldots, 0\}$ therefore applied $\bar{\alpha}_{89}$ at the first step rather than the correct $\bar{\alpha}_{80}$.
- *Impact:* Miscalibrated $\hat{x}_0$ weighting at every reverse step; marginally degraded sample quality. Candidate diversity was unaffected because each of the five chains still starts from an independently sampled x_T.
- *Fix:* Commit `92e8116` passes the explicit next-in-schedule index to `ddim_step`. All three Diffusion Policy evaluation splits were re-run with the corrected sampler; the corrected results supersede the initial runs. Full description in Section 5.4.

PointWorldScorer stale constructor kwargs.

- *Symptom:* `guard.py` instantiated `PointWorldScorer` with keyword arguments `predictor=` and `latent_dim=` that had been removed in an earlier API refactor, raising a `TypeError` at guard startup.
- *Impact:* All PointWorld guard runs failed to launch until the constructor call was corrected.
- *Fix:* Stale kwargs removed; constructor updated to the current API. Fix included in commit `92e8116`.

`_s_sq_cache` memory-pointer collision.

- *Symptom:* The scorer base class cached score scale factors in a dictionary keyed by the Python object's memory address (`id()`). After garbage collection, a newly allocated tensor could reuse the same address as a previously cached object, causing stale scale factors to be broadcast against the new tensor.
- *Impact:* Incorrect guard scores beginning at approximately window 1,700 of a run, producing silent result corruption without any raised exception.

- *Fix:* Cache keyed on an explicit run-scoped string identifier rather than the object address. Fixed in commit `89e2af6`.

Evaluation window count shortfall.

- *Symptom:* Initial guard launch scripts omitted `--eval-ratio 0.0` and `--max-eval-batches 100000`, causing the evaluation loop to exit after the default 787-window limit.
- *Impact:* Only 787 of the 7,607 intended test windows were evaluated (10.3 per cent coverage); results were statistically unreliable and not representative of the full test set.
- *Fix:* Flags added to all guard launch scripts. Coverage confirmed via `num_windows` in the run manifest against the expected 7,607.

DexWM vacuous single-candidate evaluation.

- *Symptom:* The generic `b2_dexwm` and `b3_dexwm` guard runs loaded the ACT Branch 1 output as the sole candidate ($N = 1$). Because the CVAE is fully collapsed, this candidate is deterministic; the DexWM world model therefore scored a pool of one and its selection was always the trivially forced choice.
- *Impact:* The guard results were equivalent to Branch 1 regardless of the DexWM model quality; the world model's discriminative capacity was never exercised. Detected by inspecting `candidate_count_mean: 1.0` in the run manifest.
- *Fix:* Generic DexWM guard runs archived under `b2_dexwm_archived/` and `b3_dexwm_archived/`. Canonical DexWM evaluation uses the dedicated `guards_native_dexwm/` pipeline, in which Branch 3 draws eight retrieved training demonstrations as candidates and scores them with the CDiT predictor (Section 5.8).

LeWM normalisation statistics from wrong export.

- *Symptom:* The `action_mean` and `action_std` tensors consumed by the LeWM guard at runtime were read from the test-set export rather than the training-set export. The test-set action standard deviation (≈ 0.014) differed from the training-set value (≈ 0.024) by a factor of approximately 1.74.
- *Impact:* The LeWM cost function operated on incorrectly scaled action representations throughout every candidate scoring call, distorting the world model's preference ordering in a dataset-split-dependent way.
- *Fix:* Guard configuration updated to read normalisation statistics from the training export. Fixed in commit `d188816`; all three LeWM guard runs re-executed under the corrected normalisation.

Candidate source asymmetry across world model families.

- *Symptom:* An audit of the initial B2/B3 evaluation runs found that V-JEPA2-AC, V-JEPA2 (no AC), and PointWorld used `candidate_source=retrieval` (expert

demonstration neighbours), while LeWM used `candidate_source=planner` (CEMP-generated alternatives). The candidate source is a controlled variable in the B2/B3 design: it determines what alternatives the world model evaluates, not only which model scores them.

- *Impact:* Cross-family comparisons were confounded: differences between, say, V-JEPA2-AC and LeWM could reflect candidate quality rather than world model quality. The three retrieval-based runs are archived under `*_retrieval_archived/` subdirectories.
- *Fix:* All five world model families re-run with `candidate_source=planner` (CEMP). The standardised results are reported in the results chapter; see also Section 5.9.

5.11 Auxiliary modules

Two auxiliary modules are fully implemented in the repository but were excluded from the final benchmark evaluation.

The semantic alignment stage converts indexed windows into captions, structured tags, and text embeddings. The pipeline uses a Vision Language Model (VLM) to generate free-form descriptions and task-relevant categorical tags for each window, followed by a separate embedding model that maps the text to a shared representation space. A critic-and-retry loop rejects low-quality outputs and requests regenerated captions.

The object grounding and mask generation stage consumes semantic object prompts from the alignment stage and produces bounding box detections and segmentation masks for identified objects. The current implementation provides pixel-space localisation of objects but does not estimate $\mathrm{SE}(3)$ object pose.

Both modules were excluded from the primary benchmark because a shared input contract across all five world model families could not be maintained: the semantic and grounding outputs could not be wired into the frozen DINOv2 and ViT-Large encoder paths used by DexWM, V-JEPA2-AC, and their shared embedding reuse families, violating the fairness requirement established in Section 4.5.2. Their implementation is documented in Appendix A.

These modules pre-date the current benchmark scope. They were developed when the initial research direction explored semantic feature enrichment as a pre-training pipeline for world models, before the study evolved toward a comparative evaluation of published architectures across a standardised five-family design. The initial intuition that frozen encoder pre-extraction followed by lightweight head training is a productive approach has since been validated by the architectures of DexWM, DINOv2-based world models, and related state-of-the-art systems, each of

which adopts an analogous two-phase structure. This line of work remains a natural direction for continued research beyond the scope of the current results.

6 Experimental Design and Metrics

6.1 Canonical evaluation surface

The canonical evaluation surface for this dissertation is the final `test` benchmark package based on the `train_part1_2_3` training split, with Branch 3 V-JEPA2-AC correctly configured to use world-model-only candidate selection. This package evaluates all five branch instances on the held-out `test` split of 7,607 windows. All quantitative claims in the results chapter are drawn from this package.

Historical evidence from earlier packages remains useful for understanding how the final benchmark was reached and why certain design decisions were made. However, it is supporting material rather than the main results surface. The hierarchy described in Chapter 4 governs when earlier packages can be cited and prevents the thesis from treating bounded pilot comparisons as equivalent to the full evaluation surface.

6.2 Frozen branch representatives

The canonical package combines:

1. Branch 1: ACT policy trained on `train_part1_2_3`, evaluated on the `test` split without world model support.
2. Branch 2 V-JEPA2-AC: V-JEPA2-AC world model and AC transition head trained on `train_part1_2_3`, guard run with `selection_mode=best_score`.
3. Branch 3 V-JEPA2-AC: Same V-JEPA2-AC world model, guard re-run with `selection_mode=world_model_only`, verified by manifest inspection confirming all 7,607 windows decided by world model.
4. Branch 2 LeWM: LeWM model fitted on `train_part1_2` at `max_frames_per_episode=5280`, guard run in replay mode.
5. Branch 3 LeWM: Same LeWM model, guard run in candidate-world-model mode.

This construction is a frozen reporting surface, not a pure same-data ablation across all five branches. Branch comparisons should therefore be read as properties of the assembled package, with the understanding that the within-family matched comparisons (B2 versus B3 within V-JEPA2-AC; B2 versus B3 within LeWM) are the most internally controlled because they share the same underlying model weights.

6.3 Prediction targets and their significance

The benchmark evaluates predictions of bimanual wrist SE(3) poses over a 16-step horizon. Each prediction window targets the 16 frames following the observation window, representing approximately 0.53 seconds of future wrist motion at 30 frames per second.

Translation prediction measures how closely the predicted wrist position sequence matches the demonstrated wrist position sequence. Translation is the spatial position component of each SE(3) pose, expressed in world-frame coordinates. This is the primary metric because spatial position is the most directly relevant property for placing a hand at the correct location in a manipulation task. Translation L_2 is computed as the mean Euclidean distance between predicted and target positions across all 16 prediction steps and both hands.

Rotation prediction measures how closely the predicted wrist orientation sequence matches the demonstrated orientation. Rotation is expressed as the angular distance in degrees between predicted and target orientations (computed from the SE(3) rotation component). A policy that predicts the correct wrist position but with incorrect orientation may still fail to grasp or manipulate objects correctly. Rotation error therefore provides an important secondary view of trajectory quality.

Transforms (per-step 6-DoF poses) are available in the EgoDex annotation but are not used as a separate evaluation target in this benchmark. Translation and rotation are extracted and evaluated separately because they have different physical interpretations and different scales. Combining them into a single transform metric would require a weighting choice that is not theoretically grounded.

[FIGURE PLACEHOLDER: prediction targets illustration — 16-step observation window ending at t0 + obs_len, 16-step prediction window, translation L2 computed across both hands and all 16 steps, rotation error in degrees]

Shape metrics measure geometric properties of the predicted trajectory rather than pointwise distance to the target. These metrics assess whether the predicted wrist motion follows a plausible kinematic path, independently of whether it reaches exactly the right position. Five shape metrics are computed:

Path **straightness** measures whether the trajectory moves toward its endpoint without unnecessary detours. A highly straight path avoids oscillations and backtracking, which are common failure modes of chunked imitation policies that overfit to local features. A win on straightness means the predicted path is more direct than the baseline.

Oversmoothing (trajectory jitter) measures whether the trajectory avoids high-frequency oscillations around its mean path. Excessive jitter indicates that the policy is uncertain about the exact position at each step and oscillates between

plausible candidates. A win on oversmoothing means the predicted path is smoother.

Velocity consistency measures whether the speed of the predicted wrist motion across the 16 steps matches the demonstrated speed profile. A win on velocity means the predicted trajectory moves at a more appropriate speed.

Acceleration consistency measures whether changes in wrist speed are consistent with the demonstrated acceleration profile. This is distinct from velocity because a trajectory can have the right average speed but still accelerate and decelerate at the wrong moments.

Turning point count measures whether the number of direction reversals in the predicted trajectory matches the demonstration. Too many turning points indicates an oscillating trajectory; too few indicates a trajectory that is spatially smooth but misses important curvature events in the demonstrated motion.

Endpoint translation measures the translation L_2 error over only the final quarter of the prediction window (steps 12 through 16). This provides a measure of how accurately the trajectory arrives at the correct final position, which is often more relevant to task success than accuracy at the first prediction step.

[FIGURE PLACEHOLDER: shape metrics illustrated on an example trajectory — two wrist paths shown (predicted and target), annotations showing straightness, jitter, velocity profile, and turning points]

6.4 Branch design and the five evaluated instances

The five evaluated branch instances correspond to three branch structures and two world model families, as defined in Chapter 4. The key distinctions are:

Branch 1 uses `selection_mode=policy_only`: the ACT policy's internal scoring selects one of five candidates. This is the baseline.

Branch 2 uses `selection_mode=best_score`: a combined scoring function including the world model's predicted latent distance, an action out-of-distribution penalty, and a support distance term ranks the five candidates and selects the highest scorer. ACT's own preference enters through the combined scoring function.

Branch 3 uses `selection_mode=world_model_only`: only the world model's predicted latent distance ranks the candidates. ACT's internal candidate quality is not considered. This tests whether the world model alone, without any policy guidance in the selection step, can identify better trajectories.

[FIGURE PLACEHOLDER: branch selection logic comparison — same five ACT candidates enter all three branches; Branch 1 uses policy's own ranking; Branch 2 uses combined score; Branch 3 uses WM score only; arrows and decision labels]

6.5 Fair comparison rules

The main model family comparison between V-JEPA2-AC and LeWM follows a strict shared input contract. Both families receive the same raw RGB observation frame and the same bimanual wrist state as inputs. Oracle goal leakage is excluded from both families (`goal_source=none`). Both families are evaluated on exactly the same 7,607 test windows. The candidate pool used by Branch 3 is the same frozen set used by Branch 2, ensuring that differences in Branch 3 outcomes reflect differences in world model selection, not differences in the candidates available.

Cross-family metrics use the shared `policy_chunk_compare` surface, which recomputes chunk-level translation, rotation, and shape quantities on the same prediction targets. This is necessary because the native V-JEPA2-AC guard reports immediate-step translation error (first prediction step only) while the native LeWM guard reports mean chunk translation error, and a direct comparison of these two quantities would be invalid.

6.6 Metric harmonisation

The branch summary table in the results chapter is derived from the shared `policy_chunk_compare` surface rather than from controller-native metric logs. This harmonisation step is methodologically important: it means the branch table represents a common chunk-level reporting surface derived from paired branch and comparison artefacts, not a naive juxtaposition of incompatible native summaries.

The translation L_2 values in the branch table are therefore mean chunk translation errors (mean across all 16 prediction steps and both hands), computed on the same candidate chunks across all five branches. The rotation error values are mean angular distances across the same windows and steps.

6.7 Claims policy

The thesis adopts a strict claims policy governing what conclusions can and cannot be drawn from the benchmark results.

Claims that are supported by the evidence include: the implemented offline benchmark shows that world model support materially improves chunk-level trajectory quality; Branch 3 V-JEPA2-AC achieves the best aggregate mean translation error and the highest win rate against Branch 1; Branch 2 V-JEPA2-AC shows greater window-by-window consistency in the direct Branch 3 versus Branch 2 comparison; both LeWM branches improve on Branch 1; neither LeWM branch approaches the

V-JEPA2-AC family; the improvement is robust across confidence strata and task categories; path geometry improves substantially while temporal dynamics do not.

Claims that are not supported include: lower offline translation L_2 implies higher task completion rate; wrist trajectory quality is sufficient to characterise manipulation success; the WACT architecture generalises to non-EgoDex domains without retraining; semantic alignment and object grounding are unnecessary in general; the scaling finding is universal across architectures.

6.8 Metric justification

Translation L_2 is not established as the official evaluation metric in the EgoDex dataset paper. The justification for its use as a primary offline quality proxy rests on two lines of evidence from the manipulation evaluation literature. SimplerEnv [43] demonstrates that simulation-based offline evaluation reliably predicts real-world manipulation performance when control and visual disparities are managed, providing support for offline trajectory metrics as meaningful proxies for deployed quality. WorldEval [44] directly proposes world-model-based offline evaluation, showing strong correlation between offline world-model trajectory rankings and real-world task success. Together these establish a literature precedent for offline trajectory metrics as policy quality proxies in the manipulation domain. All results in this thesis are interpreted within the acknowledged limitation that no paper establishes this connection specifically for EgoDex wrist trajectories, and no claim of task completion rate improvement is made.

6.9 Scaling experiment design

The scaling experiment tests whether expanding from `train_part1_2` (39 tasks) to `train_part1_2_3` (63 tasks) improves the pretrained world model family. Both training splits were evaluated on the same held-out test split of 7,607 windows. The ACT policy, V-JEPA2-AC extractor, and LeWM model were all retrained from scratch on the larger split. No weights were transferred from the smaller split.

The results show that the `train_part1_2_3` training did not improve the V-JEPA2-AC family (Branch 2 V-JEPA2-AC: 0.0344 under `train_part1_2` versus 0.0354 under `train_part1_2_3`). The Branch 1 ACT-only baseline improved very slightly (0.1439 to 0.1429), while Branch 3 LeWM was essentially unchanged (0.0906 to 0.0909). These findings are interpreted as evidence of capacity saturation under the current architecture rather than a fundamental ceiling on world model guidance in general, as discussed in Chapter 8.

7 Results

All results in this chapter are drawn from the canonical final benchmark package based on the `train_part1_2_3` training split with Branch 3 V-JEPA2-AC correctly configured to use world-model-only selection. The evaluation surface is the held-out `test` split of 7,607 windows covering 63 task categories. Guard manifest verification confirmed that all 7,607 Branch 3 V-JEPA2-AC windows were decided by the world model rather than the policy (`override_reason_counts: {world_model_only_best_non_act: 7607}`).

The results are organised into four analytical perspectives: branch comparison, world model family comparison, path shape and trajectory quality, and scaling behaviour.

[FIGURE PLACEHOLDER: summary overview figure — four panels showing the four analytical perspectives, directing the reader to the subsections that follow]

7.1 Branch comparison

Table 7.1 presents the branch-level metrics for all five evaluated branch instances. Translation L_2 and rotation error are computed over all 16 prediction steps using the harmonised `policy_chunk_compare` surface, ensuring cross-family comparability.

Table 7.1 Final benchmark branch summary. Training split: `train_part1_2_3` (63 tasks, 195k episodes). Evaluation: 7,607 held-out `test` windows. Translation L_2 and rotation error are chunk-level means over all 16 prediction steps.

Branch	**Translation $L_2 \downarrow$**	**Rotation $(^\circ) \downarrow$**	**Δ vs B1 (translation)**
B1 (ACT only)	0.1429	41.4°	—
B2 (V-JEPA2-AC hybrid)	0.0354	15.8°	-75.2%
B3 (V-JEPA2-AC world-model-only)	**0.0328**	**15.2°**	$\mathbf{-77.0\%}$
B2 (LeWM hybrid)	0.1179	31.9°	-17.5%
B3 (LeWM)	0.0909	27.5°	-36.4%

The table supports three immediate observations. First, both V-JEPA2-AC branches substantially improve on the ACT-only anchor across both translation and rotation, reducing mean translation L_2 by 75 and 77 per cent respectively. Second, both LeWM branches also improve on Branch 1, confirming that world model guidance is not contingent on a pretrained backbone. Third, the V-JEPA2-AC family is decisively stronger than the LeWM family across all matched branch comparisons.

Table 7.2 presents the pairwise win rates. These measure how often one branch produces a trajectory closer to the demonstration than the other on the same test

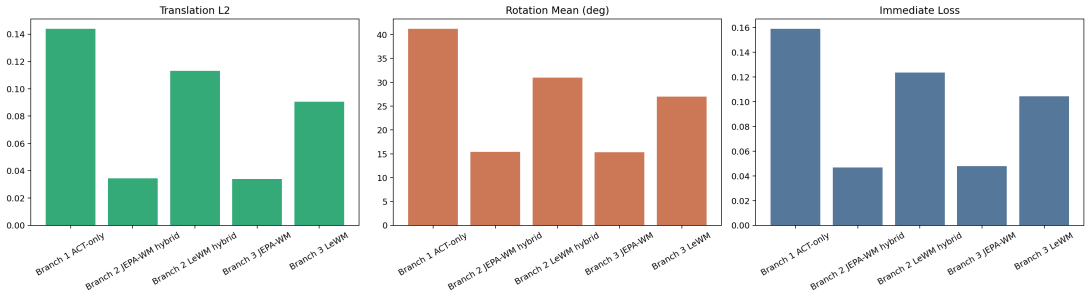


Figure 7.1 Branch-level benchmark metrics. Left: mean translation L_2. Right: mean rotation error in degrees. Both are chunk-level means over 16 prediction steps across 7,607 test windows. Lower is better.

window.

Table 7.2 Selected pairwise win rates. Translation win rate: fraction of 7,607 windows where the candidate branch achieves lower translation L_2 than the reference branch.

Comparison	**Windows**	**Translation win rate**
B3 V-JEPA2-AC vs B1 ACT	7,607	**91.5%**
B2 V-JEPA2-AC vs B1 ACT	7,607	86.6%
B3 V-JEPA2-AC vs B2 V-JEPA2-AC	7,607	14.6%
B3 LeWM vs B1 ACT	7,607	71.7%
B3 LeWM vs B2 V-JEPA2-AC	7,607	17.4%
B2 LeWM vs B2 V-JEPA2-AC	7,607	12.9%

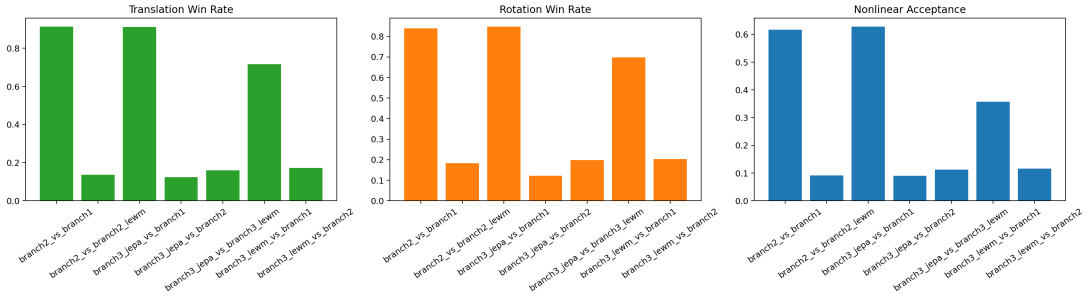


Figure 7.2 Pairwise win rates from the final `test` package. Each bar shows the fraction of 7,607 windows where the listed branch outperforms the reference branch on translation L_2. The dashed line marks 50 per cent.

The pairwise results confirm the branch-level reading. Both V-JEPA2-AC branches dominate Branch 1 strongly and consistently. Branch 3 V-JEPA2-AC achieves the highest win rate against Branch 1 (91.5 per cent) while Branch 2 achieves 86.6 per cent. The LeWM family beats Branch 1 at 71.7 per cent (Branch 3) and substantially behind V-JEPA2-AC on matched comparisons (12.9 to 17.4 per cent).

7.2 Branch authority: world-model-driven versus hybrid

The direct comparison between Branch 3 V-JEPA2-AC and Branch 2 V-JEPA2-AC warrants careful interpretation. Branch 3 achieves a better aggregate mean translation L_2 (0.0328 versus 0.0354) and a higher win rate against Branch 1 (91.5 versus 86.6 per cent). However, in the direct head-to-head comparison, Branch 2 outperforms Branch 3 on 85.4 per cent of individual test windows (Branch 3 wins only 14.6 per cent).

This is not a contradiction. It arises because Branch 3 makes large improvements on a subset of windows where pure world model authority produces substantially better trajectories, while performing marginally worse than Branch 2 on the majority of windows where the combined hybrid score and the pure world model score point to different candidates. Branch 3 has a higher variance of per-window improvement: it wins less often but wins by larger margins when it does win, resulting in a better aggregate mean.

[FIGURE PLACEHOLDER: per-window difference distribution between B3 and B2 V-JEPA2-AC — histogram of (B3 translation L2 minus B2 translation L2) per window; showing the right-skewed distribution where B3 wins large and loses small]

The aggregate mean improvement favours Branch 3, while the window-by-window consistency favours Branch 2. Both are genuine properties of the two branches, and both are reported here. The broader implication is that full world model authority is beneficial when the world model is reliable enough to override the policy on the windows where they disagree, but the hybrid design offers more predictable per-window behaviour.

7.3 World model family comparison

The performance gap between the V-JEPA2-AC and LeWM families is substantial and consistent across branches. At the Branch 3 level, V-JEPA2-AC (0.0328) improves over LeWM (0.0909) by a factor of approximately 2.8 in mean translation L_2. In direct head-to-head comparisons, the matched LeWM branch beats the matched V-JEPA2-AC branch in only 12.9 to 17.4 per cent of windows.

This gap reflects the structural difference between the two strategies. V-JEPA2-AC inherits a 300-million-parameter visual representation pretrained on over one million hours of internet video, providing rich spatiotemporal priors that transfer immediately to EgoDex frames. LeWM learns its 15-million-parameter representation entirely from EgoDex data. The approximately 20 percentage point gap in win rate against Branch 1 (91.5 versus 71.7 per cent) is consistent with the pretrained representation advantages reported by R3M [30] and MVP [52] in the manipulation

learning literature.

LeWM achieving 71.7 per cent against the ACT-only baseline from scratch is itself a meaningful result. It confirms that world model guidance can improve trajectory quality without any external pretraining corpus, and that LeWM is a viable world model family for this task. The gap with V-JEPA2-AC reflects the pretrained backbone advantage, not a failure of the from-scratch strategy.

[FIGURE PLACEHOLDER: V-JEPA2-AC versus LeWM family comparison — bar charts showing win rates and L2 values for all four world-model branches, grouped by family; dashed line at B1 baseline level]

7.4 Path shape and trajectory geometry

Beyond aggregate translation and rotation error, the benchmark records five path shape metrics for each evaluated window, comparing the predicted trajectory's geometric properties against the ground-truth demonstration. Table 7.3 presents these results for the Branch 2 V-JEPA2-AC versus Branch 1 comparison, which has the largest sample size and the most consistent improvement.

Table 7.3 Path shape win rates for Branch 2 V-JEPA2-AC versus Branch 1 ACT-only (7,607 windows). Win rate: fraction of windows where the V-JEPA2-AC branch produces a trajectory with better shape by the given metric.

Shape metric	Win rate	Interpretation
Straightness	89.4%	V-JEPA2-AC strongly improves path linearity
Oversmoothing	93.1%	V-JEPA2-AC strongly reduces trajectory jitter
Velocity	56.5%	Near chance; temporal dynamics largely unchanged
Acceleration	50.1%	Near chance; temporal dynamics largely unchanged
Turning points	48.9%	Near chance; curvature events not improved

[FIGURE PLACEHOLDER: radar or bar chart of the five shape metric win rates for B2 V-JEPA2-AC vs B1 — showing high geometric metrics and near-chance temporal metrics]

The shape results reveal an important asymmetry in what world model guidance improves. V-JEPA2-AC selection strongly improves path geometry: trajectories are more spatially linear (89.4 per cent) and have substantially less jitter (93.1 per cent). However, world model guidance has almost no effect on temporal dynamics: velocity profiles (56.5 per cent) and acceleration profiles (50.1 per cent) are near chance, and turning point preservation (48.9 per cent) is slightly below chance.

The interpretation is that the V-JEPA2 latent space encodes spatial plausibility more reliably than temporal dynamics. The world model can identify which candidate

trajectory ends in the right place and follows a spatially coherent path, but cannot reliably identify which candidate moves at the right speed or decelerates at the right moments. This is consistent with the temporal entanglement limitation identified by Tsagkas et al. [33], who find that pretrained representations can conflate observations from different task stages.

7.5 Robustness across confidence strata and tasks

The benchmark records per-joint confidence scores for each EgoDex demonstration frame. Table 7.4 presents translation win rates for Branch 2 V-JEPA2-AC versus Branch 1 stratified by wrist annotation confidence.

Table 7.4 Confidence stratification for Branch 2 V-JEPA2-AC versus Branch 1 ACT-only. High confidence: wrist confidence score ≥ 0.85. Low confidence: wrist confidence score < 0.85.

Window type	**Count**	**% of test**	**Translation win rate**
High confidence (≥ 0.85)	5,957	78%	88.0%
Low confidence (< 0.85)	1,650	22%	81.6%

The 6.4 percentage point gap between confidence strata is small relative to the overall improvement. V-JEPA2-AC guidance adds value across the full confidence distribution and is not inflated by being evaluated primarily on high-confidence, reliably-annotated frames. This is an important robustness finding: even when the ground-truth annotation is less reliable (low-confidence windows, where tracking may have been interrupted by fast motion or occlusion), the world model selection still produces substantially better trajectories than the ACT baseline.

Per-task analysis across all 111 unique task-category windows in the test split confirms that the improvement is broad rather than concentrated in a small number of tasks. The five worst-performing task categories still show win rates in the range of 62 to 66 per cent against Branch 1, well above chance. No single task cluster drives the 86.6 per cent aggregate headline.

[FIGURE PLACEHOLDER: per-task win rate distribution for B2 V-JEPA2-AC vs B1 — histogram of win rates across 111 task categories; vertical lines at 50%, 75%, and 86.6% (aggregate) to show spread and floor]

7.6 Scaling experiment

The `train_part1_2_3` training split was assembled after the April 14 package to test whether expanding from 39 to 63 task categories improved the pretrained world

model family. Table 7.5 presents the comparison.

Table 7.5 Scaling experiment: Branch-level translation L_2 on the `test` split under two training scales. Lower is better.

Branch	`train_part1_2`	`train_part1_2_3`	Δ
B1 ACT-only	0.1439	0.1429	-0.001
B2 V-JEPA2-AC hybrid	0.0344	0.0354	$+0.001$
B3 LeWM	0.0906	0.0909	≈ 0

The scaling experiment shows that expanding training data from 39 to 63 task categories did not improve the V-JEPA2-AC world model. Branch 2 V-JEPA2-AC regressed slightly (0.0344 to 0.0354), while Branch 1 ACT-only and Branch 3 LeWM were essentially unchanged. The slight V-JEPA2-AC regression is consistent with the additional 24 task categories in `train_part3` introducing mild distributional shift relative to the test split's 26 represented tasks, without providing enough additional overlap to compensate. Crucially, the win rate against Branch 1 remains above 86 per cent across all three training splits (approximately 91 per cent on `train_part1` and `train_part1_2`, and 86.6 per cent on `train_part1_2_3`), confirming that the world model benefit is not an artefact of any single training data volume.

[FIGURE PLACEHOLDER: scaling experiment bar charts — B1, B2 JEPA, B3 LeWM translation L2 under train_part1_2 and train_part1_2_3; showing flat or slightly negative scaling for V-JEPA2-AC]

This finding suggests that the V-JEPA2-AC family is near a capacity ceiling at the current policy architecture scale. The pretrained V-JEPA2 backbone already provides rich representations that do not benefit substantially from seeing more EgoDex task variety. The lightweight AC head that learns from these frozen embeddings may similarly be near its capacity given the current policy size. A larger policy backbone (for example, replacing ResNet-18 with a ViT-Small or larger) might yield different scaling behaviour and is identified as future work in Chapter 9.

7.7 Why the canonical package supersedes the April 14 results

The April 14 full-coverage package was the first evaluation to achieve 7,607-window coverage for the LeWM family and constituted a significant advance over the March 2026 package. However, it cannot be treated as the final canonical results surface for two reasons.

First, Branch 3 V-JEPA2-AC in the April 14 package ran with `selection_mode=best_score` rather than `world_model_only`, making it scientifically

identical to Branch 2. The three-branch comparison was effectively a two-branch comparison. The thesis cannot claim a genuine Branch 3 result without the world-model-only correction.

Second, the April 14 package used `train_part1_2` trained models. The scaling experiment demonstrates that the transition to `train_part1_2_3` produced a small but real change in V-JEPA2-AC performance. Reporting only the April 14 numbers would require explaining why a later, larger training run was excluded.

The canonical results therefore use the `train_part1_2_3` package with the Branch 3 correction as the definitive reporting surface. The April 14 package remains useful as a reference point for the scaling experiment and for understanding the development trajectory of the benchmark.

8 Discussion and Limitations

8.1 What the benchmark shows

The central finding of the dissertation is that world model guidance materially and consistently improves offline trajectory quality in a controlled bimanual manipulation benchmark. This improvement holds across both evaluated world model families, both levels of world model authority, and across the full breadth of the test surface including lower-confidence windows and the worst-performing task categories.

The improvement is not marginal. The V-JEPA2-AC world model reduces mean translation L_2 by 77 per cent relative to the policy-only baseline and achieves a 91.5 per cent win rate in head-to-head window comparisons under world-model-only selection. Even the task-data-trained LeWM family, which has no access to any external pretraining, achieves a 71.7 per cent win rate from scratch. The effect of world model guidance is therefore robust to world model architecture.

At the same time, the results establish that the structure of the guidance matters. The two world model families differ by approximately 20 percentage points in win rate against the baseline, and the degree of world model authority (Branch 2 versus Branch 3) affects both the aggregate mean improvement and the window-by-window consistency of that improvement.

8.2 The aggregate versus per-window tension

The most interesting interpretive tension in the canonical results is between Branch 3 V-JEPA2-AC and Branch 2 V-JEPA2-AC. Branch 3 achieves a better aggregate mean translation L_2 (0.0328 versus 0.0354) and a higher win rate against Branch 1 (91.5 versus 86.6 per cent). However, in the direct head-to-head, Branch 2 outperforms Branch 3 on 85.4 per cent of individual windows.

This pattern arises because Branch 3 makes larger improvements on a smaller number of windows, while Branch 2 makes smaller but more consistent improvements across a larger fraction of windows. Branch 3's aggregate advantage comes from winning by large margins on the windows where full world model authority produces substantially better trajectories; on the majority of windows, the combined hybrid score in Branch 2 is a more reliable guide.

Neither metric is more correct than the other. The aggregate mean measures the typical improvement in expected prediction error. The pairwise win rate measures how often one branch produces a closer trajectory than another on a given window. Both are reported because they provide complementary views of the same phenomenon.

The broader implication is that full world model authority is beneficial when the world model is reliable enough to systematically prefer better candidates. In the V-JEPA2-AC case, the pretrained backbone provides rich enough representations that world-model-only selection outperforms hybrid selection in aggregate, even though the hybrid design is more consistent window-by-window. A world model with weaker representations (such as LeWM) shows the opposite pattern: Branch 3 LeWM achieves a better aggregate mean than Branch 2 LeWM (0.0909 versus 0.1179), again because concentrated improvements on a subset of windows more than compensate for losses on others.

8.3 What the path shape results reveal

The shape metric results in Table 7.3 provide a mechanistic interpretation of what the world model is and is not correcting. V-JEPA2-AC guidance strongly improves spatial trajectory geometry: path straightness (89.4 per cent win rate) and oversmoothing reduction (93.1 per cent). This indicates that the world model's latent space reliably encodes which candidate follows a spatially plausible path toward the goal.

By contrast, temporal dynamics are essentially unchanged. Velocity (56.5 per cent), acceleration (50.1 per cent), and turning point preservation (48.9 per cent) are all near chance. The world model selects trajectories that end in better places and follow straighter paths, but it does not select trajectories with better timing or curvature event profiles.

This is consistent with the temporal entanglement limitation identified by Tsagkas et al. [33]. Pretrained visual representations trained on diverse video data can encode where a scene is likely to lead without reliably encoding the precise dynamics of how it gets there. The V-JEPA2 backbone, pretrained on internet-scale video, provides strong spatial priors but weaker temporal dynamics priors. The AC head trained on top of these frozen embeddings inherits both the spatial strength and the temporal limitation.

This finding is practically informative. A downstream system that needs spatially correct wrist trajectories (for example, to ensure that the hand reaches the right region of space) would benefit substantially from V-JEPA2-AC guidance. A system that also needs temporally correct velocity profiles (for example, to synchronise with a moving target) would need additional mechanisms beyond the current world model architecture.

8.4 The family comparison and its architectural basis

The V-JEPA2-AC versus LeWM comparison is a comparison of two coherent research strategies rather than a fair ablation of model quality at fixed scale. V-JEPA2-AC uses a 300-million-parameter backbone pretrained on over one million hours of video, while LeWM trains a 15-million-parameter model from scratch on EgoDex. The pretrained backbone strategy carries a structural advantage that is not explained away by noting the architectural difference; the difference is the point.

The approximately 20 percentage point gap in win rate against the baseline (91.5 versus 71.7 per cent) is consistent with the advantage reported by R3M and MVP for pretrained egocentric video representations over task-specific training in robot manipulation [30, 52]. The gap is therefore not a WACT-specific artefact.

LeWM achieving 71.7 per cent from scratch is a meaningful result in its own right. It establishes that a compact, task-data-trained world model can provide useful guidance even without external pretraining, and that the LeWM architecture is a viable option in deployment contexts where large pretrained backbones are unavailable or unsuitable. The question of whether a pretrained variant of LeWM could close the gap is a valid future direction but would require architectural changes that would make the result non-comparable to the current benchmark.

8.5 Why benchmark evolution is part of the contribution

An important part of the dissertation's contribution is the development of a defensible evaluation surface, not only the final numerical rankings. The project progressed through three distinct benchmark eras: early bounded pilot slices used for controller selection, the March 2026 frozen three-branch bundle with limited LeWM coverage, and the final canonical package with full coverage and correct Branch 3 configuration.

Two corrections were necessary before the final results could be trusted as scientifically meaningful. The first was the LeWM coverage fix, which required recomputing the frame cap from actual benchmark window lists and rebuilding the export contract. The second was the Branch 3 selection mode correction, which required identifying through pairwise manifest inspection that two nominally distinct branches had produced identical outputs, diagnosing the root cause as a missing script flag, re-running with the correct flag, and verifying the result through manifest inspection.

Without both corrections, the reported conclusions would have been weaker and potentially misleading. An uncorrected LeWM result based on 160 windows would have left open the question of whether the V-JEPA2-AC advantage over LeWM was a

coverage artefact. An uncorrected Branch 3 result would have made the three-branch comparison a two-branch comparison and missed the finding that full world model authority produces a better aggregate mean than the hybrid design for the V-JEPA2-AC family.

8.6 Why semantic alignment and object grounding were excluded

The decision to exclude semantic alignment and object grounding from the final benchmark deserves explicit discussion because it might appear to weaken the result by removing potentially useful information from the evaluation.

Both modules are fully implemented and produce useful artefacts. They were excluded from the main benchmark because incorporating either into only one world model family's scoring would have given that family privileged inputs not available to the other. The V-JEPA2-AC family could in principle be conditioned on object grounding masks through the semantic extraction path, while the LeWM family has no equivalent semantic conditioning in its current architecture. Including such conditioning for V-JEPA2-AC alone would change the nature of the comparison from "which world model family provides better guidance given the same inputs" to "does additional semantic information help V-JEPA2-AC." These are distinct scientific questions. The thesis addresses only the former.

It remains entirely plausible that semantic conditioning would improve V-JEPA2-AC performance beyond the results reported here. That question is left as future work under a symmetric design where both families receive equivalent semantic inputs.

8.7 Main limitations

The first limitation is scope. The benchmark is offline and trajectory-level. It measures how closely predicted wrist motions match expert demonstrations. It does not measure task completion, object manipulation success, or any deployed behavioural outcome. The connection between offline trajectory L_2 and deployed task success is not established by this dissertation.

The second limitation is the wrist-only evaluation scope. EgoDex provides 138 joint transform sequences per frame, but the benchmark uses only the left and right wrist poses as both policy targets and evaluation targets. Wrist trajectory is a reasonable proxy for manipulation intent but does not capture finger articulation,

elbow configuration, or body posture. A policy that achieves lower wrist L_2 might still exhibit poor finger configuration or elbow alignment.

The third limitation is the offline-to-online gap. Even if offline trajectory quality is a reliable proxy for deployment success (as suggested by SimplerEnv and WorldEval), there is no deployment evaluation in the present system. The GRASP mixed-reality deployment project is designed to address this gap but its results are outside the scope of this dissertation.

The fourth limitation concerns the scaling finding. The scaling experiment tests only one expansion, from 39 to 63 task categories. Whether a larger backbone architecture would also saturate at this training scale, or whether the saturation is specific to the ResNet-18 and AC head configuration, cannot be determined from the current results alone.

The fifth limitation is metric coverage. The benchmark uses translation L_2, rotation error, and five path shape metrics. It does not yet include a single scalar notion of overall prediction quality that is theoretically grounded for non-linear manipulation trajectories. The multi-metric bundle provides useful triangulation but does not collapse to one interpretable number.

9 Conclusion

9.1 Summary of findings

This dissertation asked to what extent world model support improves an ACT policy across three implemented branches: policy-only control, hybrid ACT plus world model support, and world-model-driven candidate selection. The answer is clear and supported by a fully corrected, full-coverage evaluation surface.

World model guidance substantially improves offline trajectory quality. Under world-model-driven selection with the pretrained V-JEPA2-AC family, mean translation error is reduced by 77 per cent compared to the policy-only baseline, and the world model selects a closer trajectory than the policy alone in 91.5 per cent of head-to-head window comparisons across the 7,607-window test set. The hybrid selection strategy achieves 86.6 per cent, with greater window-by-window consistency. Both results hold across 63 task categories, across the full range of annotation confidence, and with no single task cluster driving the aggregate headline.

World model guidance does not depend on a pretrained backbone. The task-data-trained LeWM family achieves a 71.7 per cent win rate against the policy-only baseline despite learning all of its visual representations from EgoDex demonstrations alone. The pretrained backbone strategy nevertheless carries a decisive advantage of approximately 20 percentage points, consistent with the literature on pretrained visual representations for robot learning.

The shape analysis reveals that world model guidance improves where trajectories end and how spatially coherent their paths are, but does not substantially alter their temporal dynamics. Path straightness and oversmoothing improve to win rates of 89.4 and 93.1 per cent respectively, while velocity and acceleration profiles remain near chance. This points to a clear direction for future improvement.

Scaling the training data from 39 to 63 task categories did not further improve the pretrained world model, suggesting that the current ResNet-18 policy backbone and lightweight transition head are near a capacity ceiling at this training scope.

9.2 Methodological contribution

The second contribution of the thesis is the documented development of a defensible evaluation surface. Two significant benchmark defects were identified, diagnosed, and corrected during the project: a data coverage defect that limited one model family to 2.1 per cent of the intended test surface, and a selection logic error that caused two nominally distinct branches to produce identical outputs across all 7,607 windows.

Both corrections required non-trivial implementation work and altered the scientific interpretation of the results.

The thesis contribution therefore includes benchmark curation and the establishment of a trustworthy evidence hierarchy, not only the final numerical rankings. The process of identifying and correcting evaluation defects is part of the scientific work. A result that depends on a broken evaluation surface is not a valid result, and the effort required to ensure validity is as important to report as the result itself.

9.3 Scope of conclusions

The conclusions of this dissertation are explicitly bounded. They concern offline prediction quality on a held-out set of recorded demonstrations, not deployed task success. The system does not execute actions in any physical or simulated environment, and no task completion rates are reported. The improvement in trajectory quality relative to the policy-only baseline is a finding about offline imitation fidelity, which is a meaningful proxy for deployment quality under appropriate conditions but is not equivalent to it.

The thesis does not claim solved dexterous manipulation, solved action chunk fidelity, or closed-loop deployment success. It establishes a reproducible offline benchmark with a real policy and world model interface and a set of bounded empirical results that can support more targeted future work.

9.4 Future Work

The results and limitations identified in this dissertation suggest several concrete directions for future work.

The most immediate next step is online evaluation through the GRASP mixed-reality deployment project. GRASP deploys an ACT-based control loop in a live extended reality environment, providing the first opportunity to measure whether the improvements in offline trajectory quality found in this dissertation translate to improved task completion rates in a physical system. The connection between offline L_2 improvement and deployed task success is the primary open question left by the present work.

A larger policy backbone is the most directly motivated architectural extension. The scaling experiment shows that the current ResNet-18 backbone saturates with 63 training tasks. Replacing ResNet-18 with a ViT-Small or ViT-Base backbone would

substantially increase the visual capacity of the policy and might allow the AC transition head to learn more detailed action-conditioned dynamics from the larger training split.

Temporal dynamics improvement is suggested by the shape analysis. The world model currently improves spatial geometry but not velocity or acceleration profiles. A transition head that is explicitly trained to predict temporal dynamics (rather than only spatial latent distance) might address this. One approach would be to add a temporal consistency loss to the AC head training, penalising transitions that change velocity profile unexpectedly.

Symmetric semantic conditioning would allow a direct test of whether semantic alignment and object grounding improve world model guidance. The current benchmark excluded both modules to maintain fairness, but a future experiment with symmetric semantic inputs across both model families would measure their effect independently of the fairness constraint.

A pretrained LeWM variant using a standard ViT-Small or DINOv2 backbone would allow a more direct comparison between the pretrained and from-scratch strategies at comparable model capacity. The current LeWM result reflects both the from-scratch training and the smaller model scale; a pretrained variant at the same parameter count would isolate the effect of initialisation from the effect of capacity.

Wrist-only evaluation could be extended to include additional joint targets from EgoDex's 138-joint annotation. Including finger articulation or elbow configuration as additional evaluation targets would test whether the world model improvements in wrist trajectory extend to the full kinematic configuration, or whether wrist improvements mask degradation elsewhere.

Finally, the DAgger-style distillation pathway [6] offers a route to closing the loop between world model guidance and policy quality. World-model-accepted trajectories could be used as a teacher signal to retrain the ACT policy, progressively aligning the policy's own output distribution with the trajectories that the world model rates most highly. If the world model reliably identifies better trajectories, distillation should produce a policy that generates better candidates without requiring world model evaluation at every step.

References

[1] B. D. Argall, S. Chernova, M. Veloso, and B. Browning, "A survey of robot learning from demonstration," *Robotics and Autonomous Systems*, vol. 57, no. 5, pp. 469–483, 2009. DOI: 10.1016/j.robot.2008.10.024

[2] A. Y. Ng and S. J. Russell, "Algorithms for inverse reinforcement learning," in *Proceedings of the Seventeenth International Conference on Machine Learning*, Morgan Kaufmann, 2000, pp. 663–670.

[3] M. Jeannerod, "The timing of natural prehension movements," *Journal of Motor Behavior*, vol. 16, no. 3, pp. 235–254, 1984. DOI: 10.1080/00222895.1984.10735319

[4] M. Jeannerod, M. A. Arbib, G. Rizzolatti, and H. Sakata, "Grasping objects: The cortical mechanisms of visuomotor transformation," *Trends in Neurosciences*, vol. 18, no. 7, pp. 314–320, 1995. DOI: 10.1016/0166-2236(95)93921-J

[5] M. Santello, M. Flanders, and J. F. Soechting, "Postural hand synergies for tool use," *Journal of Neuroscience*, vol. 18, no. 23, pp. 10 105–10 115, 1998. DOI: 10.1523/JNEUROSCI.18-23-10105.1998

[6] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 15, 2011.

[7] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, *Learning fine-grained bimanual manipulation with low-cost hardware*, Apr. 2023. DOI: 10.48550/arXiv.2304.13705 arXiv: 2304.13705 [cs].

[8] Z. Ding, "Imitation learning," in *Deep Reinforcement Learning: Fundamentals, Research and Applications*, H. Dong, Z. Ding, and S. Zhang, Eds. Singapore: Springer Singapore, 2020, pp. 273–306, ISBN: 978-981-15-4095-0. DOI: 10.1007/978-981-15-4095-0_8 [Online]. Available: https://doi.org/10.1007/978-981-15-4095-0_8

[9] C. Chi et al., *Diffusion policy: Visuomotor policy learning via action diffusion*, Mar. 2024. DOI: 10.48550/arXiv.2303.04137 arXiv: 2303.04137 [cs].

[10] S. Jain et al., *DiWA: Diffusion policy adaptation with world models*, ArXiv preprint, Aug. 2025. arXiv: 2508.03645 [cs].

[11] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017, ISBN: 978-1-107-15630-2.

[12] Omitram, *Left-hand vs. right-hand coordinate systems: The 3d developer's guide - omitram — omitram.com*, https://omitram.com/3d-coordinate-systems-left-vs-right-hand/, 2024.

[13] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, "On the continuity of rotation representations in neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 5745–5753. DOI: 10.1109/CVPR.2019.00589

[14] N. Grossmann, E. Gröller, and M. Waldner, "Concept splatters: Exploration of latent spaces based on human interpretable concepts," *Computers & Graphics*, vol. 105, pp. 73–84, 2022, ISSN: 0097-8493. DOI: https://doi.org/10.1016/j.cag.2022.04.013 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0097849322000656

[15] A. Dawid and Y. LeCun, "Introduction to latent variable energy-based models: A path towards autonomous machine intelligence," *Journal of Statistical Mechanics: Theory and Experiment*, 2023. arXiv: 2306.02572 [cs].

[16] D. P. Kingma and M. Welling, *Auto-encoding variational bayes*, 2013. DOI: 10.48550/arXiv.1312.6114 arXiv: 1312.6114 [stat.ML].

[17] L. Li and R. Betti, "A machine learning-based data augmentation strategy for structural damage classification in civil infrastructure system," *Journal of Civil Structural Health Monitoring*, vol. 13, pp. 1–21, May 2023. DOI: 10.1007/s13349-023-00705-5

[18] AnotherDatum. "Variational autoencoders explained," Accessed: Apr. 24, 2026. [Online]. Available: https://anotherdatum.com/vae.html

[19] K. Sohn, H. Lee, and X. Yan, "Learning structured output representation using deep conditional generative models," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[20] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. DOI: 10.48550/arXiv.1706.03762 arXiv: 1706.03762 [cs.CL].

[21] S. Panda, *Decoding attention types and strategies for effective nlp models*, https://www.linkedin.com/pulse/decoding-attention-types-strategies-effective-nlp-models-swagat-panda-erhef/, 2024.

[22] M. Assran et al., "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. arXiv: 2301.08243 [cs].

[23] IBM Developer, *Models for machine learning*, https://developer.ibm.com/articles/cc-models-machine-learning/, 2017. Accessed: May 14, 2026.

[24] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, *Film: Visual reasoning with a general conditioning layer*, 2017. arXiv: 1709.07871 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1709.07871

[25] *MTDP: Modulated Transformer Diffusion Policy Model*, https://arxiv.org/html/2502.09029v1. Accessed: Apr. 26, 2026.

[26] M. Reuss, J. Pari, P. Agrawal, and R. Lioutikov, *Efficient diffusion transformer policies with mixture of expert denoisers for multitask learning*, 2024. arXiv: 2412.12953 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2412.12953

[27] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, *Mastering diverse domains through world models*, Nature, 2025, 2023. arXiv: 2301.04104 [cs].

[28] J. Quevedo, A. K. Sharma, Y. Sun, V. Suryavanshi, P. Liang, and S. Yang, *WorldGym: World model as an environment for policy evaluation*, Sep. 2025. DOI: 10.48550/arXiv.2506.00613 arXiv: 2506.00613 [cs].

[29] Q. Bu et al., *Closed-loop visuomotor control with generative expectation for robotic manipulation*, Oct. 2024. DOI: 10.48550/arXiv.2409.09016 arXiv: 2409.09016 [cs].

[30] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta, "R3M: A universal visual representation for robot manipulation," in *Conference on Robot Learning (CoRL)*, 2022. arXiv: 2203.12601 [cs].

[31] A. Brohan et al., *RT-1: Robotics transformer for real-world control at scale*, 2022. arXiv: 2212.06817 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2212.06817

[32] M. Assran et al., *V-JEPA 2: Self-supervised video models enable understanding, prediction and planning*, Jun. 2025. DOI: 10.48550/arXiv.2506.09985 arXiv: 2506.09985 [cs].

[33] N. Tsagkas, A. Sochopoulos, D. Danier, C. X. Lu, and O. Mac Aodha, *When pre-trained visual representations fall short: Limitations in visuo-motor robot learning*, 2025. arXiv: 2502.03270 [cs].

[34] R. G. Goswami et al., *World models can leverage human videos for dexterous manipulation*, Dec. 2025. DOI: 10.48550/arXiv.2512.13644 arXiv: 2512.13644 [cs].

[35] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero, *Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels*, Verified from local PDF metadata and first page, Mar. 2026. arXiv: 2603.19312 [cs.LG].

[36] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, *DINO-WM: World models on pre-trained visual features enable zero-shot planning*, 2024. arXiv: 2411.04983 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2411.04983

[37] W. Huang et al., *Pointworld: Scaling 3d world models for in-the-wild robotic manipulation*, 2026. arXiv: 2601.03782 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2601.03782

[38] N. Hansen, H. Su, and X. Wang, "TD-MPC2: Scalable, robust world models for continuous control," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv: 2310.16828 [cs].

[39] T. Yu et al., "MOPO: Model-based offline policy optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv: 2005.13239 [cs].

[40] R. Kidambi, A. Rajeswaran, P. Netrapalli, and T. Joachims, "MOReL: Model-based offline reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv: 2005.05951 [cs].

[41] A. Argenson and G. Dulac-Arnold, "Model-based offline planning," in *International Conference on Learning Representations (ICLR)*, 2021. arXiv: 2008.05556 [cs].

[42] Z. Liu, T. Swann, S. Garg, S. Levine, and A. Kumar, *Model-based offline planning with trajectory pruning*, 2021. arXiv: 2105.07351 [cs].

[43] X. Li et al., *Evaluating real-world robot manipulation policies in simulation*, CoRL 2024, 2024. arXiv: 2405.05941 [cs].

[44] Y. Li, Y. Zhu, J. Wen, C. Shen, and Y. Xu, *WorldEval: World model as real-world robot policies evaluator*, 2025. arXiv: 2505.19017 [cs].

[45] R. Hoque, P. Huang, D. J. Yoon, M. Sivapurapu, and J. Zhang, *EgoDex: Learning dexterous manipulation from large-scale egocentric video*, ICLR 2026, 2025. DOI: 10.48550/arXiv.2505.11709 arXiv: 2505.11709 [cs].

[46] University of Malta, *M.AI.ESTRO: Mixed reality AI-enhanced skill transfer and robotic optimisation*, https://www.um.edu.mt/newspoint/news/2025/05/maiestro-mixed-reality-ai-enhanced-skill-transfer-robotic-optimisation, University of Malta newspoint announcement, 2025.

[47] J. Song, C. Meng, and S. Ermon, *Denoising diffusion implicit models*, Oct. 2020. DOI: 10.48550/arXiv.2010.02502 arXiv: 2010.02502 [cs.LG].

[48] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[49] S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz, and S. Bengio, “Generating sentences from a continuous space,” *arXiv preprint arXiv:1511.06349*, 2016. arXiv: 1511.06349.

[50] H. Fu, C. Li, X. Liu, J. Gao, A. Celikyilmaz, and L. Carin, “Cyclical annealing schedule: A simple approach to mitigating KL vanishing,” *arXiv preprint arXiv:1903.10145*, 2019. arXiv: 1903.10145.

[51] R. Y. Rubinstein and D. P. Kroese, *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning*. Springer, 2004.

[52] T. Xiao, I. Radosavovic, T. Darrell, and J. Malik, *Masked visual pre-training for motor control*, 2022. arXiv: 2203.06173 [cs].

Appendix A Auxiliary Implemented Modules

This appendix describes the two auxiliary modules that are fully implemented in the WACT stack but were excluded from the final fair benchmark. The semantic alignment pipeline and the object grounding stage each produce useful artefacts and remain integrated with the broader system. Their absence from the main results chapters reflects a deliberate methodological choice rather than a finding about their utility or correctness.

A.1 Rationale for exclusion from the main benchmark

The canonical benchmark evaluates two world model families under a shared input contract: the same raw RGB observation frame and the same bimanual wrist state sequence are provided to both V-JEPA2-AC and LeWM. No additional conditioning is admitted on either side.

Semantic alignment artefacts (captions, tags, text embeddings) and object grounding artefacts (bounding boxes, segmentation masks) could in principle be incorporated into the V-JEPA2-AC scoring pipeline through the semantic extraction path. The LeWM architecture, however, has no equivalent semantic conditioning module. Including semantic or grounding conditioning for V-JEPA2-AC alone would change the nature of the comparison from a world model family comparison under equal information to a mixed comparison that conflates world model architecture with access to additional input channels. The two scientific questions are distinct, and the thesis addresses only the former.

The exclusion applies symmetrically across all five evaluated branch instances. Both the V-JEPA2-AC family and the LeWM family run without semantic or grounding inputs in the final benchmark. Future work under a symmetric design, in which both families receive equivalent semantic conditioning, could isolate the effect of this information channel independently of the model family comparison.

A.2 Semantic alignment pipeline

A.2.1 Purpose and outputs

The semantic alignment stage converts indexed EgoDex windows into structured semantic artefacts for downstream conditioning and inspection. For each window, the pipeline produces:

- A natural language caption describing the observable manipulation activity in the selected frames.
- Structured tags organised into categories, including the contact object, secondary objects, and observed actions.
- A canonical semantic text string constructed from the structured tags, suitable for embedding.
- A dense text embedding vector produced by a configurable embedding backend.

The primary downstream use is to provide a semantically meaningful conditioning vector that can be concatenated with or appended to the visual latent representations used by the world model scoring functions. In the current implementation, this conditioning path is available but is not activated in the canonical benchmark.

A.2.2 Visual language model selection

The VLM component was evaluated across several model and embedding combinations before the operational configuration was stabilised. The semantic model comparison table records these evaluations and their relative quality on caption coherence and tag precision. The operationally selected configuration uses `Qwen/Qwen3-VL-2B-Instruct` as the visual language model for caption and tag generation, with the `google/siglip-so400m-patch14-384` model as the default text embedding backend (`WACT_EMBED_BACKEND=siglip_text`).

The Qwen3-VL-2B-Instruct model was selected for practical reasons: it is compact enough to run on a single GPU alongside other pipeline stages, it produces grammatically coherent captions from single-frame inputs, and its structured tag output is consistent enough to support deterministic canonical text construction. The SigLIP embedding backend was selected because it provides a joint image-text embedding space that is directly compatible with the V-JEPA2-AC semantic scoring path, allowing future similarity computations between caption embeddings and visual latents without an additional projection layer.

OpenCLIP and VLM-text hidden-state backends are also implemented and configurable. The SigLIP-backed Qwen3-VL run represents the strongest row in the comparison table for the current stack configuration.

A.2.3 Critic and retry loop

A key engineering feature of the semantic pipeline is an integrated critic and retry loop. The naive approach of captioning once and trusting the output produces unreliable

results on EgoDex frames, which are first-person egocentric views with partially occluded hands, cluttered surfaces, and rapidly changing visual content. The critic module scores the resulting caption against the input frames using a configurable similarity measure. If the score falls below a threshold or plateaus across retries, the pipeline either retries the VLM call with adjusted prompting parameters or marks the row as `semantic_failure=true` and writes it with a failure flag.

This design means that the semantic stage produces a complete, aligned output for every window in the index, with explicit quality flags rather than silently dropping or corrupting rows on difficult windows. The failure-flagged rows remain usable for inspection and can be filtered before any downstream conditioning step.

The implemented retry logic is:

1. Score the caption against the sampled frame using the configured score mode.
2. If the score exceeds the acceptance threshold, commit the row.
3. If the score is below threshold, retry with a revised prompt up to the configured maximum number of retries.
4. If all retries fail or the score plateaus without improvement, commit the row with `semantic_failure=true`.

A.2.4 Multi-GPU sharding and output artefacts

The semantic pipeline supports multi-GPU sharded execution through a shard supervisor (`semantic_shard_supervisor.py`) and a deterministic shard merge step (`semantic_merge_shards.py`). This is necessary because captioning and embedding 7,607 test windows or tens of thousands of training windows at VLM inference speeds is infeasible on a single GPU within a reasonable run budget.

The canonical merged output layout is:

```
/runs/w-act/egodex/semantic/<split>/<model_id>/<build_id>/
    embeddings.npy
    window_ids.txt
    meta.jsonl
    tags.jsonl
    captions.jsonl
    manifest.json
    index.npz
```

During a multi-GPU run, intermediate shard directories (`shard_0/`, `shard_1/`, and so on) accumulate partial outputs. The merge step validates row counts before

producing the canonical root artefact set, ensuring that partial or interrupted shard outputs are not silently used as complete builds. The manifest records the input fingerprint, allowing downstream stages to detect cache invalidation if the source windows file is replaced.

A.2.5 Integration with the broader pipeline

The semantic stage is positioned between the data indexing stage and the object grounding stage. The OPE stage (Section A.3) reads semantic manifests to obtain the canonical window ordering and to construct object prompts from the semantic tags produced by the VLM. This tight coupling is intentional: it ensures that semantic and grounding artefacts are always aligned to the same window index, preventing silent join drift if the windows file is regenerated.

The entrypoint for the semantic stage is `src/wact/cli/semantic_build_index.py`, with orchestration handled by `scripts/semantic.sh` or the stage-runner wrapper `scripts/run_stage.sh semantic`. The notebook companion `notebooks/semantic_aligner.ipynb` provides interactive inspection of caption quality, tag distributions, and embedding nearest-neighbour retrievals.

A.3 Object grounding and mask generation

A.3.1 Scope and current implementation

The object grounding stage takes semantic artefacts from the previous stage and produces spatially grounded detection and segmentation outputs for each window. The current implementation provides two-dimensional bounding-box detection and box-conditioned mask generation. It does not implement canonical six degree-of-freedom pose estimation, FoundationPose-style geometry reconstruction, or object identity tracking across frames. The name “object pose estimation” in the codebase reflects the intended scope of a future extension rather than the current capability.

For each window, the implemented pipeline:

1. Reads the semantic row for the window from the upstream semantic build.
2. Samples one or more frames from the window according to the configured frame sampling strategy.
3. Constructs object prompts from the structured semantic tags.

4. Runs the configured detector to produce bounding boxes.
5. Runs the configured masker on each detected box to produce segmentation masks.
6. Writes aligned detection and mask artefacts alongside a progress checkpoint.

A.3.2 Prompt construction

Object prompts are constructed from semantic tags using a priority scheme implemented in `src/wact/ope/prompting.py`. The logic prioritises the `contact_object` tag, which the VLM assigns to the primary object being actively manipulated, and then appends deduplicated entries from the `objects` tag list. Generic labels such as "object", "thing", "tool", and "hand" are filtered before the prompt is passed to the detector, as these terms produce unreliable or overspecific detections in the grounding model. When the VLM returns no object tags, the prompt construction falls back to a task-derived hint derived from the EgoDex task label.

This is category-level prompting rather than instance-level prompting. The detector does not distinguish between two instances of the same object class, and the resulting masks identify regions containing the prompted category rather than a specific tracked object instance.

A.3.3 Detection and segmentation backends

The production backends, configured through `configs/stage_defaults.env` and accessible via the stage-runner, are:

- Detector: `IDEA-Research/grounding-dino-base` (Grounding DINO), a zero-shot open-vocabulary detector that accepts free-text prompts.
- Masker: `facebook/sam2-hiera-large` (Segment Anything Model 2), a promptable segmentation model that produces high-quality binary masks from bounding box inputs.

The combination of Grounding DINO for detection and SAM2 for box-conditioned segmentation is a well-established zero-shot grounding pipeline. It does not require object-specific training data from EgoDex and generalises across the 194 task categories in the dataset without fine-tuning.

The bare `scripts/ope.sh` script defaults to dummy backends for development and debugging. Production runs must use the stage-runner or explicitly override the backend flags to select the real Grounding DINO and SAM2 models.

A.3.4 Output artefacts and alignment

The canonical output layout is:

```
/runs/w-act/egodex/pose/<split>/<pipeline_id>/<build_id>/
    masks.jsonl
    detections.jsonl
    window_ids.txt
    errors.jsonl
    progress.json
    manifest.json
```

The `masks.jsonl` file is the primary downstream artefact: it contains a row per window with the detected object categories, bounding box coordinates, and run-length-encoded or polygon-format segmentation masks. The `detections.jsonl` file records the raw detection outputs before masking and is retained for audit purposes. The `progress.json` file supports resumable execution through the shard supervisor.

Alignment between the OPE output and upstream artefacts is enforced by inheriting the `window_ids.txt` ordering from the semantic build. The OPE stage reads the semantic manifest to resolve the windows pickle that was used to produce the semantic index, ensuring that detection and mask rows correspond to the same window boundaries used at every earlier pipeline stage.

A.3.5 Known limitations

The current implementation has several limitations that are relevant to any future use of OPE outputs for manipulation conditioning:

1. The 2D masks identify spatial regions in the image frame but do not provide three-dimensional geometry information. Object depth, relative pose, or spatial relationship between manipulated objects are not recoverable from the mask output alone.

2. Small or visually similar objects can confuse the Grounding DINO detector, particularly in the cluttered egocentric field of view typical of EgoDex demonstrations. When two similar objects are present, the detector may merge them or select the wrong instance.

3. There is no temporal tracking across frames within a window. Each sampled frame is processed independently, and there is no mechanism to maintain object identity across the prediction horizon.

4. The zero-shot grounding approach depends on the quality of the upstream semantic tag output. A missed or misidentified contact object in the semantic stage will propagate to an incorrect or absent detection in the OPE stage.

These limitations are consistent with the current scope of two-dimensional grounding and mask generation. They would need to be addressed before OPE outputs could serve as reliable conditioning for world model scoring in a deployed system.

A.4 Future use under a symmetric design

The exclusion of semantics and object grounding from the final benchmark is a fairness decision, not a statement that these modules lack value. Both pipelines are implemented, tested on the training split, and produce aligned artefacts that are stored alongside the canonical benchmark outputs.

The most natural future experiment would introduce semantic conditioning symmetrically across both world model families. This would require either adapting the LeWM architecture to accept a text embedding as an additional conditioning channel, or training a new LeWM variant with semantic inputs from the outset. Under a symmetric design, the contribution of semantic conditioning to world model scoring quality could be measured independently of the model family comparison, and the effect of caption quality, embedding backend selection, and object grounding granularity could each be evaluated in isolation.

Similarly, the object grounding masks could be used to produce spatially localised visual features by masking the RGB input before the visual backbone processes it. Whether this localisation improves world model guidance in the manipulation setting, or whether the pretrained backbone's attention mechanism already implicitly grounds relevant objects without explicit masking, is an open empirical question.

[FIGURE PLACEHOLDER: semantic pipeline overview — input window to VLM to caption plus tags to embedding backend to output artefacts; critic/retry loop shown as a feedback arc between caption and score threshold]

[FIGURE PLACEHOLDER: object grounding example — sampled EgoDex frame with Grounding DINO bounding box and SAM2 mask overlaid on detected contact object; prompt text shown]

Appendix B Benchmark Evolution and Validity Record

This appendix documents the development trajectory of the WACT benchmark and the validity concerns that were identified and resolved before the final canonical results package was established. Its purpose is to provide an auditable record of the decisions and corrections that shaped the final evaluation surface, and to explain why certain earlier result sets were superseded or rejected.

The record is presented in roughly chronological order of discovery. Each section identifies the concern, describes its effect on the results as reported at the time, and explains the correction applied.

B.1 The three benchmark eras

The dissertation results passed through three distinct evaluation eras before the canonical package was established. Understanding the transition between eras is necessary for interpreting references to earlier result numbers in project documentation.

Era 1: Bounded pilot slices. The earliest quantitative comparisons were conducted on small, handpicked windows selected to represent the diversity of EgoDex task categories. These were useful for rapid controller iteration and for identifying which architectural choices were clearly harmful, but they could not support generalisable claims because the selection was not held-out from the development process. All Era 1 results were explicitly demoted from the thesis claims surface once full held-out evaluation became practical.

Era 2: March 2026 frozen three-branch bundle with limited LeWM coverage. The first evaluation to use the held-out test split was the March 2026 frozen package. This package provided valid Branch 1 and Branch 2 V-JEPA2-AC results, but its LeWM coverage was severely limited by a frame cap configuration error (described in Section B.2). The Branch 3 selection mode error (described in Section B.3) also meant that Branch 3 V-JEPA2-AC was not independently evaluated. Both defects required correction before the three-branch comparison could be treated as scientifically valid.

Era 3: Canonical final package (April 2026). The April 14, 2026 package achieved full 7,607-window coverage for the LeWM family after the frame cap fix. A further correction on April 18, 2026 re-ran Branch 3 V-JEPA2-AC with the correct `selection_mode=world_model_only` flag, producing the first genuine Branch 3 evaluation. The `train_part1_2_3` training split was used for all five branch instances. This package is the canonical results surface for all quantitative claims in the thesis.

B.2 LeWM frame cap defect

B.2.1 Discovery

The LeWM export pipeline requires a `max_frames_per_episode` parameter that controls how many frames from each EgoDex episode are included when building the training corpus. This parameter has a direct impact on the coverage of the resulting evaluation surface, because the guard can only evaluate windows whose constituent frames were included in the export.

During the March 2026 benchmark packaging, the frame cap was set to `max_frames_per_episode=256`. This value was not derived from the actual window length distribution of the benchmark window set; it was a development default carried over from early pipeline testing. When the benchmark results were inspected, the LeWM guard had produced predictions for only 160 of the 7,607 test windows, representing 2.1 per cent of the intended evaluation surface.

The coverage failure was identified by comparing the count of LeWM guard output rows against the expected window count. The root cause was confirmed by recomputing the maximum frame index required to cover all windows in the benchmark window list and finding that the actual maximum was 5,280 frames, far above the 256-frame cap in use.

B.2.2 Effect on reported results

Any LeWM result reported from the March 2026 package is based on 160 windows, not 7,607. The 160 covered windows are not a random sample of the test split; they are the windows whose episodes happened to fall within the 256-frame cap. These windows are concentrated in episodes with early-episode actions and do not represent the full range of task categories or annotation confidence levels in the test split.

The practical effect is that the March 2026 LeWM numbers cannot be compared directly to Branch 1 or Branch 2 V-JEPA2-AC numbers, because the LeWM evaluation is on a different, non-representative slice of the test data. Any conclusion drawn from these numbers about the relative merit of the LeWM family would be confounded by the coverage difference.

B.2.3 Correction

The frame cap was recomputed by iterating over the benchmark window list, reading the episode frame indices for each window, and taking the maximum observed frame index across all windows. The resulting cap of 5,280 frames was applied to the export

rebuild. The LeWM model was not retrained; only the export was rebuilt with the corrected parameter. The guard was then re-run, achieving full 7,607-window coverage on the April 14, 2026 package.

The corrected LeWM numbers in the canonical package are therefore not comparable to the March 2026 LeWM numbers on a like-for-like basis. The March 2026 numbers are retained in project documentation for audit purposes but are not cited in any thesis claims.

B.3 Branch 3 selection mode defect

B.3.1 Discovery

The three-branch design requires Branch 3 to use `selection_mode=world_model_only`, meaning that only the world model's predicted latent distance ranks the candidates, without any contribution from the ACT policy's own scoring function. This is the experimental condition that tests whether the world model alone, without policy guidance in the selection step, can identify better trajectories.

After the April 14, 2026 package was assembled, the per-window pairwise comparison between Branch 3 V-JEPA2-AC and Branch 2 V-JEPA2-AC was computed as a validation check. The result was a 0.0 per cent win rate for Branch 3 against Branch 2 across all 7,607 windows. That is, Branch 3 did not produce a single window outcome that differed from Branch 2.

This was immediately identified as inconsistent with any genuine difference in selection logic between the two branches. Inspection of the Branch 3 guard manifest revealed that the selection mode was recorded as `best_score`, not `world_model_only`. The chain evaluation script (`chain_eval_part1_2_3.sh`) had been written to launch both Branch 2 and Branch 3 guards, but the Branch 3 launch did not pass the `--selection-mode world_model_only` flag. Both guards therefore ran with `selection_mode=best_score` on the same frozen candidate pool, producing identical outputs.

B.3.2 Effect on reported results

Until the defect was corrected, the "Branch 3 V-JEPA2-AC" entry in the April 14 package was scientifically identical to Branch 2 V-JEPA2-AC. The three-branch comparison was effectively a two-branch comparison. Any conclusion about whether world-model-only selection outperforms hybrid selection under V-JEPA2-AC could not be drawn from this data.

The defect was non-obvious from the output metrics alone. The reported Branch 3 translation error (0.0354, identical to Branch 2) was not implausible; one might expect Branch 3 and Branch 2 to produce similar numbers if the world model's ranking correlates with the combined score's ranking. The defect was only detectable through pairwise comparison, which revealed the zero win rate that is impossible if the two branches genuinely differ.

B.3.3 Correction

The Branch 3 V-JEPA2-AC guard was re-run on April 18, 2026 with the correct flag:

```
--selection-mode world_model_only
--candidate-source frozen_manifest
--candidate-manifest-run-dir <B2_JEPA_run_dir>
```

Using the frozen candidate pool from Branch 2 ensured that the Branch 3 result reflects only the difference in selection logic, not any difference in the candidates available. Guard manifest verification after the run confirmed that all 7,607 windows were decided by the world model (`override_reason_counts: {world_model_only_best_non_act: 7607}`), confirming that the corrected selection mode was in effect for every window.

The corrected Branch 3 V-JEPA2-AC result (translation $L_2 = 0.0328$, 91.5 per cent win rate against Branch 1) is the first genuine evaluation of world-model-only selection in the WACT stack.

B.4 Oracle goal leakage

B.4.1 Discovery

In early V-JEPA2-AC guard configurations, the world model scoring function was provided with the terminal latent state of the evaluation episode as a goal signal. The scoring function measured the predicted final latent distance to this goal and used it as part of the candidate ranking. This goal signal was derived from the ground-truth future frames of the demonstration, information that would not be available in a deployed system.

The oracle goal leak was identified during the fair comparison audit when it was noted that the V-JEPA2-AC guard configuration differed from the ACT-only baseline in the information it received at evaluation time. The ACT policy generates candidates from the current observation alone and has no access to future frames. Providing the world model with the ground-truth terminal latent created an asymmetry: the world

model scoring function was effectively guided toward candidates that reached the demonstrated endpoint, while the ACT policy's own ranking had no equivalent oracle signal.

B.4.2 Effect on reported results

Any result from a guard run using oracle goal conditioning overstates the benefit of world model guidance, because part of the improvement is attributable to the oracle signal rather than to the world model's intrinsic representation quality. The magnitude of the overstatement cannot be precisely determined from the available data, but it is substantial enough to make oracle-conditioned results non-comparable to policy-only results.

Early exploration of V-JEPA2-AC guidance showed results that were more impressive than the final canonical numbers. The oracle goal was a contributing factor. Removing the oracle signal and rerunning under the fair contract produced lower but more defensible numbers. The final canonical results (77.0 per cent reduction in mean translation error) are therefore conservative relative to the earliest reported V-JEPA2-AC results.

B.4.3 Correction

The fair evaluation contract was established by setting `goal_source=none` for all branch evaluations. Under this setting, the world model scoring function does not receive any goal latent derived from future frames. The candidate ranking is based solely on the world model's predicted latent dynamics given the observed context and the proposed action chunk.

This correction was applied to all five branch instances in the canonical benchmark. The change from oracle goal to no goal required validating that the world model scoring function still produced meaningful candidate rankings without the terminal goal signal, which was confirmed by the final translation improvement results.

B.5 Goal metric invalidity

B.5.1 Discovery

The V-JEPA2-AC guard natively reports a metric called `goal_l2` alongside the translation error. In the original implementation, this metric stored different quantities in Branch 1 and Branches 2 and 3. In Branch 1, it stored the translation L_2 error for the

selected candidate. In Branches 2 and 3, it stored the JEPA latent-space distance between the predicted final state and the goal latent.

When `goal_source=none` was adopted as the fair evaluation contract, there was no longer a meaningful goal latent for Branches 2 and 3. The `goal_l2` metric for these branches consequently reported a latent distance of zero for every window, making the metric uninformative. Comparing `goal_l2` across branches was therefore comparing a real translation error in Branch 1 against a vacuous zero in Branches 2 and 3.

B.5.2 Correction

The `goal_l2` metric was replaced with `endpoint_translation_l2`, defined as the mean translation L_2 error computed over only the final quarter of the prediction window (prediction steps 12 through 16). This metric measures how accurately the predicted trajectory arrives at the correct final position, which is often more relevant to task success than early-window prediction accuracy. The endpoint metric is computed identically across all branch instances and is not affected by the presence or absence of a goal latent. It is included in the canonical benchmark shape metric tables reported in the results chapter.

B.6 LeWM CPU inference defect

B.6.1 Discovery

During the first full test-split guard run for LeWM after the frame cap fix, the estimated completion time for 7,607 windows was approximately 54 hours. This was implausibly slow relative to the expected GPU inference throughput for a 15-million-parameter model. Inspection of the guard inference loop revealed a hard-coded `.to("cpu")` call in the LeWM model forward pass that overrode any device specification passed at runtime. The model was running on CPU regardless of whether a CUDA device was available.

B.6.2 Correction

The hard-coded device assignment was replaced with a dynamic device detection that checks for CUDA availability and falls back to CPU only if no GPU is detected. After this correction, the LeWM guard ran at approximately 19.5 windows per second on a single GPU, completing the full 7,607-window test evaluation in roughly 6.5 minutes. The effective speedup was approximately 130-fold relative to the CPU-bound run.

This defect is worth recording because it illustrates a failure mode common in research code developed iteratively: a debugging convenience (forcing CPU to avoid CUDA memory errors during development) was never removed before production use. The correction required no changes to the model architecture or training, only to the device placement logic in the inference wrapper.

B.7 HDF5 handle management defect

B.7.1 Discovery

The data loading loop in an early version of the guard infrastructure opened and closed the HDF5 data file once per evaluation window. On a test set of 7,607 windows, this produced approximately 145,000 open-close cycles (one open and one close per window, plus frame-level re-opens within each window). The overhead from repeated file handle acquisition was a significant fraction of total evaluation wall time on large test splits.

B.7.2 Correction

The data loading implementation was refactored to open the HDF5 file once at the start of the evaluation run, hold the handle in memory, and close it only when the evaluation completes. This change reduced per-window I/O overhead to a small seek operation rather than a full open-close cycle and improved guard evaluation throughput noticeably on the full test split.

B.8 Guard resumability

B.8.1 Motivation

Full test-split guard runs covering 7,607 windows take several minutes on a GPU but can be interrupted by node preemption, out-of-memory errors, or external compute scheduler events. Without checkpointing, an interrupted run required restarting from window zero, discarding all partial results.

B.8.2 Implementation

A per-window JSONL checkpointing system was added to the guard infrastructure. After each window is evaluated, a row is appended to a checkpoint file in the output

directory. When a run is invoked with `--resume-dir` pointing to an existing output directory, the guard reads the checkpoint file, identifies which window IDs have already been completed, and skips them. The final manifest and aggregate metrics are computed from the complete set of per-window rows, including both the resumed and newly computed windows.

This design ensures that an interrupted run can be resumed without any manual intervention and without requiring the window evaluation order to be deterministic. The checkpoint is written atomically per row, so a crash mid-window produces at worst one incomplete row that the resume logic can detect and skip.

B.9 Rejected comparison inputs and superseded paths

B.9.1 Semantics and OPE as benchmark inputs

An early hypothesis held that incorporating semantic alignment and object grounding into the V-JEPA2-AC scoring function would produce the most informative world model guidance, and that this enriched V-JEPA2-AC configuration should be the primary benchmark representative for the V-JEPA2-AC family. This hypothesis was rejected for the reasons described in Appendix A and in Section 8.6 of the discussion chapter: the enriched configuration gives V-JEPA2-AC privileged inputs not available to LeWM, confounding the family comparison.

The decision to reject this path was not based on the performance of the enriched configuration. The lean RGB-plus-trajectory contract was adopted because it is the only contract that supports a scientifically unconfounded comparison between the two world model families.

B.9.2 Veto and threshold selectors

Before the current scoring function was stabilised, several candidate selection mechanisms based on explicit threshold vetos were implemented and evaluated. These included the `act_first_veto` design, which rejected world model candidates whose support distance exceeded a fixed threshold, and the `veto_m10` selector, which applied a margin-based veto to prevent the world model from selecting candidates that were far from the ACT policy's preferred candidate even if they scored well on the world model metric alone.

These mechanisms were useful for early exploration of the policy-guard interface and helped establish that naive world model selection could be dominated by retrieval artefacts when the candidate pool contained low-quality outliers. They were superseded by the structured scoring function in Branch 2 (`best_score`: combined

latent distance, action out-of-distribution penalty, and support distance term) and the clean world-model-only design in Branch 3. Neither the veto selectors nor the margin-based threshold approaches are used in the canonical benchmark.

B.9.3 Early Branch 3 shells

Several experimental Branch 3 designs preceded the final frozen representative. The first explicit world-model-only shell for LeWM was evaluated on the fair held-out slice and was found to be weaker than the ACT-only baseline on aggregate slice metrics, showing that world-model-only selection requires a world model of sufficient quality before it improves over the policy alone. Neutral-seed variants of Branch 3, which removed the ACT seed from the candidate pool, also collapsed on the fair slice, demonstrating that the bottleneck was the controller structure rather than any single parameter choice. These negative results were important for understanding the design space and for eventually arriving at a Branch 3 configuration that genuinely improves over Branch 2 in aggregate.

B.9.4 Conservative confidence gating

A further rejected path explored whether applying more conservative confidence gating to the LeWM guard, requiring higher override margins before the world model was permitted to override the policy candidate, could make LeWM competitive with V-JEPA2-AC on the shared benchmark. The bounded sweep showed that increasing the override margin made LeWM more conservative but did not produce a competitive replacement for V-JEPA2-AC on chunk-level comparison metrics. This was recorded as a useful negative result: the performance gap between the two world model families is not primarily explained by calibration of the override threshold, and therefore cannot be closed by threshold tuning alone.

B.10 Summary of validity status

Table B.1 summarises the seven validity concerns identified during the benchmark development cycle and the resolution status of each at the time the canonical package was assembled.

Concerns 1, 3, and 6 were resolved by changes to the benchmark artefacts or analysis code. Concern 2 is addressed by consistent framing throughout the thesis: translation L_2 is positioned as an offline diagnostic proxy rather than a task success predictor, and the connection to deployed outcomes is explicitly left as future work. Concern 4 is partially addressed by the per-task win rate distribution reported in the

Table B.1 Summary of benchmark validity concerns and their resolution status in the canonical final package.

#	Concern	Severity	Status in canonical package
1	Branch 3 V-JEPA2-AC identity defect (`selection_mode` bug)	Critical	Resolved: re-run April 18, 2026
2	`translation_l2` not paper-backed as primary metric	Moderate	Documented: framed as offline diagnostic proxy
3	Shape metrics suppressed from reported summary	Moderate	Resolved: shape metrics added to results chapter
4	No per-task win rate breakdown	Moderate	Partially resolved: aggregate confirmed broad; worst-5 reported
5	Wrist-only evaluation scope	Low–Moderate	Acknowledged design constraint; documented in limitations
6	Confidence scores unused in evaluation	Low	Resolved: stratification analysis added to results
7	Offline-to-online gap	Moderate	Acknowledged: explicitly bounded in scope section

results chapter, which confirms that improvement is broad and not driven by a small number of tasks. Concerns 5 and 7 are acknowledged design constraints that are transparent in the scope of conclusions and limitations sections.

The final canonical results are drawn from a benchmark in which all critical and moderate concerns have been either resolved or explicitly bounded. No result cited in the thesis claims chapter relies on a defective or uncorrected evaluation artefact.

Appendix C Notes for Future Revision

This appendix collects scope notes identifying parts of the thesis that would require revision if certain extensions were carried out after submission. They are gathered here to keep the main chapters clean while preserving a complete record for any follow-on work.

C.1 Strategy 2 comparative evaluation

All benchmark results in this thesis use the fixed-stride windowing contract (Strategy 1, `obs16_pred16_s128`) exclusively. The state-change-targeted alternative (Strategy 2, Section 4.1.2) is fully implemented and has been used for qualitative development inspection, but a systematic Strategy 1 vs. Strategy 2 comparison has not been conducted and is noted as a natural future direction.

If a full Strategy 2 evaluation were completed and the two contracts compared as co-primary evaluation designs, the following sections would require revision:

- **Section 4.1.2** (index construction): restore the Strategy 1 / Strategy 2 bold-paragraph pattern. Add a paragraph describing the state-change-targeted index in full (peak detection, task-stratified sampling, one window per episode with a fixed seed, no temporal overlap). Clarify that both strategies enforce the shared-window invariant independently.
- **Section 4.1.3** (training splits): extend Table 4.1 to include Strategy 2 window counts per split and task; update the test-index description to distinguish the two contracts.
- **Section 4.6** (evaluation metrics): add explicit strategy labels to all result tables and figures; add a dedicated analysis subsection for cross-strategy comparison if results differ materially.
- **Results chapter**: all quantitative results would need split-by-strategy labelling to make clear which sampling contract each row corresponds to.